Introduction to Machine Learning

Mihir Rao

Contents

1 Introduction and ML Problem Framing **6**
1.1 Machine learning, AI, and neighboring fields . . . 6
1.2 Which problems are machine learning problems . . . 6
1.3 Learning settings . . . 7
1.4 A first classifier: the taste test . . . 8
1.5 Training, testing, and memorization . . . 9
1.6 A brief history . . . 10
1.7 The ML lifecycle . . . 10

2 Data Tools **11**
2.1 pandas data structures . . . 11
2.2 Exploring a DataFrame . . . 12
2.3 Selecting data . . . 12
2.4 Filtering with Boolean arrays . . . 13
2.5 Modifying a DataFrame . . . 13
2.6 Aggregation and grouping . . . 14
2.7 Joining DataFrames . . . 14
2.8 Visualization . . . 15

3 Machine Learning Mechanics: Terminology and Techniques **16**
3.1 Framing the learning problem . . . 16
3.2 The taxonomy of learning problems . . . 16
3.3 Evaluating a classifier . . . 17
3.4 Generalization and data splits . . . 17
3.5 Feature engineering . . . 18
3.6 Model design . . . 19
3.7 Optimization and hyperparameters . . . 21
3.8 Prediction and evaluation . . . 22

4 Clustering, Probability Review **23**
4.1 Unsupervised learning . . . 23
4.2 Clustering . . . 23
4.3 K-means . . . 24
4.4 Distances, metrics, and similarities . . . 26
4.5 Choosing K, applications, and weaknesses . . . 26
4.6 Probability review . . . 27
4.7 Bayes' theorem: the wake-word detector . . . 29

5 Introduction to Maximum Likelihood Estimation **30**
5.1 The abstract learning recipe . . . 30
5.2 Probability distributions . . . 31
5.3 Maximum likelihood estimation . . . 31

5.4 MLE for a univariate Gaussian . . . 32
5.5 MLE gives a point estimate . . . 33
5.6 MLE for the multinomial distribution . . . 33

6 Multivariate Gaussians and Mixtures of Gaussians **35**
6.1 The multivariate Gaussian . . . 35
6.2 From independent coordinates to the general case . . . 35
6.3 Expectation, variance, and covariance . . . 36
6.4 The covariance matrix and independence . . . 36
6.5 Level sets of the density are ellipses . . . 37
6.6 Gaussian mixture models . . . 37
6.7 *k*-means versus mixtures of Gaussians . . . 39

7 Mixtures of Gaussians and Linear Regression **40**
7.1 Fitting a mixture by gradient ascent . . . 40
7.2 Regression . . . 40
7.3 Basis expansions . . . 41
7.4 Gaussian linear regression and maximum likelihood . . . 42
7.5 Solving least squares with vector calculus . . . 43
7.6 The geometric view . . . 44
7.7 What can go wrong . . . 45

8 Linear Regression **46**
8.1 When the normal equations fail . . . 46
8.2 Infinitely many maximum likelihood solutions . . . 46
8.3 Ridge regression . . . 47
8.4 Bayesian modeling and MAP estimation . . . 47
8.5 Choosing the regularization strength . . . 49
8.6 Sparsity and the lasso . . . 49

9 Linear Regression and Regularization **51**
9.1 The singular value decomposition . . . 51
9.2 The Moore–Penrose pseudo-inverse . . . 52

10 Linear Regression and Regularization (continued) **54**
10.1 Regularization and generalization . . . 54
10.2 Ridge regression and numerical stability . . . 54
10.3 Model selection and held-out data . . . 55
10.4 Regression metrics . . . 55
10.5 Norms, the lasso, and the elastic net . . . 56
10.6 From regression to classification . . . 56
10.7 Discriminative versus generative classifiers . . . 57

11 Classification **59**
11.1 Gaussian discriminant analysis . . . 59
11.2 Fitting GDA by maximum likelihood . . . 59
11.3 Decision boundaries . . . 60
11.4 LDA and logistic regression . . . 62
11.5 Discrete features and naive Bayes . . . 63
11.6 Kinds of classification problems . . . 64

12 Logistic Regression and Classifier Accuracy **65**
12.1 From generative posteriors to the sigmoid . . . 65
12.2 Logistic regression . . . 65
12.3 Multi-class logistic regression . . . 66

12.4 Maximum likelihood for logistic regression . . . 67
12.5 Choosing between classifiers . . . 68
12.6 Confusion matrices and rates . . . 68
12.7 ROC curves . . . 70
12.8 Class balance, precision, and recall . . . 71
12.9 Metrics as losses and cost-sensitive learning . . . 71

13 Convergence, Momentum, Adam, and Stochastic Gradient Descent **73**
13.1 Gradient descent . . . 73
13.2 Quadratic approximation and the Hessian . . . 74
13.3 Convergence on the quadratic approximation . . . 75
13.4 Momentum . . . 76
13.5 Learning-rate schedules . . . 76
13.6 Adaptive learning rates . . . 77
13.7 Stochastic gradient descent . . . 78

14 MLE, MAP, and the Bias-Variance Trade-off **81**
14.1 MLE versus MAP: the bias of a coin . . . 81
14.2 A Gaussian prior on the weights . . . 81
14.3 Bias, noise, and model variance . . . 82
14.4 The bias-variance decomposition . . . 83
14.5 Regularization trades variance for bias . . . 83
14.6 Application: ranking language models . . . 84

15 Learning with Gradient Descent **86**
15.1 A worked example . . . 86
15.2 The rest of the toolkit . . . 87

16 Entropy, Information, and Logistic Regression **88**
16.1 Entropy and compression . . . 88
16.2 Why entropy: typical sequences . . . 89
16.3 KL divergence and cross entropy . . . 90
16.4 Logistic regression and the cross-entropy loss . . . 90
16.5 Multiclass logistic regression . . . 91
16.6 The entropy of English . . . 92

17 Neural Networks: Depth, Universal Approximation, and Activation Functions **93**
17.1 Neurons . . . 93
17.2 Feed-forward neural networks . . . 94
17.3 Universal approximation . . . 95
17.4 Activation functions . . . 95
17.5 Learning features: the XOR example . . . 96
17.6 Building and training networks in PyTorch . . . 98

18 Neural Networks and Backpropagation **99**
18.1 A simple example by the chain rule . . . 99
18.2 Computation graphs and backpropagation . . . 100
18.3 Message passing and cost . . . 102
18.4 Activation functions revisited . . . 102
18.5 Alternatives to backpropagation . . . 102
18.6 Why do deep networks work so well? . . . 103

19 Backpropagation (continued) and Convolutional Neural Networks **104**
19.1 From backpropagation to automatic differentiation . . . 104
19.2 Initialization . . . 104

19.3 Batch normalization and input normalization . . . 104
19.4 Why convolutional networks . . . 105
19.5 Convolution . . . 106
19.6 Size and cost of a convolutional layer . . . 108
19.7 Pooling and the full architecture . . . 108

20 Convolutional Neural Networks (continued) 110
20.1 Finishing convolutional networks . . . 110
20.2 Training convolutional networks . . . 110
20.3 Early stopping . . . 111
20.4 Dropout . . . 111
20.5 Double descent . . . 111

21 Transformers 113
21.1 Deep networks as feature transformations . . . 113
21.2 Convolutional networks and their receptive field . . . 113
21.3 Representing documents as vectors . . . 114
21.4 Recurrent neural networks . . . 115
21.5 Soft and hard attention . . . 115

22 Transformers (continued) 117
22.1 Content-based attention . . . 117
22.2 Self-attention . . . 117
22.3 Attention as a soft lookup table . . . 119
22.4 Multi-head attention . . . 120
22.5 The transformer layer . . . 120
22.6 Token order and positional encoding . . . 121
22.7 The encoder transformer . . . 123

23 LLM Training and Applications 124
23.1 Transformers for classification and self-supervision . . . 124
23.2 Causal language modeling . . . 124
23.3 A modern LLM . . . 126
23.4 Post-training . . . 126
23.5 Prompting and test-time compute . . . 128

24 Self-Supervised Learning 129
24.1 Self-supervision and transfer learning . . . 129
24.2 Generative pretext tasks . . . 130
24.3 Discriminative pretext tasks . . . 131
24.4 Contrastive representation learning . . . 132

25 AI for Protein Engineering 134
25.1 Structure prediction . . . 134
25.2 Design and why structure prediction does not solve it . . . 135
25.3 Trends: everything is density estimation . . . 135
25.4 In design we are the adversary . . . 136
25.5 Conditioning a generative model for design . . . 136
25.6 Guidance on discrete state spaces . . . 137
25.7 Designing a library instead of a sequence . . . 137
25.8 Fitness landscapes and epistasis . . . 138

27 LLMs, Agents, and Environments 140
27.1 From an LLM to an agent . . . 140
27.2 How agents are built, and the long-horizon problem . . . 141

27.3 Evaluation: from questions to environments 141
27.4 Using environments to improve an agent 143
27.5 GEPA 143

Lecture 1: Introduction and ML Problem Framing

Machine learning (ML) studies software systems that improve, or *learn*, through data. The basic recipe has two phases: training uses examples (data) to fit a model, and inference uses the fitted model to make predictions or decisions. Fitting a line to a scatter plot of price against a score and reading off the price of a new score is already machine learning.

1.1 Machine learning, AI, and neighboring fields

Two classic examples show when learning is needed. Spam is difficult to define, and a face detector is difficult to program, yet both are easy to demonstrate with examples. Artificial intelligence (AI) is the capability of computational systems to perform tasks typically associated with human intelligence, such as learning, reasoning, problem solving, perception, and decision making. AI is the goal; ML is the method, and the primary technology behind modern AI. The terms are often used interchangeably.

Several overlapping labels describe the landscape.

- *Statistics* is the science of learning from data, with its own research venues (Annals of Statistics, JASA, JRSS).
- *Data science* is a broad, modern term with more software engineering: Python pipelines, distributed analytics on very large datasets (Spark, MapReduce), data collection and cleaning, and connections to databases and web front ends. It contains parts of ML and AI but is not a defined research community.
- *Machine learning* is the more mathematical, modeling part: supervised and predictive modeling, unsupervised learning such as clustering, and text and image understanding. Its main venues are NeurIPS, ICML, and ICLR, with KDD on the applied side.
- *Artificial intelligence* contains ML as a subdiscipline and also covers perception, translation, game playing, logic-based symbolic AI, reasoning, planning, and robotics (venues also include AAAI and IJCAI).
- *Data mining* is an applied version of ML focused on large-scale software and performance, intersecting databases (KDD, SDM); *big data* refers to scaling analytics to very large datasets.

1.2 Which problems are machine learning problems

Problems fall into three kinds.

- An ENGINEERING PROBLEM can be solved by a direct, specifiable algorithm or a set of hand-written rules.
- A MACHINE LEARNING PROBLEM is one whose solution is easy to demonstrate or evaluate but difficult to implement directly.
- A HUMAN PROBLEM cannot be well specified, or requires human judgment.

Real systems often combine engineering, ML, and humans. In a machine learning solution the system learns the desired behavior (a prediction, a representation, or a policy) from demonstration or experience. Detecting spam text messages is a machine learning problem: spam depends on the receiver and is hard to define, but examples are easy to collect. Holding a productive conversation is another: the rule-based ELIZA (1966) shows how hard it is to program directly, while good conversations are easy to demonstrate and to judge.

Formally, ML is LEARNED FUNCTION APPROXIMATION. A model is a function h_w from an input X (for example a text message) to an output Y (spam, 1, or not, 0), with MODEL PARAMETERS w. Learning is the process of choosing w from data or from interaction with the world.

1.3 Learning settings

The three settings differ in what is observed.

Setting	Observed	Learns from
SUPERVISED	pairs $\{(X, Y)\}$	demonstration
UNSUPERVISED	inputs $\{X\}$ only	structure in the data
REINFORCEMENT	X and a reward function	reward

Supervised learning learns a relationship between observed pairs. When Y is a category the task is CLASSIFICATION: BINARY with two classes, MULTI-CLASS with more. When Y is continuous it is REGRESSION. Examples include image labeling (X an image, Y a class), next-word prediction (X a prompt, Y the next word), stock prediction (X the history, Y the next value), and image generation (X a prompt and noise, Y pixel values). Supervised problems arise with tabular data (house prices from square footage, bedrooms, and ZIP code; loan default; repeat purchases; re-hospitalization), images (where the features are pixels, for example steel defect detection or dogs versus cats), time series (seizure prediction from EEG, forecasting, speech recognition), and natural language (sentiment, article topics, toxic content).

Unsupervised learning models data without explicit labels. Its tasks include CLUSTERING (assigning similar points to the same group; for example k-means or DBSCAN), DIMENSIONALITY REDUCTION (finding a lower-dimensional representation that keeps the important directions; for example PCA and its sparse and nonnegative variants, t-SNE, and learned deep representations), association

analysis, anomaly detection, and generation of artificial data that looks natural. These are used for visualization and as steps within other ML tasks.

Reinforcement learning learns from reward signals, often over long sequences of discrete or continuous actions: moves in a game with reward win or lose, joint-angle changes of a robot, or generated tokens rewarded by answer quality.

1.4 A first classifier: the taste test

A focus group rates beverages, and we want to predict whether a new beverage will be rated as having great taste. Each row of the training data is a TRAINING SAMPLE. Acidity A and sweetness S are the FEATURES (in statistics also *covariates*, *factors*, or *regressors*), and y is the TARGET.

	Acidity A	Sweetness S	y (great taste)
Bev1	0.8	0.8	1
Bev2	0.3	0.25	0
Bev3	0.2	0.8	0
Bev4	0.3	0.7	0
Bev5	0.9	0.7	1

Here y is binary. With levels such as poor, mediocre, good, and great it would be multi-class classification; with a taste score from 1 to 100 it would be regression.

A lookup table of the five rows is not a prediction rule, since new beverages have new feature values. We need a rule, ideally a simple one. Each binary classifier f has a DECISION REGION: the way it partitions the feature space into points predicted 1 and points predicted 0. Performance is measured by training accuracy, the fraction of training samples with $f(x) = y$ (0-1 accuracy).

Example: *Decision stump.* A DECISION STUMP is a tree with one split. Model 1 predicts 1 if $S \geq 0.75$ and 0 otherwise. Its predictions are $(1, 0, 1, 0, 0)$, so it misclassifies Bev3 and Bev5 and has training accuracy $3/5$. Its decision region is the half-plane above the line $S = 0.75$.

Example: *Depth-2 tree.* Model 2 first splits on $A \geq 0.5$; points with $A < 0.5$ go to leaf 1. Points with $A \geq 0.5$ are split again on $S > 0.75$, into leaf 3 if so and leaf 2 otherwise. Leaf 1 receives Bev2, Bev3, Bev4; leaf 2 receives Bev5; leaf 3 receives Bev1. If the leaves are labeled $1, 0, 1$, the predictions are $(1, 1, 1, 1, 0)$ and the training accuracy is only $1/5$. For a fixed tree, the labels that maximize training accuracy are found in one pass over the data: place every sample in its leaf and label each leaf by the majority of its training labels. Here that gives labels $0, 1, 1$ and all five samples correct.

Example: *Linear classifiers.* Model 3 predicts 1 if $A + S - 1 \geq 0$ and 0 otherwise; its decision boundary is the line $A + S = 1$. The sums $A + S$ are $1.6,\ 0.55,\ 1.0,\ 1.0,\ 1.6$, so the predictions are $(1, 0, 1, 1, 1)$ and the accuracy is $3/5$: Bev3 and Bev4 lie exactly on the boundary and are predicted 1. Model 4 shifts the threshold, predicting 1 if $A + S - 1.3 \geq 0$. Its predictions $(1, 0, 0, 0, 1)$ are all correct.

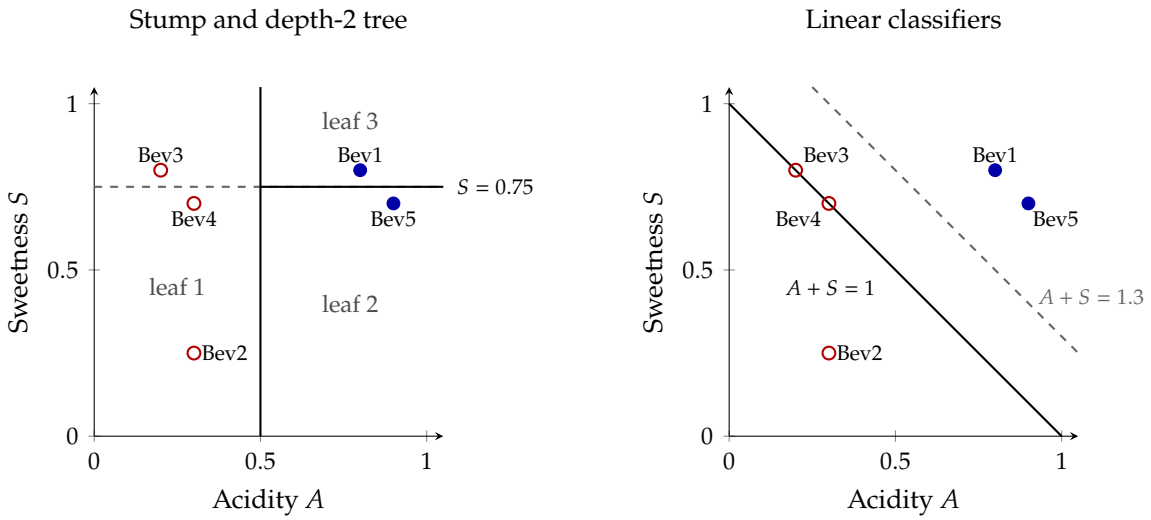


Figure 1: The taste-test data in feature space (filled: $y = 1$; open: $y = 0$). Left: the stump boundary $S = 0.75$ (dashed) and the depth-2 tree, which splits at $A = 0.5$ and then at $S = 0.75$ for $A \geq 0.5$ (solid), with its three leaves. Right: the linear boundaries $A + S = 1$ of Model 3, which passes through Bev3 and Bev4, and $A + S = 1.3$ of Model 4.

1.5 Training, testing, and memorization

The goal of prediction is to predict the future, not the past: we want to predict ratings of new beverages (Bev6 with $A = S = 0.7$, Bev7 with $A = S = 0.1$, and possibly billions of other feature combinations) without running a study for each. This raises three questions: how to build the prediction model, how to evaluate whether it works (*model evaluation*), and whether it works for the right reasons (*model interpretability*).

Proposition 1.1 [There is always a very bad model]. *For any training set, however large, there is a model with perfect training accuracy and terrible predictive performance.*

Proof. Let $M_{\text{bad}}(A, S)$ output y_i if (A, S) equals the features of training row i, and output 0 otherwise. It reproduces every training label, but on new inputs it ignores the data entirely and predicts 0. □

M_{bad} *memorizes* the training set but cannot *generalize*. Ideally we want the simplest model with good training accuracy, following Occam's razor: the simplest explanation that fits the data is usually the correct one.

The whole pipeline has two phases. Training turns data into a model (possibly a big one); at inference time an application sends a query to the model and receives a decision.

1.6 A brief history

The *early days* (1950s–60s) produced the perceptron (1957) and a self-learning checkers program (1959). The 1970s–80s brought challenges and advances: decision trees, the basics of reinforcement learning, and the rediscovery of neural networks. The 1990s saw the rise of statistical ML, with probabilistic models and a focus on mathematical foundations. In the *big data era* of the 2000s, datasets grew, computation became cheaper, and data mining and data science rose. The *deep learning revolution* began around 2012, and the present is dominated by generative models (models that can generate the data) and large language and vision models.

1.7 The ML lifecycle

Machine learning practice cycles through four stages.

1. **Learning problem (L).** Frame the real-world question as something a model can answer. *Target:* what to predict, and which ML task this is. *Objective:* how to evaluate success, and which loss to use. *Data:* what data is available, how to represent it (feature engineering), and how to split it into training and test sets.
2. **Model design (M).** Choose the model family or architecture, which fixes the hypothesis space and the inductive biases (assumptions).
3. **Optimization (O).** Adjust the parameters to minimize error: define a loss, choose an optimization method such as gradient descent, and manage regularization and overfitting.
4. **Predict and evaluate (P).** Test how well the model performs, evaluating its predictions with evaluation metrics.

The course is organized around applications first: when to use ML, how to frame the problem, how to prepare the data, and then how to train and evaluate models.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 1 (Spring 2026).

Lecture 2: Data Tools

Every model begins and ends with data: it is the raw material for training and evaluation, and success in ML depends on how well the inputs and outputs of experiments are processed. This unit covers `pandas` for tabular data and two plotting libraries.

2.1 pandas data structures

`pandas` (from "panel data") is the standard tool for tabular data. It arranges data in tables, extracts information filtered by conditions, derives new information, and applies vectorized NumPy operations for speed.

Definition 2.1 [Series, DataFrame, Index]. A Series is a one-dimensional labeled array. It has *values*, the stored data, and an index, the labels of the data points. A DataFrame is a table: a collection of Series, one per column, sharing a common index.

For `pd.Series(["welcome", "to", "CS 189"])`, `.values` is the array of the three strings and `.index` is `RangeIndex(start=0, stop=3, step=1)`. A DataFrame can be created from a CSV file, a dictionary, a list with column names, or Series. The running example is built from a dictionary:

```
import pandas as pd
data = {
    'Landmark': ['Sather Gate', 'Campanile', 'Doe Library',
                 'Memorial Glade', 'Sproul Plaza'],
    'Type': ['Gate', 'Tower', 'Library', 'Open Space', 'Plaza'],
    'Height': [30, 307, 80, 0, 0],
    'Year Built': [1910, 1914, 1911, None, 1962]}
df = pd.DataFrame(data)
event_data = pd.read_csv("data/uc_berkeley_events.csv", index_col='Year')
```

	Landmark	Type	Height	Year Built
0	Sather Gate	Gate	30	1910.0
1	Campanile	Tower	307	1914.0
2	Doe Library	Library	80	1911.0
3	Memorial Glade	Open Space	0	NaN
4	Sproul Plaza	Plaza	0	1962.0

The missing year becomes `NaN`, which turns the column into floats. The `index_col` argument of

`read_csv` makes a column (here `Year`) the index.

2.2 Exploring a DataFrame

A quick overview of structure and content is the first step of any analysis.

- `df.head(n)` and `df.tail(n)` show the first and last n rows (default 5); `df.tail(3)` shows rows 2–4.
- `df.shape` is (rows, columns), here $(5, 4)$, and `df.size` is the number of entries, $5 \times 4 = 20$.
- `df.info()` and `df.describe()` summarize column types and statistics.
- `df.sample(n=k)` draws k random rows, `sample(frac=f)` a fraction f of them; `replace=True` allows a row to appear repeatedly, and `random_state=42` makes the draw reproducible.
- For a Series, `value_counts()` counts the occurrences of each unique value (each type appears once here), and `unique()` returns the array of distinct values.

2.3 Selecting data

There are three selection mechanisms.

- `.iloc[]` selects by integer *position*, starting at 0. Its arguments can be a list, a slice that *excludes* its right end, or a single value.
- `.loc[]` selects by *label*. Its arguments can be a list, a slice that *includes* its right end, or a single value.
- `[]` is context dependent and takes one argument: a slice of row numbers, a list of column labels, or a single column label.

Position-based selection	Result
`df.iloc[[1, 2]]`	rows 1 and 2 (Campanile, Doe Library)
`df.iloc[[1, 2], [1, 2, 3]]`	those rows, last three columns
`df.iloc[2:4]`	rows 2 and 3
`df.iloc[:, 1:2]`	the Type column
`df.iloc[:, 0]`	the Landmark column as a Series
`df.iloc[2]`	row 2 as a Series indexed by column
`df.iloc[0, 1]`	the single entry `'Gate'`

Label or bracket selection	Result
`df.loc[2:3]`	rows labeled 2 *and* 3
`df.loc[:, 'Landmark':'Height']`	Landmark through Height, inclusive
`df.loc[[1, 2], ['Type', 'Height', 'Year Built']]`	as `iloc[[1, 2], [1, 2, 3]]`
`df.loc[0, 'Type']`	`'Gate'`
`df[1:3]`	rows 1 and 2
`df[['Landmark', 'Year Built']]`	two columns as a DataFrame
`df['Year Built']`	one column as a Series

Here row labels coincide with positions; they differ once the index is not $0, \ldots, n-1$, for example after sorting.

2.4 Filtering with Boolean arrays

To extract rows satisfying a condition, pass a Boolean array to `.loc` or `[]`: rows marked `True` are kept. The expression `df['Height'] > 50` is a Boolean Series (F, T, T, F, F), so `df[df['Height'] > 50]` and `df.loc[df['Height'] > 50]` both return the Campanile and Doe Library rows. Conditions combine with the elementwise (bitwise) operators `~p` (not), `p | q` (or), `p & q` (and), and `p ^ q` (exclusive or). For example,

```
df[(df['Height'] > 50) & (df['Type'] == 'Library')]
```

returns only Doe Library. The parentheses are needed because `&` binds more tightly than comparisons.

2.5 Modifying a DataFrame

To add a column, assign a Series or array of the right length to a new label:

```
df['Experience'] = [2, 5, 1, 8, 4]
df['Height_Increase'] = df['Height'] * 0.1
```

Most DataFrame operations are *not in place*: `df.drop(columns=['Experience'])` returns a new DataFrame and leaves `df` unchanged. Either pass `inplace=True` or reassign the result to `df`.

`sort_values(by=..., ascending=...)` sorts by one column or by a list of columns, with one direction per column; `ascending` defaults to `True`. Sorting by `'Height'` orders the rows 3, 4, 0, 2, 1 (ties keep their order). Passing `by=['Height', 'Type']` with `ascending=[True, False]` breaks the tie at height 0 by type in descending order, giving Sproul Plaza before Memorial Glade.

Missing values are common in real data. `isnull()` returns a Boolean DataFrame marking them (both `NaN` and `None`). They are handled by removing or imputing: `dropna()` drops every row with a missing entry, and `fillna(value)` replaces missing entries by `value`, for example 0. In a copy of the landmarks table with four missing entries spread over four rows, `dropna()` leaves only the Campanile.

2.6 Aggregation and grouping

Aggregations reduce a column to summary values: basic ones (`sum`, `mean`, `median`, `min`, `max`, `count`, `nunique`, `prod`), statistical ones (`std`, `var`, `sem` for the standard error of the mean, `skew`), and logical or index-based ones (`any`, `all`, `first` and `last` non-null values, `idxmin`, `idxmax`). For the heights, `mean()` is 83.4, `std()` is 129.20 (the sample standard deviation, dividing by $n - 1$), and `idxmax()` is 1, the Campanile.

`groupby` gathers rows of the same category and aggregates across each group. It *splits* the DataFrame into groups, *applies* a function to each, and *combines* the results into one table indexed by the group keys:

```
augmented_df.groupby('Type').agg('mean')
augmented_df.groupby(['Type', 'Campus'])[['Height']].agg('max')
```

Here `augmented_df` extends the landmarks with a `Campus` column (UC Berkeley, MIT, Stanford). The argument of `.agg` is any function that reduces several values to one: a built-in Python function (`sum`), a NumPy function (`np.mean`), or a pandas name (`"mean"`, `"first"`). Common ones can be called directly, as in `df.groupby("Type").mean()`. Grouping by two columns yields a two-level index. A PIVOT TABLE spreads the second key across the columns instead:

```
pd.pivot_table(augmented_df, index='Type', columns='Campus',
               values='Height', aggfunc='max')
```

2.7 Joining DataFrames

A join combines tables from different sources on a common key. For example, joining the landmarks with a table of campus events by year built:

```
landmarks.join(event_data.set_index('Year'), on='Year Built', how='inner')
landmarks.merge(event_data, how='inner', left_on='Year Built', right_on='Year')
```

Only the Campanile (1914) matches an event (the completion of the Campanile), so the inner join has one row. The join types are:

- INNER: only rows whose keys match in both tables;
- OUTER: all rows of both tables, with `NaN` where there is no match;
- LEFT (RIGHT): all rows of the left (right) table, with matching rows from the other.

2.8 Visualization

Visualization serves *insight*, discovering trends in the data, and *communication*, conveying them to others; the emphasis here is on insight. `Matplotlib` (`import matplotlib.pyplot as plt`) is the long-standing standard for static, publication-quality plots: line, scatter, histogram, bar, heatmap, and box plots. Its plots are static, which makes interactive slicing difficult. `Plotly`, a cross-language interactive plotting library, and Weights and Biases, a commercial service for tracking training runs and model artifacts, are designed for interaction and give insight more quickly.

Plotly can be used in three ways, from easiest to most flexible: through pandas' built-in plotting functions, through Plotly Express, and by building plots from graphics objects (as in Matplotlib). In the first two, a plot is specified by its kind and by how columns map to visual elements (`x`, `y`, `color`, `size`, shape, and so on):

```
pd.set_option('plotting.backend', 'plotly')
mpg.plot(kind='scatter', x='weight', y='mpg', color='origin',
         title='MPG vs. Weight by Origin', width=800, height=600)

import plotly.express as px
px.scatter(mpg, x='weight', y='mpg', color='origin', size='cylinders',
           hover_data=mpg.columns, title='MPG vs. Weight by Origin')
```

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 2 (Spring 2026).

Lecture 3: Machine Learning Mechanics: Terminology and Techniques

This unit walks once through every stage of the ML lifecycle of Lecture 1 (learning problem, model design, optimization, prediction and evaluation), introducing the main concepts at a high level and their implementation in `scikit-learn`, the standard Python package for data preparation, feature engineering, classic models, and evaluation. All of these topics return later in more formal form.

3.1 Framing the learning problem

Framing fixes the *target* (what to predict, and hence the ML task), the *objective* (how success is evaluated and which loss is used), and the *data* (what is available, how it is represented, and how it is split). In the running example, a clothing-trading website wants to tag uploaded pictures with clothing categories, and has example pictures with category labels. The target is the type of clothing, the data are labeled (image, category) pairs, and a first objective is overall accuracy.

Before modeling, look at the data: sample it, read it, plot it. The dataset is an $N \times D$ table of features, possibly with a column of labels.

- How much data is there (N)?
- What are the features: how many dimensions (D), with what distribution, all numeric or not, with missing values or not?
- Are there labels? Are they discrete, how are they distributed, and are some missing or wrong?

For the clothing images, each example is a 28×28 grayscale image with one of ten labels (T-shirt/top, trouser, pullover, dress, coat, sandal, shirt, sneaker, bag, ankle boot).

3.2 The taxonomy of learning problems

Supervised learning uses labeled data, *unsupervised learning* unlabeled data (dimensionality reduction, clustering), and *reinforcement learning* a reward. Supervised learning learns from examples of a functional relationship. It is the most commonly used setting and the "fastest" way to learn, requiring the least data, but it needs examples of the relationship, which may be hard to obtain. A quantitative label gives regression; a categorical label gives classification.

In classification the labels are discrete classes. They are often encoded as numbers (for example 0 = T-shirt/top, 1 = trousers, 2 = pullover), but they remain categories, not quantities. Binary classification has two classes (spam or not spam); multi-class classification has more (hot dog,

pizza, burrito, and so on).

3.3 Evaluating a classifier

The basic metric is

$$\text{accuracy} = \frac{\#\text{ correct predictions}}{\#\text{ examples}};$$

a classifier correct on 85 of 100 test examples has test accuracy 85%. Accuracy alone can mislead. A network that predicts COVID from a photograph with about 99% accuracy may still be useless: if only a small fraction of people are infected, always predicting "healthy" already achieves high accuracy. Accuracy mixes two different kinds of error.

Definition 3.1 [TYPE I AND TYPE II ERRORS]. A TYPE I ERROR, or FALSE POSITIVE, incorrectly rejects the null hypothesis. A TYPE II ERROR, or MISSED DETECTION (false negative), fails to reject a null hypothesis that is false.

The two errors typically have very different costs. Suppose 1 in 100 cars has a faulty steering-wheel fastener. Discarding some good parts raises costs by about 1%, while shipping faulty cars can hurt people and end the company. It is the data scientist's job to account for these costs: when specific costs can be assigned to the different mistakes, a problem-specific metric can be designed.

3.4 Generalization and data splits

Fitting the training data is not the goal; giving students the exam and its solutions to study would not show that they learned the material.

Definition 3.2 [GENERALIZATION]. GENERALIZATION is the ability of a model to perform well on new, unseen data sampled from the same distribution as its training data.

Evaluating generalization requires data not used for training. The standard TRAIN-TEST SPLIT shuffles the data and splits it into a larger training part (about 80%), used to develop and train the model, and a smaller test part (about 20%), used to evaluate generalization. The test set should be used only once, after development: if it is used to tune the model, it no longer measures generalization.

To tune during development, split the training part again into a training set and a VALIDATION SET. The model is fit on the training set and its design is tuned on the validation set, while the test set stays untouched. In the exam analogy, the training set is the practice questions with answers, the validation set is a practice exam that tells you whether your studying works, and the test set is the exam itself. In the clothing demo, two 80/20 splits of 60,000 images (`train_test_split(..., test_size=0.2, random_state=42)`) give 38,400 training, 9,600 validation, and 12,000 test images.

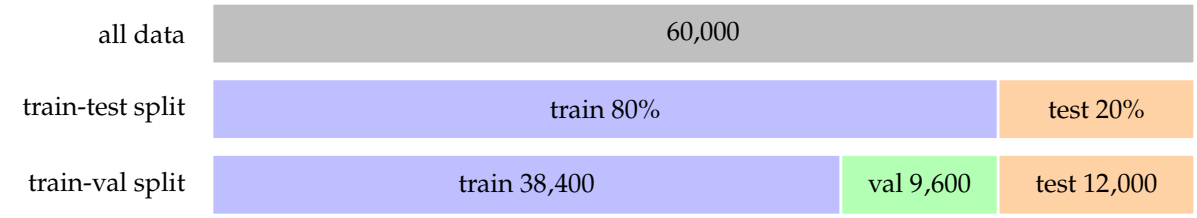


Figure 2: The two successive 80/20 splits of the clothing demo. The test set is set aside first; the validation set is carved out of the remaining training data.

Even a carefully held-out test set reuses one data-collection process. ImageNetV2 was collected in the same way as ImageNet and looks visually very similar, yet ImageNet classifiers achieve lower accuracy on it than on the original test set: plotting new against original accuracy, the models fall on a line below the diagonal that would mean no drop.

3.5 Feature engineering

Feature engineering is the process of selecting and encoding input features from the raw features. *Selecting* the right features improves performance: adding features from other data sources can help, while too many features can hurt when data is limited. *Encoding* transforms features such as categories, text, and images into numeric representations. The core innovation of deep learning is to *learn* these encodings.

Numerical features are often used unchanged, with three important exceptions.

- *Categorical features* given as numbers (a ZIP code, a product SKU, a numerical coding red = 1, blue = 2) are usually one-hot encoded.
- *Heavily skewed features* with extreme values (click counts, prices) are often log-transformed.
- Features with very different magnitudes and variability are usually *standardized*.

Definition 3.3 [One-hot encoding]. One-hot encoding replaces a categorical feature by one binary feature per possible value, and sets to 1 only the column of the observed value.

A color feature with values red, blue, green, yellow, and missing becomes five binary columns (Color:Red, Color:Blue, Color:Green, Color:Yellow, Color:Missing). The value green becomes $(0, 0, 1, 0, 0)$, and a missing value gets its own column.

Definition 3.4 [Standardization]. Compute the mean μ and variance σ^2 of a feature x on the training data, and replace x by

$$z = \frac{x - \mu}{\sigma},$$

which has zero mean and unit variance on the training data.

Standardization will later be seen to improve model fitting. The same transformation, with the *training* μ and σ, must be applied at test time.

Text has several encodings. Categorical strings (color, state, name) are one-hot encoded. Multi-word strings (emails, messages) classically use a BAG-OF-WORDS: one column per vocabulary word, holding that word's count. "Learning about machine learning is fun" has 2 in the column *learning*, 1 in *machine* and *fun*, and 0 in *aardvark* and almost every other column. STOP WORDS such as *the*, *is*, *of* carry little information and are often dropped from the vocabulary. Bag-of-words ignores word order: "New buildings need ML inspired art!" and any shuffle of its words have the same encoding. Today, large language models often map text to learned vector embeddings instead.

An image is a three-dimensional tensor (height, width, color channel). It can be flattened into a vector, which is naive but effective for small monochromatic images; transformed (color space, pixel normalization); described by hand-crafted features (edge detectors, texture descriptors); or embedded by a neural network. All of these encodings, for text and images alike, produce high-dimensional vectors.

Scikit-learn's transformations follow a common pattern: construct the transformer, `fit` it to data, then `transform` data.

```
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder()           # construct
ohe.fit(df[["color"]])          # fit (learn the categories)
ohe.transform(df[["color"]])    # transform
```

3.6 Model design

Machine learning is function approximation: a model h_w maps inputs X to outputs Y. Splitting it as

$$h_w = g_{w_2} \circ f_{w_1}$$

separates a featurization f_{w_1} (hand-designed or learned), which maps inputs to feature vectors, from a learned function g_{w_2} of the features. This allows general model families for g_{w_2} with corresponding training algorithms.

Definition 3.5 [MODEL FAMILY, HYPOTHESIS SPACE]. A MODEL FAMILY determines the form of the function g_{w_2} and its output space (for example classification or regression). The HYPOTHESIS SPACE is the set of all models in the family.

The LINEAR REGRESSION model family is

$$g_w(\mathbf{x}) = w_0 + \sum_{d=1}^{D} w_d x_d.$$

Each weight vector w gives a model: $g_{[1,1]}(x) = 1 + x_1$, $g_{[1,-1]}(x) = 1 - x_1$, $g_{[2,2]}(x) = 2 + 2x_1$. For one feature, the hypothesis space is all lines with real slopes and intercepts.

Model families are characterized by their inductive biases and by the complexity they can represent.

Definition 3.6 [INDUCTIVE BIAS]. An INDUCTIVE BIAS is an assumption made in the model design to enable generalization beyond the training data.

With training data $\{(-1, 1), (1, 1)\}$ and a test point $x = 0$, infinitely many models fit the data exactly, and any of them could be correct. Choosing the linear family is an inductive bias; it predicts $y = 1$. The NO FREE LUNCH THEOREM of ML says there is no universally best model, so we must choose a model with the right inductive biases. Feature engineering also introduces inductive biases into h_w.

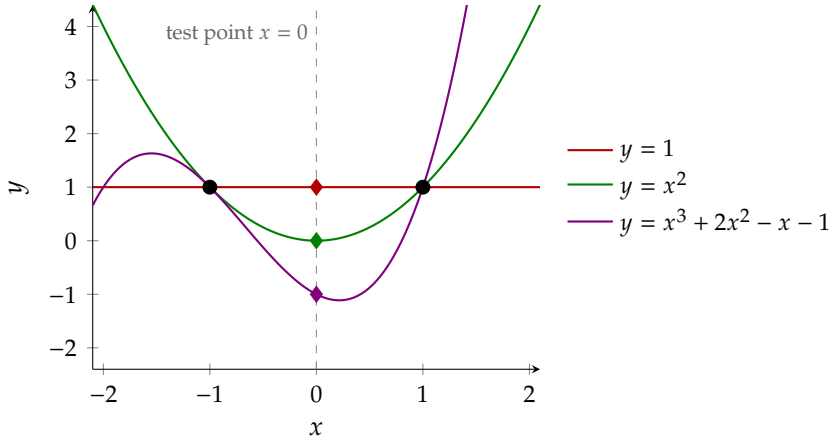


Figure 3: Three models that fit the training points $(-1, 1)$ and $(1, 1)$ exactly but disagree at the test point $x = 0$: the line $y = 1$, the quadratic $y = x^2$, and the cubic $y = x^3 + 2x^2 - x - 1$ (illustrative choices).

Nonlinear models are more expressive than linear ones: a nonlinear regression curve can follow wiggly data, and a nonlinear decision boundary can separate classes that no line separates. More complex is not always better. Fitting models of increasing complexity to samples of noisy data moves from UNDERFITTING (too simple to capture the trend) through a "sweet spot" to OVERFITTING (the fitted functions vary wildly from sample to sample and follow the noise).

Definition 3.7 [REGULARIZATION]. REGULARIZATION adds constraints or penalties to the learning process to improve generalization.

More regularization moves a model from overfitting toward underfitting, and many models and algorithms have a parameter that tunes it.

Definition 3.8 [Parametric and non-parametric models]. A parametric model family has a fixed number of parameters, independent of the training-set size; linear regression is an example. A non-parametric family has "parameters" that grow with the training data.

The nearest neighbor model is non-parametric. Given training data $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$, it predicts the label of the closest training point,

$$g_{\mathcal{D}}(\mathbf{x}) = y_{i(\mathbf{x})}, \qquad i(\mathbf{x}) = \arg\min_n \|\mathbf{x}_n - \mathbf{x}\|.$$

Its "parameters" are all the training data, and its prediction is constant on each cell of the Voronoi diagram of the training points.

To choose a model family: determine the learning problem (classification, regression, clustering, dimensionality reduction); start with linear models and good features, a common way to encode inductive biases; then try increasingly complex models while checking validation performance.

Logistic regression is a linear model for classification,

$$g_w(\mathbf{x}) = \sigma\Big(w_0 + \sum_{d=1}^{D} w_d x_d\Big), \qquad \sigma(t) = \frac{1}{1+e^{-t}},$$

where the nonlinear *sigmoid* σ maps a real number into $(0, 1)$ and is used to model probabilities: $P(y = 1 \mid \mathbf{x}) = g_w(\mathbf{x})$. Its decision boundary is linear, and it extends to multi-class classification.

3.7 Optimization and hyperparameters

Training (fitting) a model typically means solving

$$\hat{w} = \arg\min_{w \in \Theta} \ \mathrm{Error}\big[h_w; \mathfrak{D}_{\text{training}}\big] + \lambda\, \mathrm{Reg}[w]$$

with an iterative optimization algorithm. Such algorithms can be slow, require tuning, are often approximate, and may return local minima; the tuning itself affects generalization.

Definition 3.9 [Hyperparameter]. A hyperparameter is a parameter held constant during the optimization (training) algorithm, such as λ above. Hyperparameters are chosen to improve generalization as measured on the validation set.

A common method is grid search: train one model for each combination of hyperparameter values and keep the combination with the smallest validation error.

```python
for hp1 in [0.1, 1, 10]:
    for hp2 in [0.1, 1, 10]:
        model = MyModel(hp1=hp1, hp2=hp2)
        model.fit(X_train, y_train)
        error[hp1, hp2] = error(y_val, model.predict(X_val))
hp1_best, hp2_best = arg_min(error)
```

hp1\hp2	0.1	1	10
0.1	.8	.7	.7
1	.4	**.2**	.3
10	.5	.6	.6

In this grid of validation errors the best choice is hp1 = hp2 = 1.

When a regularization hyperparameter is tuned against the validation set, the typical picture is: training accuracy exceeds validation accuracy; as regularization decreases, training accuracy keeps increasing; and validation accuracy first increases and then decreases. The sweet spot is where validation accuracy peaks.

3.8 Prediction and evaluation

Inference is making predictions with a trained model. A *label prediction* (a class, or a regression value) gives the most likely value but no uncertainty; in scikit-learn it is `model.predict()`. A *predicted distribution* (class probabilities, or a mean and variance for regression) encodes the uncertainty; for classifiers it is `model.predict_proba()`. Many modern techniques, ChatGPT among them, predict distributions.

Success at test time is measured by accuracy or another error metric on the test data, but not all errors are equal. For spam detection the outcomes are:

	Spam	Not spam
Predicted spam	true positive	false positive
Predicted not spam	false negative	true negative

Falsely classifying a legitimate message as spam has real risks. Decisions are therefore often made from predicted probabilities with decision theory, which requires quantifying the costs of decisions, not just the model. In the clothing demo, a scikit-learn `LogisticRegression()` model is fit with `model.fit(X=X_tr, y=y_tr)` on the training images.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 3 (Spring 2026).

Lecture 4: Clustering, Probability Review

4.1 Unsupervised learning

Supervised learning uses data $\mathcal{D} = \{(x_i, y_i)\}$ with $x \in \mathbb{R}^d$ and $y \in \mathbb{R}$ or $y \in \mathbb{Z}$, for example predicting the price y_i of a house from its ZIP code and square footage x_i. Much of ML instead models data $\mathcal{D} = \{x_i\}$ without labels. This is unsupervised learning, which includes

(i) *clustering*, $z = f_\theta(x) \in \mathbb{Z}$;

(ii) *dimensionality reduction*, $z = f_\theta(x) \in \mathbb{R}^m$ with $m \ll d$;

(iii) *representation learning*, $z \in \mathbb{R}^m$ with $z = f_\theta(x)$ or $z \sim p_\theta(x)$;

(iv) *generative modeling*, sampling $x \sim p_\theta(x)$;

(v) *density estimation*, estimating $p_\theta(x)$.

For example, each eruption of the Old Faithful geyser gives a point $x_i = (x_i^1, x_i^2)$, the eruption duration and the waiting time; the points form two visible groups.

4.2 Clustering

Clustering infers or assigns discrete "class labels" to unlabeled data, based on the structure of the input space. It is used for data exploration, to find hidden subgroups: patterns in nature (geyser eruptions), types of customers from their activity, disease subtypes from patient charts, cell types from single-cell data, ancestry groups from genetic data, and topics in documents. It is also used for outlier detection, where points that belong to no cluster are flagged as noise.

There are three broad approaches.

- *Hierarchical clustering* builds a tree, bottom-up or top-down, representing distances among data points (for example single- or average-linkage clustering).
- *Partitional approaches* define and optimize a "cost" over partitions (for example spectral clustering and graph-cut methods).
- *Model-based approaches* maintain cluster "models" and infer cluster membership, for example by assigning each point to the closest center (k-means, Gaussian mixture models).

A clustering should have high intra-cluster similarity (small distances within clusters) and low inter-cluster similarity (large distances between clusters); which similarity or distance to use is up to the modeler.

4.3 K-means

In CENTROID-BASED CLUSTERING, each cluster is represented by a point of the input space, its CENTROID $c_k \in \mathbb{R}^d$. K-MEANS is the most common centroid-based approach: the centroids are the model parameters, chosen so that the distance of each point to its assigned centroid is minimized.

Definition 4.1 [K-MEANS OBJECTIVE]. Given training data $X = \{x_i\}_{i=1}^n$ with $x_i \in \mathbb{R}^d$, a CLUSTER PARTITION $S = C_1 \cup \cdots \cup C_K$ assigns every x_i to exactly one of K clusters. K-means solves

$$\underset{S=C_1\cup\cdots\cup C_K,\ \{c_1,\ldots,c_K\}}{\arg\min} \sum_{k=1}^{K} \sum_{x\in C_k} \|x - c_k\|^2, \tag{4.1}$$

jointly over the partition and the centroids.

The joint problem is hard, but each half is easy when the other is fixed.

Proposition 4.2 [OPTIMAL CENTROIDS]. *For a fixed partition, the optimal centroid of cluster k is its mean,*

$$\hat{c}_k = \underset{c_k}{\arg\min} \sum_{x\in C_k} \|x - c_k\|^2 = \frac{1}{|C_k|} \sum_{x\in C_k} x.$$

Proof. The objective separates over clusters, so each c_k is optimized separately. The function $\sum_{x\in C_k}(x - c_k)^\top(x - c_k)$ is convex in c_k, with gradient

$$\frac{\partial}{\partial c_k} \sum_{x\in C_k} (x - c_k)^\top (x - c_k) = -2 \sum_{x\in C_k} (x - c_k).$$

Setting it to zero gives $\sum_{x\in C_k} x = \sum_{x\in C_k} c_k = |C_k|\, c_k$. □

Proposition 4.3 [OPTIMAL ASSIGNMENTS]. *For fixed centroids, the optimal partition assigns each point to its closest centroid:*

$$z_i = \underset{k}{\arg\min} \|x_i - c_k\|^2, \qquad \hat{C}_k = \{x_i : z_i = k\}.$$

Proof. Each point contributes exactly one term $\|x_i - c_{z_i}\|^2$ to (4.1), and its choice of cluster affects no other term. Each term is therefore minimized separately by the closest centroid. □

Alternating these two steps is the K-means algorithm.

Algorithm 4.1: K**-means (Lloyd's algorithm)**

Input: Data $x_1, \dots, x_n \in \mathbb{R}^d$ and the number of clusters K.
Output: A partition $C_1, \dots, C_K$ and centroids $c_1, \dots, c_K$.

Initialize $c_1, \dots, c_K$, e.g. as K training points chosen at random
repeat
  for $i \leftarrow 1$ **to** n **do**
    $z_i \leftarrow \arg\min_k \|x_i - c_k\|^2$
  $C_k \leftarrow \{x_i : z_i = k\}$ for $k = 1, \dots, K$
  $c_k \leftarrow \frac{1}{|C_k|} \sum_{x \in C_k} x$ for $k = 1, \dots, K$
until the assignments do not change
return $C_1, \dots, C_K$ and $c_1, \dots, c_K$

Theorem 4.4 [Convergence of K-means]. *Each step of Lloyd's algorithm does not increase the objective (4.1), and the algorithm terminates after finitely many iterations at a local minimum: a pair (partition, centroids) in which each is optimal given the other.*

Proof. By Propositions 4.2 and 4.3, the assignment step minimizes the objective over partitions with the centroids fixed, and the centroid step minimizes it over centroids with the partition fixed. Neither can increase it. Break ties in the assignment step in favor of a point's current cluster. Then whenever some assignment changes, that point moves to a strictly closer centroid, so the objective strictly decreases. Hence no partition can occur twice, and since there are only finitely many partitions of n points into K clusters, the assignments stop changing after finitely many iterations. At that point each half is optimal given the other, which is a local, not necessarily global, minimum. □

On three well-separated groups of points with randomly initialized centroids, the iterations alternate between recoloring every point by its nearest centroid and moving each centroid to the mean of its points. Within a few iterations the centroids settle at the centers of the three groups.

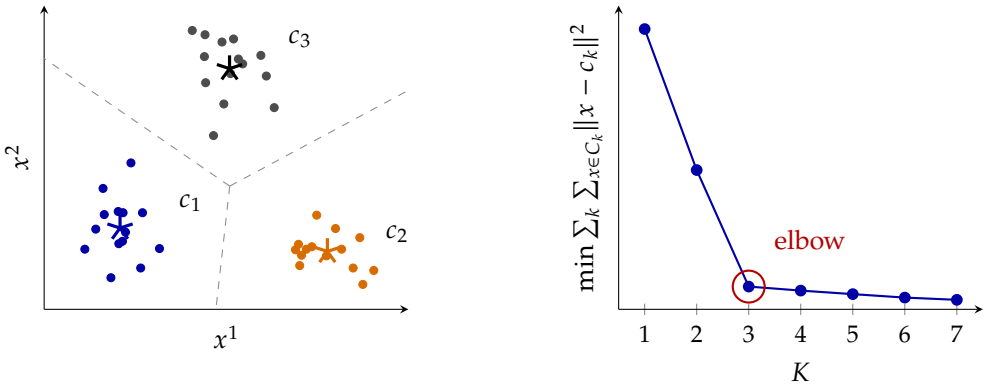


Figure 4: Synthetic illustration: three Gaussian groups of 14 points each. Left: the converged K-means solution with $K = 3$; the dashed lines bound the nearest-centroid regions, so each cluster is a cell of a Voronoi tessellation. Right: the minimum of the objective (4.1) on the same points for $K = 1, \dots, 7$; it always decreases, and the "elbow" at $K = 3$ suggests the number of clusters.

4.4 Distances, metrics, and similarities

Clustering asks points to be similar within clusters and dissimilar across them, that is, distances to be small or large.

Definition 4.5 [METRIC]. A distance function d is a METRIC if for all points i, j, k:

1. $d(j,k) = 0$ if and only if $j = k$;
2. $d(j,k) > 0$ if and only if $j \neq k$;
3. it is symmetric, $d(j,k) = d(k,j)$;
4. it satisfies the triangle inequality, $d(i,j) + d(i,k) \geq d(j,k)$.

The KL divergence is not a distance in this sense because it is not symmetric. Nor is the K-means "distance" $\|x - c\|^2$ a metric: for the points $0, 1, 2$ on the line it gives $d(0,1) + d(1,2) = 2 < 4 = d(0,2)$, violating the triangle inequality. A DISSIMILARITY is intuitively related to a distance but need not satisfy the metric properties, and a SIMILARITY is its "complement", for example similarity$(j,k) =$ $1 -$ dissimilarity(j,k).

4.5 Choosing K, applications, and weaknesses

The optimal objective value always decreases as K grows (with $K = n$ it is 0), so it cannot be minimized over K. Instead, plot it against K and look for an ELBOW, the point after which adding clusters gives only small improvements. On data with four visible groups, the elbow is at $K = 4$.

Two applications: clustering yeast genes by their expression measured over time, where each centroid is a typical expression profile; and PIXEL K-MEANS. A color image of size height $\times$ width with three color channels is flattened into height $\cdot$ width vectors in RGB space, which has 256^3 possible values. Clustering these vectors and replacing each pixel by its centroid renders the image with only K colors.

K-means has weaknesses.

- It can converge to bad local minima, for example two centroids sharing one true cluster while another centroid covers two.
- It is not probabilistically framed, so its assumptions and their implications are hard to understand. It implicitly assumes clusters of similar volume sitting in a "blobby" Voronoi tessellation, which is often undesirable: a large spread-out cluster next to two small tight ones is split incorrectly.
- It is hard to generalize.
- Each step makes a *hard* assignment of every point to one cluster, which is brittle.

A probabilistic version of K-means, the mixture of Gaussians, fixes these issues.

4.6 Probability review

Probability is a framework for quantifying and manipulating uncertainty. In the FREQUENTIST VIEW it describes objective, repeatable phenomena: a fair coin flipped infinitely many times lands heads half the time. In the BAYESIAN VIEW it represents one's uncertainty rather than a physical frequency, such as a doctor's belief that a genetic variant causes cancer in one particular patient. Both views are useful.

Definition 4.6 [JOINT DISTRIBUTION]. Let X and Y be discrete random variables with values $X \in \{x_1, \dots, x_L\}$ and $Y \in \{y_1, \dots, y_M\}$. The JOINT PROBABILITY DISTRIBUTION $p(X = x_i, Y = y_j)$ is the probability of the event that $X = x_i$ and $Y = y_j$. It satisfies

$$p(X = x_i, Y = y_j) \geq 0 \text{ for all } i, j, \qquad \sum_{i=1}^{L} \sum_{j=1}^{M} p(X = x_i, Y = y_j) = 1.$$

The SUM RULE, or MARGINALIZATION, gives the distribution of a subset of the variables:

$$p(X = x_i) = \sum_{j=1}^{M} p(X = x_i, Y = y_j).$$

The CONDITIONAL DISTRIBUTION of Y given that we observed $X = x_i$ is

$$p(Y = y_j \mid X = x_i) = \frac{p(X = x_i, Y = y_j)}{p(X = x_i)}.$$

Example: *A joint table.* Take $L = 3$, $M = 2$ and the joint distribution below, with its row and column sums (the marginals):

	y_1	y_2	$p(X = x_i)$
x_1	0.2	0.1	0.3
x_2	0.1	0.1	0.2
x_3	0.15	0.35	0.5
$p(Y = y_j)$	0.45	0.55	1

Conditioning on $X = x_3$ restricts to the third row and renormalizes:

$$p(Y = y_1 \mid X = x_3) = \frac{0.15}{0.15 + 0.35} = \frac{0.15}{0.5} = 0.3, \qquad p(Y = y_2 \mid X = x_3) = 1 - 0.3 = 0.7.$$

Given N samples $(X_1, Y_1), \dots, (X_N, Y_N)$ of (X, Y), let n_{ij} count the samples with $X = x_i$ and $Y = y_j$.

The EMPIRICAL PROBABILITY DISTRIBUTION is

$$\hat{p}(X = x_i, Y = y_j) = \frac{n_{ij}}{N}.$$

In the frequentist view it converges to the true distribution,

$$p(X = x_i, Y = y_j) = \lim_{N \to \infty} \hat{p}(X = x_i, Y = y_j).$$

Multiplying the definition of the conditional by $p(X = x_i)$ gives the PRODUCT RULE,

$$p(X = x_i, Y = y_j) = p(Y = y_j \mid X = x_i)\, p(X = x_i).$$

Like a chain rule, it factorizes joint distributions into conditionals, for example $p(X, Y, Z) = p(Z \mid X, Y)\, p(X \mid Y)\, p(Y)$; here, as is common, the values x_i, y_j, z_k are dropped from the notation.

Definition 4.7 [INDEPENDENCE]. X and Y are INDEPENDENT, $X \perp Y$, if their joint distribution factorizes: $p(X, Y) = p(X)\, p(Y)$.

For independent variables, conditioning has no effect:

$$p(Y \mid X) = \frac{p(Y, X)}{p(X)} = \frac{p(Y)\, p(X)}{p(X)} = p(Y).$$

Data $x_1, \ldots, x_N$ are INDEPENDENT AND IDENTICALLY DISTRIBUTED (IID) when they are independent draws from one distribution; then their joint probability factorizes as

$$p(x_1, \ldots, x_N) = \prod_{n=1}^{N} p(X = x_n).$$

The same holds for pairs and for conditionals:

$$p(x_1, y_1, \ldots, x_N, y_N) = \prod_{n=1}^{N} p(X = x_n, Y = y_n),$$

$$p(y_1, \ldots, y_N \mid x_1, \ldots, x_N) = \prod_{n=1}^{N} p(Y = y_n \mid X = x_n).$$

IID data is a frequent modeling assumption, though often not true in practice.

4.7 Bayes' theorem: the wake-word detector

Theorem 4.8 [Bayes' theorem].

$$p(A \mid B) = \frac{p(B \mid A)\, p(A)}{p(B)} = \frac{p(B \mid A)\, p(A)}{\sum_A p(B \mid A)\, p(A)}.$$

Proof. By the product rule applied in both orders, $p(A, B) = p(A \mid B)\, p(B) = p(B \mid A)\, p(A)$; divide by $p(B)$. The denominator follows from the sum rule and the product rule, $p(B) = \sum_A p(A, B) = \sum_A p(B \mid A)\, p(A)$. □

In Bayesian statistics, $p(A)$ is the prior belief about A, $p(B \mid A)$ is the likelihood of an observation B given A, and $p(A \mid B)$ is the posterior belief about A after observing B. Bayesian updating, starting from a prior and using observations to compute a posterior, is central to Bayesian machine learning.

Example: *Wake words.* A voice assistant continuously runs a detector for its wake word (such as "Alexa") on every sound segment, and streams detected segments to the cloud. Let $X \in \{0, 1\}$ indicate whether the wake word was said and $Y \in \{0, 1\}$ whether it was detected. Wake words are rare, $p(X = 1) = 0.0001$, so $p(X = 0) = 0.9999$. The detector has true positive rate (recall) $p(Y = 1 \mid X = 1) = 0.99$ and false positive rate $p(Y = 1 \mid X = 0) = 0.001$. If the detector fires, the probability that the wake word was said is

$$\begin{aligned} p(X = 1 \mid Y = 1) &= \frac{p(Y = 1 \mid X = 1)\, p(X = 1)}{p(Y = 1 \mid X = 1)\, p(X = 1) + p(Y = 1 \mid X = 0)\, p(X = 0)} \\ &= \frac{0.99 \cdot 0.0001}{0.99 \cdot 0.0001 + 0.001 \cdot 0.9999} \approx 0.09. \end{aligned}$$

Despite a 99% recall and a 0.1% false positive rate, only about 9% of detections are real: the prior $p(X = 1) = 0.0001$ is so small that false alarms on the many negative segments outnumber true detections. The observation $Y = 1$ updates the prior 0.0001 to the posterior 0.09.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 4 (Spring 2026).

Lecture 5: Introduction to Maximum Likelihood Estimation

The k-means objective of Lecture 4,

$$\underset{S=C_1\cup\cdots\cup C_K,\ \{c_1,\ldots,c_K\}}{\arg\min} \sum_k \sum_{x\in C_k} \|x - c_k\|^2,$$

is not framed probabilistically, so its assumptions and their implications are hard to see. It implicitly favors clusters of similar volume that are roughly isotropic (a "blobby tessellation"), it is hard to generalize, and every step makes a hard assignment of each point to one cluster, which is brittle. A probabilistic model that fixes these issues is the MIXTURE OF GAUSSIANS, or GAUSSIAN MIXTURE MODEL, in which each cluster is a multivariate Gaussian. Fitting it requires two tools: multivariate Gaussians (Lecture 6) and maximum likelihood estimation, the subject of this lecture.

5.1 The abstract learning recipe

Every method in the course has the same ingredients.

- *Training data.* In SUPERVISED learning the data are labeled pairs $D = \{(x_i, y_i)\}_{i=1}^N$ with $x_i \in \mathbb{R}^D$; the label provides the supervision. It is REGRESSION when $y_i \in \mathbb{R}$ and CLASSIFICATION when, for example, $y_i \in \{-1, 1\}$. In UNSUPERVISED learning the data are $D = \{x_i\}_{i=1}^N$ alone.
- *Model class* (HYPOTHESIS CLASS), a parametrized family. Two examples: linear regression $g(x \mid w, w_0) = w^T x + w_0$, and the Gaussian density $p(x \mid \mu, \sigma) = \mathcal{N}(x \mid \mu, \sigma)$.
- *Loss function* L_i for one example. For linear regression, the squared loss $L_i = (y_i - g(x_i \mid w, w_0))^2$. For a density there is no label to compare with; the natural loss is the NEGATIVE LOG-LIKELIHOOD $L_i = -\log \mathcal{N}(x_i \mid \mu, \sigma)$.
- *Learning objective*, an optimization problem over the parameters ϕ: $\arg\min_\phi \sum_{i=1}^N L_i$.

The question is what "good" parameter values mean. Maximum likelihood gives a principled and widely used answer, and it yields the losses of statistical models from linear regression to neural networks. For example, pre-training a large language model with weights w computes the MLE of

$$\ln p(\text{Internet} \mid w) = \sum_{\text{Doc}\in\text{Internet}} \sum_{\text{Word}\in\text{Doc}} \ln p(\text{Word} \mid \text{words before it in Doc}, w).$$

5.2 Probability distributions

A random variable takes values according to a distribution. A discrete random variable has a PROBABILITY MASS FUNCTION (PMF), whose values sum to 1; a continuous one has a PROBABILITY DENSITY FUNCTION (PDF) with $\int_{-\infty}^{\infty} p(x)\,dx = 1$, and $\int_{x_1}^{x_2} p(x)\,dx$ is the probability that the value lies in $[x_1, x_2]$. Standard examples:

- BERNOULLI: one toss of a possibly biased coin, $P(\text{heads}) = p$, $P(\text{tails}) = 1 - p$.
- BINOMIAL: the number of heads k in n tosses, $P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$.
- POISSON: the number k of events (say mutations in a cell population) in a fixed time interval with mean rate λ, $P(X = k) = e^{-\lambda}\lambda^k / k!$.
- UNIFORM on $[a, b]$: constant density $1/(b-a)$ there.
- GAUSSIAN, written $X \sim \mathcal{N}(\mu, \sigma^2)$: $p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\!\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$.

In a MULTIVARIATE DISTRIBUTION the outcome is a vector. The MULTINOMIAL generalizes the binomial: draw balls from an urn whose colors green, blue, red have probabilities p_1, p_2, p_3, and count each color. In the multivariate Gaussian, the mean becomes a vector and the variance becomes a covariance matrix.

5.3 Maximum likelihood estimation

Given data $D = \{x_i\}_{i=1}^N$ with $x_i \in \mathbb{R}^d$, assume a family of distributions $\{p_\theta(x) \mid \theta \in \Theta\}$ on $\mathbb{R}^d$; for example $\theta = (\mu, \sigma^2)$ for $x \in \mathbb{R}$. We write $p_\theta(x)$ and $p(x \mid \theta)$ interchangeably. Assume the data are samples $x_i \sim p_{\theta^*}(x)$ from one member of the family, drawn INDEPENDENTLY AND IDENTICALLY DISTRIBUTED (I.I.D.). The goal is to estimate the θ that pins down the distribution the data came from.

Definition 5.1 [MAXIMUM LIKELIHOOD ESTIMATE]. The LIKELIHOOD is $p(D \mid \theta)$, viewed as a function of θ with D fixed. A MAXIMUM LIKELIHOOD ESTIMATE (MLE) is

$$\theta_{\text{MLE}} = \arg\max_{\theta \in \Theta} p(D \mid \theta).$$

By independence the likelihood factors, $p(D \mid \theta) = \prod_{i=1}^N p(x_i \mid \theta)$. A product is inconvenient to differentiate, so take logarithms. Since log is monotonically increasing, the LOG-LIKELIHOOD

$$\log p(D \mid \theta) = \sum_{i=1}^{N} \log p(x_i \mid \theta)$$

has the same maximizers: $\arg\max_\theta p(D \mid \theta) = \arg\max_\theta \log p(D \mid \theta)$. The maximizer need not be unique.

Properties of the MLE:

- Consistency: with more and more data drawn from one distribution in the family, the estimate converges to the true θ.
- Statistical efficiency: it makes good use of the data, with "least variance" parameter estimates.
- The maximum value $p(D \mid \theta_{\mathrm{MLE}})$ is invariant to re-parametrization, for instance writing the mean as $2 + \mu$ instead of μ in $\mathcal{N}(x \mid \mu, \sigma)$.
- It still produces an estimate when the data were not generated by the assumed family, which is both convenient and a caveat.

5.4 MLE for a univariate Gaussian

Central-limit arguments suggest that a quantity such as height is roughly normally distributed. Model heights $x_1, \dots, x_N$ as i.i.d. $\mathcal{N}(\mu, \sigma^2)$ and maximize

$$\ell(\mu, \sigma^2) = \sum_{i=1}^{N} \log \mathcal{N}(x_i \mid \mu, \sigma^2) = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^{N} (x_i - \mu)^2.$$

The plan is calculus: find a stationary point, where the partial derivatives vanish, then check that it is a maximum by the second derivative. With more than one parameter the check is that the Hessian is negative definite.

Proposition 5.2 [Gaussian MLE].

$$\mu_{\mathrm{MLE}} = \frac{1}{N} \sum_{i=1}^{N} x_i, \qquad \sigma^2_{\mathrm{MLE}} = \frac{1}{N} \sum_{i=1}^{N} (x_i - \mu_{\mathrm{MLE}})^2.$$

Proof. The first term of ℓ does not depend on μ, so

$$\frac{\partial \ell}{\partial \mu} = \frac{1}{\sigma^2} \sum_{i=1}^{N} (x_i - \mu) = 0 \iff \sum_{i} x_i = N\mu,$$

and $\partial^2 \ell / \partial \mu^2 = \sum_i (-1/\sigma^2) = -N/\sigma^2 < 0$, a maximum. For the variance, treat σ^2 as the variable:

$$\frac{\partial \ell}{\partial \sigma^2} = -\frac{N}{2\sigma^2} + \frac{1}{2\sigma^4} \sum_{i=1}^{N} (x_i - \mu)^2 = \frac{1}{2\sigma^2} \left(-N + \frac{1}{\sigma^2} \sum_{i=1}^{N} (x_i - \mu)^2 \right) = 0,$$

which gives the stated σ^2 at $\mu = \mu_{\mathrm{MLE}}$. □

5.5 MLE gives a point estimate

MLE returns one value of θ, not a distribution over it that would capture uncertainty. Bayesian statistics instead computes a posterior distribution $p(\theta \mid D)$; this returns in Lecture 8.

A single maximizer can be fragile. The likelihood may have a narrow spike at θ_{MLE} while a broad region contains a large set of good parameters (Figure 5). Parameters in the broad region are more robust, while the spike typically reflects a fit to accidents of the sample, as when a high-degree polynomial passes through every training point.

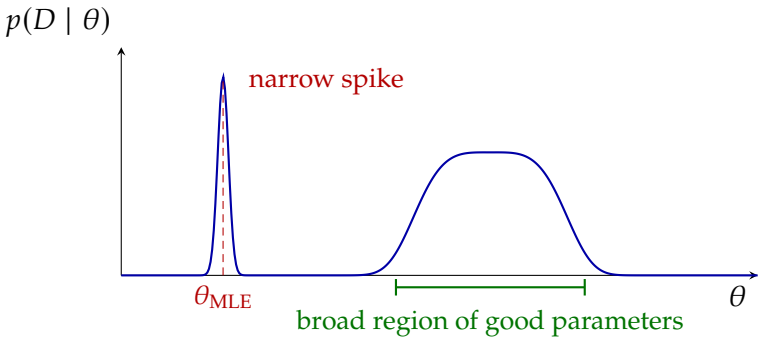


Figure 5: Schematic likelihood with a narrow "freakish" spike at θ_{MLE} and a broad region of good, robust parameter values. Not drawn from data.

5.6 MLE for the multinomial distribution

Roll a six-sided die N times and estimate the face probabilities $\theta = (\theta_1, \ldots, \theta_6)$, where $P(X = k \mid \theta) = \theta_k$ and $\sum_k \theta_k = 1$. Let $n_k = |\{i : x_i = k\}|$ be the number of rolls showing face k. Then

$$p(D \mid \theta) = \prod_{i=1}^{N} p(x_i \mid \theta) = \prod_{i=1}^{N} \prod_{k=1}^{6} \theta_k^{I[x_i=k]} = \prod_{k=1}^{6} \theta_k^{\sum_i I[x_i=k]} = \prod_{k=1}^{6} \theta_k^{n_k},$$

so the MLE solves the constrained problem

$$\theta_{\text{MLE}} = \operatorname*{arg\,max}_{\theta:\, \sum_k \theta_k = 1} \sum_{k=1}^{6} n_k \log \theta_k.$$

Proposition 5.3 [Multinomial MLE]. *$\theta_k = n_k / N$ for $k = 1, \ldots, 6$.*

Proof. Use a Lagrange multiplier and look for stationary points of

$$J(\theta, \lambda) = \sum_{k=1}^{6} n_k \log \theta_k + \lambda \Big(1 - \sum_k \theta_k\Big)$$

with respect to both θ and λ.

1. $\partial J/\partial\lambda = 0$ returns the constraint $\sum_k \theta_k = 1$.
2. $\partial J/\partial\theta_j = n_j/\theta_j - \lambda = 0$, so $\theta_j = n_j/\lambda$.
3. Substituting into the constraint, $1 = \sum_k n_k/\lambda$, so $\lambda = \sum_k n_k = N$.

Hence $\theta_k = n_k/N$. The objective is concave in θ and the constraint is linear, so this stationary point is the maximum. □

For Gaussians, multinomials, and linear regression, setting the derivative to zero gives the MLE in closed form. For a neural network such as $F(x) = w_2^T \sigma(W_1^T x + b_1) + b_2$ there is no closed form, and the MLE is found by iterative optimization (gradient descent). Its familiar squared loss $\big(y_i - F(x_i \mid W_1, w_2, b_1, b_2)\big)^2$ is itself derived from MLE, as Lecture 7 shows for linear regression.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 5 (Spring 2026).

Lecture 6: Multivariate Gaussians and Mixtures of Gaussians

The lecture first completes the multinomial MLE of Lecture 5, $\theta_k = n_k/N$, and then builds the two ingredients of a probabilistic version of k-means: the multivariate Gaussian, and the mixture of Gaussians with its likelihood.

6.1 The multivariate Gaussian

The univariate density $p(x \mid \mu, \sigma^2) = (2\pi\sigma^2)^{-1/2} \exp\big(-\frac{1}{2\sigma^2}(x-\mu)^2\big)$ extends to vectors.

Definition 6.1 [Multivariate Gaussian]. For $x \in \mathbb{R}^d$, a mean $\mu \in \mathbb{R}^d$, and a symmetric positive definite $\Sigma \in \mathbb{R}^{d\times d}$, the multivariate Gaussian (multivariate normal) density is

$$\mathcal{N}(x \mid \mu, \Sigma) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\Big(-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)\Big).$$

Σ is the covariance matrix and Σ^{-1} the precision matrix. The normalizer can be written $(2\pi)^{d/2}|\Sigma|^{1/2} = \sqrt{|2\pi\Sigma|}$.

A covariance matrix is in general only positive semidefinite; the density requires Σ^{-1}, hence positive definiteness. Multivariate Gaussians pervade machine learning: generative versus discriminative classification, principal components analysis and autoencoders, and Gaussian process regression.

6.2 From independent coordinates to the general case

Consider height $X_h \sim \mathcal{N}(\mu_h, \sigma_h^2)$ and weight $X_w \sim \mathcal{N}(\mu_w, \sigma_w^2)$. Each sample is a point of a two-dimensional distribution $p([x_h, x_w])$, whose mean is the vector of means, $\mu = [\mu_h, \mu_w] = [\mathbb{E}[x_h], \mathbb{E}[x_w]]$. Its spread is not a single number, and the product $\mathcal{N}(x_h; \mu_h, \sigma_h^2)\,\mathcal{N}(x_w; \mu_w, \sigma_w^2)$ is the joint density only if the two are independent. Height and weight are correlated: the point cloud is an ellipse tilted away from the axes.

Independent case. For independent $X \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $Y \sim \mathcal{N}(\mu_2, \sigma_2^2)$, multiply the densities and collect the exponent as a quadratic form:

$$\begin{aligned} p([x,y]) &= \frac{1}{2\pi\sqrt{\sigma_1^2\sigma_2^2}} \exp\Big[-\frac{(x-\mu_1)^2}{2\sigma_1^2} - \frac{(y-\mu_2)^2}{2\sigma_2^2}\Big] \\ &= \frac{1}{2\pi\sqrt{\sigma_1^2\sigma_2^2}} \exp\Big(-\frac{1}{2}\begin{bmatrix} x-\mu_1 & y-\mu_2 \end{bmatrix} \begin{bmatrix} \sigma_1^2 & 0 \\ 0 & \sigma_2^2 \end{bmatrix}^{-1} \begin{bmatrix} x-\mu_1 \\ y-\mu_2 \end{bmatrix}\Big), \end{aligned}$$

using $\operatorname{diag}(\sigma_1^2, \sigma_2^2)^{-1} = \operatorname{diag}(1/\sigma_1^2, 1/\sigma_2^2)$. This is $\mathcal{N}(\cdot \mid \mu, \Sigma)$ with diagonal $\Sigma = \operatorname{diag}(\sigma_1^2, \sigma_2^2)$, since $|\Sigma| = \sigma_1^2\sigma_2^2$.

Correlated case. If we rotate the coordinate system so that it is axis aligned with the ellipse, the new coordinates are independent and the independent case applies. A rotation is multiplication by an orthogonal matrix Q, with $Q^T = Q^{-1}$ and $\det Q = 1$: write $x' = Q(x - \mu)$, and suppose x' has independent components with variances $\Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_d)$. Then

$$x'^T \Lambda^{-1} x' = (x-\mu)^T Q^T \Lambda^{-1} Q (x-\mu) = (x-\mu)^T \Sigma^{-1} (x-\mu) \quad \text{for} \quad \Sigma = Q^T \Lambda Q,$$

and $|\Sigma| = |\Lambda|$. So the general density is the independent one in rotated coordinates; the axes of the ellipse are the columns of Q^T, the eigenvectors of Σ.

6.3 Expectation, variance, and covariance

The EXPECTATION is $\mathbb{E}[X] = \sum_x x\, p(x)$ for discrete and $\int x\, p(x)\, dx$ for continuous X, often denoted μ; more generally $\mathbb{E}_X[h(X)] = \int h(x)p(x)\, dx$. It is linear, $\mathbb{E}[\sum_i \alpha_i X_i] = \sum_i \alpha_i \mathbb{E}[X_i]$, without any independence assumption, and $\mathbb{E}[X + c] = \mathbb{E}[X] + c$ for a constant c. For independent $X_1, \ldots, X_n$, $\mathbb{E}[\prod_i X_i] = \prod_i \mathbb{E}[X_i]$.

The VARIANCE of X with mean μ is $\operatorname{Var}(X) = \mathbb{E}[(X-\mu)^2]$. It satisfies $\operatorname{Var}(X) = \mathbb{E}[X^2] - \mu^2$ and $\operatorname{Var}(aX + b) = a^2 \operatorname{Var}(X)$, and for independent $X_1, \ldots, X_n$ and constants α_i, $\operatorname{Var}(\sum_i \alpha_i X_i) = \sum_i \alpha_i^2 \operatorname{Var}(X_i)$.

Definition 6.2 [COVARIANCE AND CORRELATION]. The COVARIANCE of X and Y is

$$\operatorname{Cov}(X, Y) = \mathbb{E}\big[(X - \mathbb{E}X)(Y - \mathbb{E}Y)\big] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y],$$

the expected product of the two centered (mean-zero) variables. The CORRELATION is $\operatorname{Corr}(X, Y) = \operatorname{Cov}(X, Y)/\sqrt{\operatorname{Var}(X)\operatorname{Var}(Y)} \in [-1, 1]$.

Note $\operatorname{Cov}(X, X) = \operatorname{Var}(X)$, and independence implies $\operatorname{Cov}(X, Y) = 0$.

6.4 The covariance matrix and independence

For $X = [X_1, \ldots, X_d]$ the covariance matrix has entries $\Sigma_{ij} = \operatorname{Cov}(X_i, X_j)$, so its diagonal holds the variances $\Sigma_{ii} = \sigma_i^2$. It is symmetric, since $\operatorname{Cov}(X_i, X_j) = \operatorname{Cov}(X_j, X_i)$, and positive semidefinite. In the independent example above, $\Sigma = \operatorname{diag}(\sigma_1^2, \sigma_2^2)$ because $\operatorname{Cov}(X, Y) = 0$.

Proposition 6.3 [ZERO COVARIANCE FOR GAUSSIANS]. *If $X \in \mathbb{R}^d$ is multivariate Gaussian, then for all i, j, $\operatorname{Cov}(X_i, X_j) = 0$ if and only if X_i and X_j are independent. For general distributions only one direction holds: independence implies zero covariance.*

The intuition is the independent case read backwards: a diagonal covariance makes the quadratic form a sum of separate squares, so the density factors into one-dimensional Gaussians.

6.5 Level sets of the density are ellipses

The density has two parts: the quadratic form in the exponent, where most of the action happens, and the normalizing constant, which makes

$$\frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \int_{-\infty}^{\infty} \cdots \int_{-\infty}^{\infty} \exp\Big(-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)\Big)\, dx_1 \cdots dx_d = 1.$$

A level set (contour) of the density is a set where $p(x)$ is constant, that is, where the quadratic form is constant. With $\mu = 0$ this is $x^T\Sigma^{-1}x = \text{const}$; for $d = 2$ it reads $ax_1^2 + bx_1x_2 + cx_2^2 = \text{const}$, an ellipse. In higher dimensions the level sets are ellipsoids, which one can explore by plotting each pair of variables x_i, x_j (Figure 6).

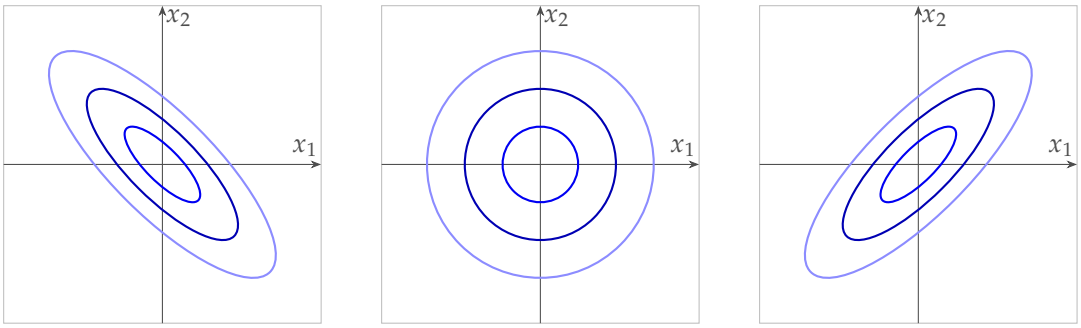


Figure 6: Level sets $x^T\Sigma^{-1}x = c^2$, $c = 0.5, 1, 1.5$, for $\Sigma = \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix}$ with covariance $\rho = -0.8, 0, 0.8$ between x_1 and x_2. The ellipse axes lie along $(1, \pm 1)$, the eigenvectors of Σ, with semi-axes $c\sqrt{1 \pm \rho}$.

6.6 Gaussian mixture models

Definition 6.4 [Gaussian mixture model]. A Gaussian mixture model (GMM, or mixture of Gaussians, MoG) with K components defines the density of a d-dimensional vector x as a weighted combination of Gaussians,

$$p(x \mid \alpha, \mu_1, \Sigma_1, \ldots, \mu_K, \Sigma_K) = \sum_{k=1}^{K} \alpha_k \, \mathcal{N}(x \mid \mu_k, \Sigma_k),$$

where $\alpha_k \geq 0$ and $\sum_{k=1}^{K} \alpha_k = 1$ is a probability distribution over clusters. The α_k are the mixing weights (Figure 7).

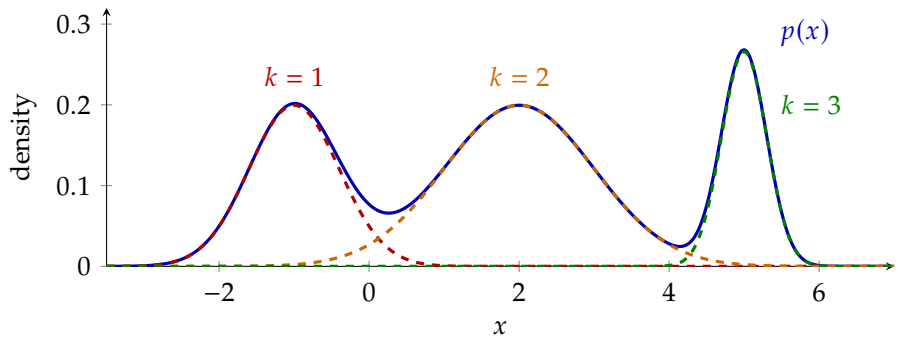


Figure 7: A one-dimensional mixture with $K = 3$ (illustrative parameters $\alpha = (0.3, 0.5, 0.2)$, $\mu = (-1, 2, 5)$, $\sigma = (0.6, 1, 0.3)$). Dashed: weighted components $\alpha_k \mathcal{N}(x \mid \mu_k, \sigma_k^2)$; solid: their sum $p(x)$.

For clustering, each cluster is now a Gaussian $\mathcal{N}(x \mid \mu_k, \Sigma_k)$ with its own mean and covariance, and the model has a likelihood that we can maximize. Let $z_i \in \{1, \dots, K\}$ be the unobserved cluster of x_i, a LATENT VARIABLE. Since its value is unknown, marginalize it out:

$$L_i = p(x_i \mid \theta) = \sum_{k=1}^{K} p(x_i, z_i = k \mid \theta) = \sum_{k=1}^{K} p(x_i \mid z_i = k, \theta)\, p(z_i = k \mid \theta)$$
$$= \sum_{k=1}^{K} \mathcal{N}(x_i \mid \mu_k, \Sigma_k)\, \alpha_k, \qquad \alpha_k \equiv p(z_i = k).$$

The parameters are $\theta = \{\mu_k, \Sigma_k, \alpha_k\}_{k=1}^{K}$, and MLE maximizes

$$LL = \log \prod_i L_i = \sum_i \log L_i = \sum_{i=1}^{N} \log \sum_{k=1}^{K} \alpha_k\, \mathcal{N}(x_i \mid \mu_k, \Sigma_k).$$

Because of the logarithm of a sum there is no closed-form solution; an iterative algorithm such as gradient descent is needed.

Once θ is estimated, $p(x \mid \theta)$ can be evaluated at any point, in the training data or not, so the model also performs DENSITY ESTIMATION. It is a GENERATIVE MODEL: it can produce new data $x \sim p(x \mid \theta)$.

Algorithm 6.1: Sampling from a Gaussian mixture

Input: Parameters $\theta = \{\mu_k, \Sigma_k, \alpha_k\}_{k=1}^{K}$; number of points n.
Output: Samples $x_1, \dots, x_n \sim p(x \mid \theta)$.

for $j \leftarrow 1$ **to** n **do**
  sample cluster $c_j \sim \text{Multinomial}(\alpha_1, \dots, \alpha_K)$
  sample $x_j \sim \mathcal{N}(\mu_{c_j}, \Sigma_{c_j})$
return $x_1, \dots, x_n$

6.7 k-means versus mixtures of Gaussians

In the zero-noise limit, with the variances of the Gaussians shrinking to zero, the mixture of Gaussians yields k-means. A separate covariance per cluster is usually helpful: it captures clusters of different sizes and shapes that k-means misassigns. The mixture states its assumptions explicitly as distributions, so it is easier to generalize while understanding those assumptions, and it gives a principled objective, a likelihood that marginalizes over the latent cluster assignment. A special form of MLE for latent-variable models, EXPECTATION MAXIMIZATION, is an alternative to generic gradient methods.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 6 (Spring 2026).

Lecture 7: Mixtures of Gaussians and Linear Regression

7.1 Fitting a mixture by gradient ascent

The mixture log-likelihood of Lecture 6,

$$LL(\theta) = \sum_i LL_i = \sum_i \log \sum_{k=1}^{K} \mathcal{N}(x_i \mid \mu_k, \Sigma_k)\, \alpha_k,$$

has no closed-form maximizer: setting $\partial LL/\partial \mu_k$, $\partial LL/\partial \Sigma_k$, and $\partial LL/\partial \alpha_k$ to zero cannot be solved analytically. Instead we climb the likelihood surface iteratively. Recall that the gradient of $f : \mathbb{R}^d \to \mathbb{R}$ is the vector $\nabla f(x) = \left(\partial f/\partial x_1, \ldots, \partial f/\partial x_d\right)^T$, the direction of maximal increase of f at x. Gradient descent is developed later in the course; in a nutshell, for maximizing LL:

Algorithm 7.1: Gradient ascent on the log-likelihood

Input: Objective LL, initial parameters $\theta_0 \in \mathbb{R}^d$, step size $\alpha > 0$.
Output: Approximate maximizer of LL.

$t \leftarrow 0$
while θ_t has not converged **do**
  $\theta_{t+1} \leftarrow \theta_t + \alpha \nabla_\theta LL(\theta_t)$
  $t \leftarrow t + 1$
return θ_t

The fitted mixture recovers clusters whose shapes k-means cannot, such as two elongated, tilted point clouds: each component's covariance matches the oblong or elliptical shape of its cluster, whereas k-means implicitly assumes round clusters.

7.2 Regression

In supervised learning we observe pairs $D = \{(x_i, y_i)\}$, where x_i may be discrete and/or continuous. In regression the label is real valued, $y_i \in \mathbb{R}$. Formally we want the conditional density $p(y \mid x)$; a point prediction is its mean, $\hat{y} = \mathbb{E}_Y[\, p(Y \mid X = x)\,]$. Examples include infection rates from zip code and vaccination rate, binding of a protein to a drug target, blood pressure from genetics, an object's location in the next video frame, and housing prices.

The data come from some joint distribution, $(X, Y) \sim p(x, y)$. There are two strategies for estimating $p(y \mid x)$.

1. Generative: estimate the joint $p(x, y \mid \theta)$, for example with a multivariate Gaussian over (X, Y),

then condition:

$$p(y \mid x, \hat{\theta}) = \frac{p(y, x \mid \hat{\theta})}{p(x \mid \hat{\theta})} = \frac{p(y, x \mid \hat{\theta})}{\int_y p(y, x \mid \hat{\theta})\, dy}.$$

2. Discriminative: treat the inputs as fixed, model only the output as random, and fit the conditional $p(y \mid x, \theta)$ directly.

Definition 7.1 [Linear regression]. Linear regression is discriminative, with predictions that are a linear function of the parameters:

$$\hat{y} = \mathbb{E}_Y[p(y \mid x)] = w^T x + w_0, \qquad w, x \in \mathbb{R}^d.$$

w_0 is the offset, bias, or intercept.

In higher dimensions the predicted values form a hyperplane in $\mathbb{R}^{d+1}$. As book-keeping, add a feature that is always 1: with $x' = [x, 1]$, the model is $\hat{y} = w^T x'$, and we drop the separate intercept from now on.

7.3 Basis expansions

A linear model is more flexible than it looks, because "linear" refers to the parameters, not the inputs. With $w, x \in \mathbb{R}$, the models $\hat{y} = w^T x$, $\hat{y} = w^T[x, x^2]$, and $\hat{y} = w^T[x, x^2, x^3]$ fit a line, a parabola, and a cubic. For $x \in \mathbb{R}^D$ an arbitrary polynomial also needs the bias and cross terms, as in the quadratic features $[1, x_1, x_2, x_1^2, x_2^2, x_1 x_2]$.

Definition 7.2 [Basis expansion]. A basis expansion of the raw inputs is a fixed map $\Phi : \mathbb{R}^d \to \mathbb{R}^k$, and the model becomes

$$\hat{y} = \mathbb{E}_Y[p(y \mid x)] = w^T \Phi(x), \qquad w \in \mathbb{R}^k,\ x \in \mathbb{R}^d.$$

For $d = 2$, the polynomial expansion of order 2 is $[x_1, x_2] \mapsto [1, x_1, x_2, x_1 x_2, x_1^2, x_2^2] \in \mathbb{R}^6$. For $d = 1$, the quadratic expansion is $\Phi(x) = [1, x, x^2]$ ($k = 3$) and the cubic is $\Phi(x) = [1, x, x^2, x^3]$ ($k = 4$). The identity expansion $\Phi(x) = x$ has $k = d$. Other one-dimensional bases include

- polynomial, $\phi_j(x) = x^j$;
- radial basis functions, $\phi_j(x) = \exp\left(-(x - \mu_j)^2 / r\right)$;
- sinusoidal, $\phi_j(x) = \sin(x_j)$ or $\cos(x_j)$; and Fourier bases, wavelets, and more.

Because Φ is chosen in advance, the change is only notational: it is as if someone handed us different raw inputs. Expansions add modeling power. Fitting a sinusoid-shaped data set with $y = f(x \mid w) = \sum_{j=0}^{5} w_j x^j$ gave mean squared error 0.013, against 0.511 for a straight line. In what follows we assume the expansion is already done and write $\hat{y} = w^T x$.

7.4 Gaussian linear regression and maximum likelihood

Standard linear regression takes the conditional distribution to be Gaussian,

$$p(y \mid x) = \mathcal{N}(y \mid w^T x, \sigma^2), \qquad \text{equivalently} \qquad Y = w^T x + \epsilon, \quad \epsilon = Y - w^T x \sim \mathcal{N}(0, \sigma^2).$$

For every value $X = x$, the target Y is Gaussian around the line with the same variance σ^2 (Figure 8). Other choices give the RESIDUAL ϵ heavier tails, with more mass far from the center; heavy-tailed noise such as $\epsilon \sim \text{Cauchy}(0, \sigma^2)$ models outliers better than a Gaussian.

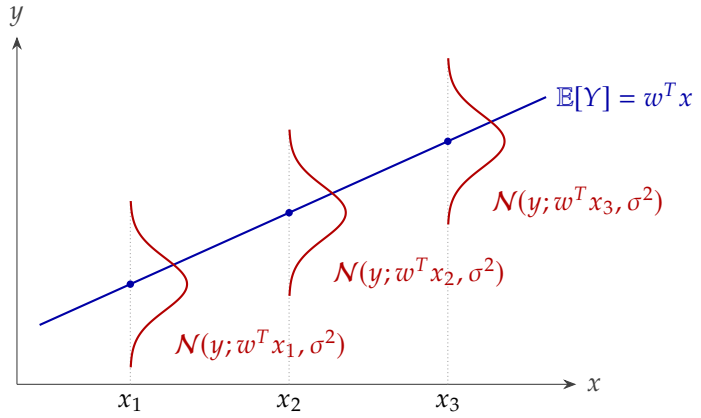


Figure 8: Gaussian linear regression: at each input x_j, Y has density $\mathcal{N}(y; w^T x_j, \sigma^2)$ centered on the line $\mathbb{E}[Y] = w^T x$, with the same spread everywhere. Schematic.

Proposition 7.3 [LEAST SQUARES FROM GAUSSIAN MLE]. *For $p(y \mid x) = \mathcal{N}(y \mid w^T x, \sigma^2)$ and data $D = \{(x_i, y_i)\}_{i=1}^n$,*

$$w_{\text{MLE}} = \arg\min_w \sum_{i=1}^n (y_i - w^T x_i)^2 = \arg\min_w \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

Proof. Treating the inputs as fixed,

$$(w_{\text{MLE}}, \sigma^2_{\text{MLE}}) = \arg\max_{w,\sigma^2} \sum_{i=1}^n \log p(y_i \mid x_i, \theta) = \arg\max_{w,\sigma^2} \sum_{i=1}^n \log \mathcal{N}(y_i \mid w^T x_i, \sigma^2)$$

$$= \arg\max_{w,\sigma^2} \; n \log \frac{1}{\sqrt{2\pi\sigma^2}} - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - w^T x_i)^2.$$

The first term does not involve w, and the factor $1/(2\sigma^2) > 0$ does not change where the second term is smallest. □

So the LEAST SQUARES loss arises naturally from conditional Gaussian MLE.

7.5 Solving least squares with vector calculus

Stack the inputs as the rows of the DESIGN MATRIX $A \in \mathbb{R}^{n\times d}$ and the labels into $y \in \mathbb{R}^n$. Since $w^T x_i = x_i^T w$ is a scalar,

$$\begin{bmatrix} y_1 - w^T x_1 \\ \vdots \\ y_n - w^T x_n \end{bmatrix} = y - \begin{bmatrix} x_1^T \\ \vdots \\ x_n^T \end{bmatrix} w = y - Aw,$$

and the objective becomes $\arg\min_w (y - Aw)^T (y - Aw) = \arg\min_w \|y - Aw\|_2^2$. Expanding, and using that $y^T Aw = w^T A^T y$ is a scalar,

$$\mathcal{L} = \|y - Aw\|_2^2 = y^T y - w^T A^T y - y^T Aw + w^T A^T Aw = y^T y - 2w^T A^T y + w^T A^T Aw.$$

Two gradient rules handle this. For $a, b \in \mathbb{R}^d$, $\partial(a^T b)/\partial a_j = b_j$, so

$$\frac{\partial(a^T b)}{\partial a} = \frac{\partial(b^T a)}{\partial a} = b \in \mathbb{R}^d$$

(this fails for the matrix ab^T). For $x \in \mathbb{R}^d$ and $\Sigma \in \mathbb{R}^{d\times d}$,

$$\frac{\partial(x^T \Sigma x)}{\partial x} = (\Sigma + \Sigma^T)x, \quad \text{which is } 2\Sigma x \text{ if } \Sigma = \Sigma^T,$$

the analogue of $\partial(ax^2)/\partial x = 2ax$.

Theorem 7.4 [NORMAL EQUATIONS]. *$\nabla_w \mathcal{L} = -2A^T y + 2A^T Aw$. Setting it to zero gives the* NORMAL EQUATIONS *$A^T Aw = A^T y$, and if $A^T A$ is invertible,*

$$w_{\text{MLE}} = (A^T A)^{-1} A^T y.$$

This is the unique minimizer when A has full column rank, and then $\sigma^2_{\text{MLE}} = \frac{1}{n}\sum_i (y_i - w^T x_i)^2$, the mean squared residual.

Proof. Apply the rules with $b = A^T y$ and the symmetric $\Sigma = A^T A$ to get the gradient. The Hessian is $\nabla_w^2 \mathcal{L} = 2A^T A$, and $v^T A^T Av = \|Av\|^2$ is positive for every $v \neq 0$ exactly when the columns of A (the features) are linearly independent. Then $\mathcal{L}$ is strictly convex and the critical point is its minimum. The variance follows as in the univariate Gaussian MLE of Lecture 5, applied to the residuals $y_i - w^T x_i$. □

A is rectangular, so it has no inverse; $(A^T A)^{-1} A^T$ is a LEFT PSEUDO-INVERSE, since $(A^T A)^{-1} A^T A = I$. More generally, the MOORE–PENROSE INVERSE A^+ gives $w = A^+ y$ even when A is not full rank, that is, when A has dependent feature vectors; it yields the w_{MLE} of minimum norm $\|w_{\text{MLE}}\|_2$.

With a basis expansion, nothing changes except the design matrix: its rows are $\Phi(x_i)^T$ instead of x_i^T, for example $\Phi(x_1, x_2) = [1, x_1, x_2, x_1 x_2, x_1^2, x_2^2]$.

7.6 The geometric view

Ideally every label is predicted exactly, $y_i = w^T x_i$ for all i, that is, $y = Aw$. Generally this is impossible because of noise or an incorrect model (missing features, say). Write $\mathbb{X} = A \in \mathbb{R}^{n \times d}$ with columns $\mathbb{X}_{:,j}$.

Definition 7.5 [Span]. The span (or column space) of $\mathbb{X}$, written span($\mathbb{X}$), is the set of all linear combinations of its columns: the vectors one can reach using the columns. It is a subspace of $\mathbb{R}^n$.

The matrix-vector product can be read by rows, one prediction $x_i^T w$ per data point, or by columns:

$$\hat{\mathbb{Y}} = \mathbb{X}w = \sum_j w_j \mathbb{X}_{:,j},$$

a linear combination of the feature vectors scaled by the parameters. So the prediction vector always lies in span($\mathbb{X}$), even when y does not, and the goal is the vector of span($\mathbb{X}$) closest to y. That means minimizing the length of the residual vector $e = y - \mathbb{X}w$, $\|e\|_2 = (\sum_i e_i^2)^{1/2}$, which happens when $\hat{\mathbb{Y}}$ is the orthogonal projection of y onto span($\mathbb{X}$) (Figure 9).

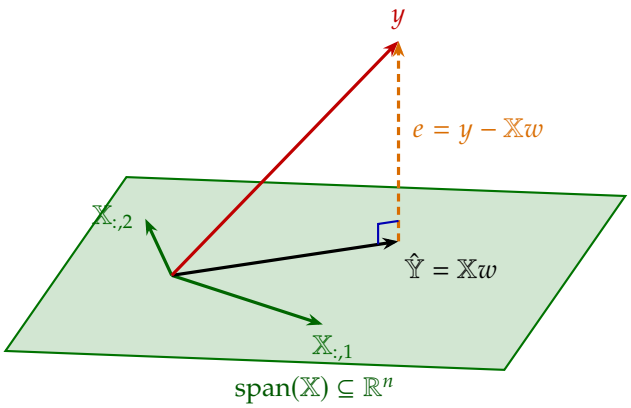


Figure 9: Least squares as projection. The prediction $\hat{\mathbb{Y}} = \mathbb{X}w$ ranges over the plane span($\mathbb{X}$); the best one is the foot of the perpendicular from y, and the residual $e = y - \mathbb{X}w$ is orthogonal to every column.

The residual is then orthogonal to every column of $\mathbb{X}$, which recovers the normal equations of maximum likelihood:

$$\mathbb{X}^T e = 0 \implies \mathbb{X}^T(y - \mathbb{X}w^*) = 0 \implies \mathbb{X}^T y = \mathbb{X}^T \mathbb{X} w^* \implies w^* = (\mathbb{X}^T \mathbb{X})^{-1} \mathbb{X}^T y \quad \text{if } \mathbb{X}^T \mathbb{X} \text{ is invertible.}$$

7.7 What can go wrong

As higher and higher order polynomial features are added:

1. the number of features d grows;
2. for $d \geq n$ the model can fit any data perfectly, since polynomials are a complete basis;
3. even without a perfect fit, the model is in danger of OVERFITTING: worse prediction on a test set. The goal is not to pass through the training data but to do well on unseen cases.

The two main fixes are to remove features until the problem is well behaved (FEATURE SELECTION), or to keep the features and add constraints that tighten the system (REGULARIZATION, Lecture 8).

A held-out test error says nothing about data from a different distribution. Suppose a house-price model with 100 parameters, fit by MLE to millions of sales, predicts held-out sale prices within ±$1000. That does not guarantee the same accuracy on any future test set: the model captures correlations in past data, not the causes of prices. Zillow's house-flipping business, built on such predictions, expected losses of more than $500 million in 2021.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 7 (Spring 2026).

Lecture 8: Linear Regression

This lecture finishes the least-squares solution of Lecture 7, asks when it breaks down, and introduces regularization, its Bayesian interpretation, and how to tune it. Throughout, $\mathbb{X} = A \in \mathbb{R}^{N\times d}$ is the design matrix and $y \in \mathbb{R}^N$ the labels. With a basis expansion Φ, the rows of the design matrix are $\Phi(x_i)^T$ and

$$w = \big(\Phi(\mathbb{X})^T\Phi(\mathbb{X})\big)^{-1}\Phi(\mathbb{X})^T y.$$

8.1 When the normal equations fail

$\text{span}(\mathbb{X})$ is a subspace of $\mathbb{R}^N$ of dimension $\text{rank}(\mathbb{X}) \le d$. The solution $w^* = (\mathbb{X}^T\mathbb{X})^{-1}\mathbb{X}^T y$ requires the $d \times d$ matrix $\mathbb{X}^T\mathbb{X}$ to be invertible, that is, to have full rank d; equivalently, it is positive definite, with all eigenvalues positive.

- This fails when the d features are linearly dependent on the given data $\{x_j\}$; then $\text{rank}(\mathbb{X}^T\mathbb{X}) < d$.
- In particular it fails when $d > N$, since then $\text{rank}(\mathbb{X}^T\mathbb{X}) \le N < d$.
- When $d = N$ and the features are independent, $\mathbb{X}$ is square and invertible, and the fit interpolates: a polynomial with as many coefficients as data points passes through every point.

Even short of a perfect fit, many features invite overfitting. As in Lecture 7, the fixes are feature selection or regularization.

8.2 Infinitely many maximum likelihood solutions

Suppose two features are linearly dependent on the training data, $\alpha x_1 = x_2$ with $\alpha \neq 0$, and $\hat{w}$ is one MLE solution. For every training point and every $\beta \in \mathbb{R}$,

$$\begin{aligned}\hat{y} = x^T\hat{w} = \hat{w}_1 x_1 + \hat{w}_2 x_2 = (\hat{w}_1 + \hat{w}_2\alpha)x_1 &= \big((\hat{w}_1 + \beta) + (\hat{w}_2\alpha - \beta)\big)x_1 \\ &= (\hat{w}_1 + \beta)x_1 + (\hat{w}_2\alpha - \beta)\frac{x_2}{\alpha} = x^T\tilde{w}, \qquad \tilde{w} = \begin{bmatrix}\hat{w}_1 + \beta \\ (\hat{w}_2\alpha - \beta)/\alpha\end{bmatrix}.\end{aligned}$$

So every $\tilde{w}$ makes the same predictions and has the same error: the MLE is not unique.

Of these, choose the one with least norm $\|w_{\text{MLE}}\|_2$. We would like each feature to affect the outcome as little as possible while still predicting well, so that the model behaves gracefully. If an input is perturbed to $x' = x + \delta$, the prediction $\hat{y} = w^T x$ changes by $w^T\delta$, and $|w^T\delta| \le \|w\|_2\|\delta\|_2$; with smaller coefficients the model is less sensitive to noisy data. The same holds for non-degenerate regression, where $A^T A$ is invertible: for many problems and models a small parameter norm is a

good idea. A norm penalty is one form of REGULARIZATION: in effect it reduces the number of free parameters while keeping the same set of parameters.

8.3 Ridge regression

Recall from Lecture 5 that MLE returns a single point estimate, which can be a non-robust spike of the likelihood. To shrink w below the norm of the MLE solution, add a penalty to the loss.

Definition 8.1 [RIDGE REGRESSION]. L_2-REGULARIZED linear regression, also called RIDGE REGRESSION, solves

$$w_{L_2} = \arg\min_w (y - Aw)^T(y - Aw) + \lambda\|w\|_2^2, \qquad \lambda > 0.$$

Proposition 8.2 [RIDGE SOLUTION]. *$w_{L_2} = (A^TA + \lambda I)^{-1}A^Ty$, and $A^TA + \lambda I$ is invertible for every $\lambda > 0$.*

Proof. Write $\lambda\|w\|_2^2 = \lambda w^T I w$ and use the gradient rules of Lecture 7:

$$\nabla_w \mathcal{L} = -2A^Ty + 2A^TAw + 2\lambda Iw = 0 \implies A^Ty = (A^TA + \lambda I)w.$$

The symmetric matrix A^TA has an eigendecomposition $A^TA = QDQ^T$ with Q orthogonal and eigenvalues $d_i = \|Aq_i\|^2 \geq 0$. Then $A^TA + \lambda I = Q(D + \lambda I)Q^T$ has eigenvalues $d_i + \lambda > 0$, so its inverse $Q(D + \lambda I)^{-1}Q^T$ exists. The objective is strictly convex, so this critical point is the unique minimum. □

When some features are linearly dependent, the negative log-likelihood is flat along a line: infinitely many equally good solutions form a *flat ridge*, and $(A^TA + 0)^{-1}$ does not exist. The penalty bends the ridge into a bowl with one stable optimum, which explains the name.

8.4 Bayesian modeling and MAP estimation

Ridge regression has a Bayesian interpretation. Bayesians put a PRIOR DISTRIBUTION $p(\theta)$ on the parameters and compute the POSTERIOR DISTRIBUTION by Bayes' rule,

$$p(\theta \mid D) = \frac{p(D \mid \theta)\, p(\theta)}{p(D)}.$$

Predictions average over the posterior, giving the PREDICTIVE DISTRIBUTION

$$p(y \mid x) = \int_\theta p(y \mid x, \theta)\, p(\theta \mid D)\, d\theta = \mathbb{E}_\theta[p(y \mid x, \theta)].$$

This is difficult in practice, because the normalizer $p(D) = \int_\theta p(D, \theta)\, d\theta = \int_\theta p(D \mid \theta)p(\theta)\, d\theta$ is usually intractable. A lazy alternative keeps the prior but returns only the posterior's peak.

Definition 8.3 [MAP ESTIMATE]**.** The MAXIMUM A POSTERIORI (MAP) estimate is

$$\theta_{\mathrm{MAP}} = \arg\max_{\theta} p(\theta \mid D) = \arg\max_{\theta} \frac{p(D \mid \theta)p(\theta)}{p(D)} = \arg\max_{\theta} p(D \mid \theta)\, p(\theta),$$

since $p(D)$ does not depend on θ.

The prior reweights the likelihood, so a narrow spike of $p(D \mid \theta)$ in a region of low prior probability no longer wins, and θ_{MAP} can lie in the broad, robust region.

Theorem 8.4 [GAUSSIAN PRIOR GIVES RIDGE REGRESSION]**.** *For the model $p(y_i \mid x_i, w) = \mathcal{N}(y_i \mid w^T x_i, \sigma^2)$ and the zero-mean prior $p(w) = \mathcal{N}(w; 0, \lambda I)$,*

$$w_{\mathrm{MAP}} = \arg\min_{w} (y - Aw)^T (y - Aw) + \lambda' \|w\|_2^2, \qquad \lambda' = \frac{\sigma^2}{\lambda}.$$

Proof. Take logarithms and switch to minimizing the negative:

$$\begin{aligned}
w_{\mathrm{MAP}} &= \arg\max_{w} \ \log p(D \mid w) + \log \mathcal{N}(w; 0, \lambda I) \\
&= \arg\max_{w} \ \sum_{i=1}^{N} \log \mathcal{N}(y_i \mid w^T x_i, \sigma^2) + \log \mathcal{N}(w \mid 0, \lambda I) \\
&= \arg\min_{w} \ \frac{1}{2\sigma^2}(y - Aw)^T (y - Aw) - \sum_{i=1}^{d} \log\Big[\frac{1}{\sqrt{2\pi\lambda}} \exp\Big(-\frac{(w_i - 0)^2}{2\lambda}\Big)\Big] \\
&= \arg\min_{w} \ \frac{1}{2\sigma^2}(y - Aw)^T (y - Aw) + \frac{1}{2\lambda}\|w\|_2^2,
\end{aligned}$$

dropping terms that do not depend on w (the Gaussian normalizers). Multiplying by $2\sigma^2 > 0$ does not move the minimizer and gives the penalty $(\sigma^2/\lambda)\|w\|_2^2$. □

With this prior the posterior $p(w \mid D)$ is also Gaussian and can be worked out analytically with multivariate Gaussians. A smaller prior variance λ means a stronger penalty λ'.

8.5 Choosing the regularization strength

As λ grows, all ridge coefficients shrink smoothly toward zero. How should λ be set? It cannot be treated as one more parameter of the loss and fit by MLE on the training data: the training loss is always smallest with no penalty. It needs independent data, a VALIDATION SET, on which to evaluate the loss.

1. Split the data into training and test data, and split a validation set off the training data.
2. Fit on the training part for each candidate λ, and pick the value that is best on the validation set.
3. Quantify the final performance on the test data.

Here "best" and "quantify" mean computing a metric such as the log-likelihood on held-out data. k-FOLD CROSS-VALIDATION splits the training data into k folds (say 5); in split j, fold j is the validation set and the other folds are the training data, and the validation results are averaged over the k splits.

8.6 Sparsity and the lasso

To make the linear function depend on only a few features, we want a SPARSE w. Penalizing the number of nonzero weights, the L_0 penalty $\lambda\|w\|_0$, is not differentiable and turns fitting into nasty combinatorial optimization. The L_1 penalty $\lambda\|w\|_1 = \lambda \sum_d |w_d|$ also tends to induce sparse w, and it is differentiable everywhere except at $w_i = 0$, which can be worked around.

Definition 8.5 [LASSO]. LASSO (L_1-penalized) regression solves

$$w_{L_1} = \arg\min_w (y - Aw)^T (y - Aw) + \lambda \|w\|_1.$$

Lasso is equivalent to MLE subject to a constraint $\|w\|_1 \le c$, and ridge to $\|w\|_2^2 \le c$. The L_1 ball is "pointy", with corners jutting out along the axes. In many cases the elliptical contours of the squared error, centered at the unconstrained solution $\hat{\beta}$, first touch the constraint set at a corner, where some coefficients are exactly 0 (Figure 10). The round L_2 ball has no corners, so ridge shrinks coefficients without zeroing them. Accordingly, as λ grows, ridge coefficients decay smoothly while lasso coefficients hit zero one by one, reducing the number of nonzero features: shrinkage versus sparsity.

Lasso also has a MAP interpretation. Ridge arose from a $\mathcal{N}(0, \lambda I)$ prior; the L_1 penalty arises from the LAPLACE PRIOR

$$p(w) = \prod_{i=1}^{d} \frac{1}{2\lambda'} \exp\Big(-\frac{|w_i|}{\lambda'}\Big) \propto \exp\Big(-\frac{\|w\|_1}{\lambda'}\Big),$$

which is more sharply peaked at zero than a Gaussian, since $-\log p(w) = \|w\|_1/\lambda' + \text{const}$. One

issue with the lasso: among highly correlated features it tends to keep one and ignore the rest. Combining the L_1 and L_2 penalties gives ELASTIC NET regression, a compromise with two parameters between sparsity and weight sharing.

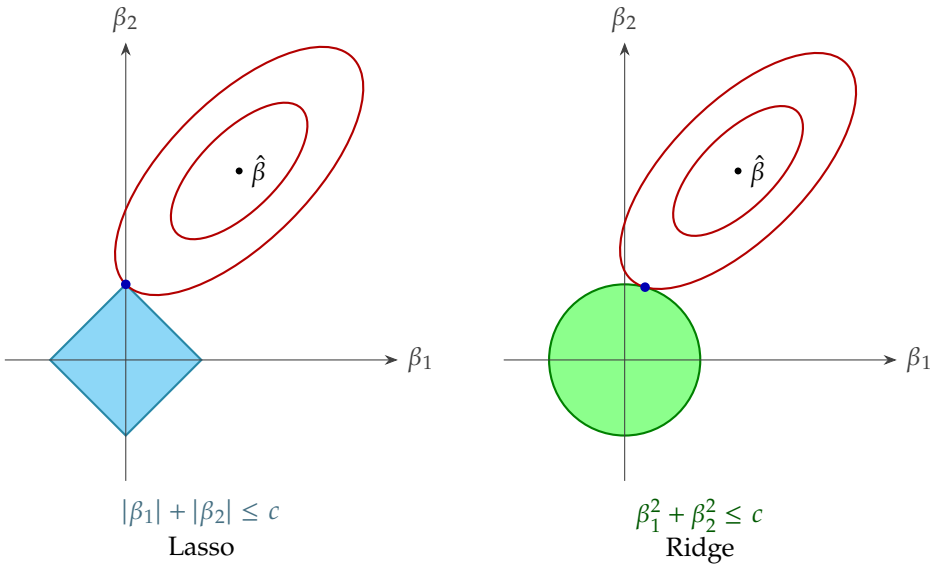


Figure 10: Constrained views of lasso ($|\beta_1| + |\beta_2| \le c$, left) and ridge ($\beta_1^2 + \beta_2^2 \le c$, right). Ellipses are level sets of the squared error around the unconstrained solution $\hat{\beta}$; the constrained optimum (dot) is where they first touch the constraint set. For the lasso this is a corner with $\beta_1 = 0$. Schematic.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 8 (Spring 2026).

Lecture 9: Linear Regression and Regularization

From both the MLE and the geometric view (Lecture 7), least squares on $\mathbb{X} \in \mathbb{R}^{n\times d}$, $y \in \mathbb{R}^n$ gives $w^* = (\mathbb{X}^T\mathbb{X})^{-1}\mathbb{X}^T y \in \mathbb{R}^d$, provided $\operatorname{rank}(\mathbb{X}^T\mathbb{X}) = d$. This fails whenever some features are dependent, which is guaranteed when there are more features than samples and can happen with fewer. The Moore–Penrose inverse $\mathbb{X}^+$ still gives a solution, $w^* = \mathbb{X}^+ y$, and among the infinitely many minimizers of $\|\mathbb{X}w - y\|^2$ it picks the one of smallest norm $\|w^*\|_2$. This lecture explains why, using the singular value decomposition. Ridge regression, MAP estimation, cross-validation, and the lasso are treated in Lecture 8.

9.1 The singular value decomposition

Definition 9.1 [Singular value decomposition]. Every real matrix $\mathbb{X} \in \mathbb{R}^{n\times d}$ can be factored as

$$\mathbb{X} = U\Sigma V^T, \qquad U \in \mathbb{R}^{n\times n}, \quad \Sigma \in \mathbb{R}^{n\times d}, \quad V \in \mathbb{R}^{d\times d},$$

where U and V are rotations with orthonormal columns ($U^TU = V^TV = I$) and Σ is a diagonal scaling,

$$\Sigma = \operatorname{diag}(\sigma_1, \dots, \sigma_r, 0, \dots, 0), \qquad \sigma_1 \geq \cdots \geq \sigma_r > 0, \quad r = \operatorname{rank}(\mathbb{X}).$$

The σ_i are the singular values. This is the singular value decomposition (SVD).

Reading $\mathbb{X}x = U(\Sigma(V^Tx))$ from the right: V^T rotates the right singular vectors v_i (the columns of V) onto the coordinate axes e_i, Σ scales axis i by σ_i, and U rotates the result, so $\mathbb{X}v_i = \sigma_i u_i$. The unit circle becomes an ellipse with semi-axes $\sigma_i u_i$ (Figure 11).

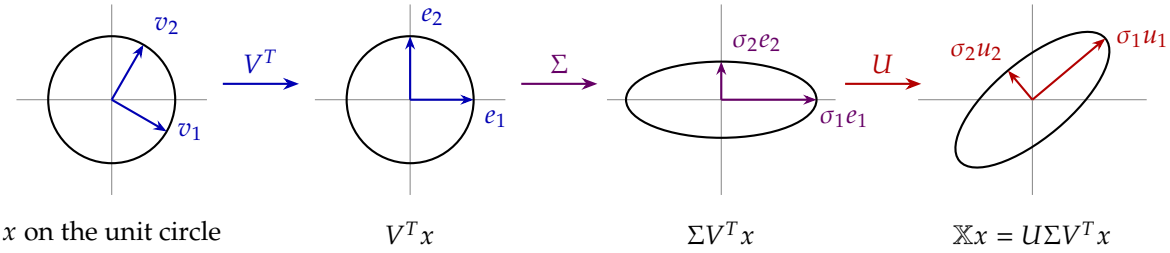


Figure 11: The SVD as rotate, scale, rotate, for a 2×2 matrix: V^T takes v_1, v_2 to e_1, e_2, Σ stretches the axes by σ_1, σ_2, and U rotates $\sigma_i e_i$ to $\sigma_i u_i$. Schematic.

If $\mathbb{X}$ is square and invertible, every $\sigma_i > 0$ and $\mathbb{X}^{-1} = V\Sigma^{-1}U^T$, since $U\Sigma V^T\, V\Sigma^{-1}U^T = U\Sigma\Sigma^{-1}U^T = I$. If some $\sigma_i = 0$, the corresponding direction is collapsed to 0 and cannot be recovered, so $\mathbb{X}$ cannot be inverted directly.

9.2 The Moore–Penrose pseudo-inverse

The pseudo-inverse inverts only what can be inverted: it "flips" the nonzero singular values and leaves the zero ones at zero.

Definition 9.2 [Moore–Penrose pseudo-inverse]. Let $\Sigma^+ \in \mathbb{R}^{d\times n}$ be diagonal with

$$\sigma_{ii}^+ = \begin{cases} 1/\sigma_i, & \sigma_i > 0, \\ 0, & \text{otherwise.} \end{cases}$$

The Moore–Penrose pseudo-inverse of $\mathbb{X} = U\Sigma V^T$ is $\mathbb{X}^+ = V\Sigma^+ U^T \in \mathbb{R}^{d\times n}$.

In floating point, "nonzero" means "not tiny": singular values below a small tolerance are treated as 0, since inverting them would blow up noise.

Proposition 9.3 [Moore–Penrose conditions]. *X^+ satisfies*

$$\text{(1) } XX^+X = X, \qquad \text{(2) } X^+XX^+ = X^+, \qquad \text{(3) } (XX^+)^T = XX^+, \qquad \text{(4) } (X^+X)^T = X^+X,$$

and these four conditions uniquely define the pseudo-inverse of X.

Proof. With $X = U\Sigma V^T$ and $X^+ = V\Sigma^+U^T$, the orthogonal factors cancel: $XX^+ = U(\Sigma\Sigma^+)U^T$ and $X^+X = V(\Sigma^+\Sigma)V^T$, where $\Sigma\Sigma^+$ and $\Sigma^+\Sigma$ are diagonal with r ones followed by zeros. Diagonal matrices are symmetric, which gives (3) and (4), and $\Sigma\Sigma^+\Sigma = \Sigma$, $\Sigma^+\Sigma\Sigma^+ = \Sigma^+$ give (1) and (2). □

Theorem 9.4 [Minimum-norm least squares]. *For any $\mathbb{X} \in \mathbb{R}^{n\times d}$ and $y \in \mathbb{R}^n$, $w^* = \mathbb{X}^+y$ minimizes $\|\mathbb{X}w - y\|_2^2$, and among all minimizers it has the smallest norm $\|w\|_2$.*

Proof. Rotations preserve length. Put $z = V^Tw$ and $c = U^Ty$, so $\|z\| = \|w\|$ and

$$\|\mathbb{X}w - y\|^2 = \|U(\Sigma V^Tw - U^Ty)\|^2 = \|\Sigma z - c\|^2 = \sum_{i=1}^{r}(\sigma_i z_i - c_i)^2 + \sum_{i=r+1}^{n} c_i^2.$$

The second sum does not depend on w. The first is zero exactly when $z_i = c_i/\sigma_i$ for $i \le r$, while $z_{r+1}, \dots, z_d$ are free: these free coordinates are the infinitely many solutions. The norm $\|w\|^2 = \sum_i z_i^2$ is smallest when the free coordinates are 0, that is, $z = \Sigma^+c$. Hence $w^* = Vz = V\Sigma^+U^Ty = \mathbb{X}^+y$. □

The pseudo-inverse agrees with the normal equations of Lecture 7 whenever they apply. If

rank($\mathbb{X}$) = d, all of $\sigma_1, \ldots, \sigma_d$ are positive and $\mathbb{X}^T\mathbb{X} = V\Sigma^T\Sigma V^T = V\,\mathrm{diag}(\sigma_i^2)V^T$, so

$$(\mathbb{X}^T\mathbb{X})^{-1}\mathbb{X}^T = V\,\mathrm{diag}(\sigma_i^{-2})V^T V\Sigma^T U^T = V\,\mathrm{diag}(\sigma_i^{-2})\Sigma^T U^T = V\Sigma^+ U^T = \mathbb{X}^+.$$

So $w^* = \mathbb{X}^+ y$ always works, even with linearly dependent features: with independent features it is the usual least-squares solution, and with dependent ones it selects, from the family of equally good MLE solutions of Lecture 8, the one of least norm.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 9 (Spring 2026).

Lecture 10: Linear Regression and Regularization (continued)

This lecture finishes regularized linear regression (numerical stability, model selection, error metrics, and a last look at L_1 penalties) and then begins classification.

10.1 Regularization and generalization

Definition 10.1 [Regularization]. Regularization is the process of adding constraints or penalties to the learning process to improve generalization.

Fitting flexible models to noisy data, too little regularization overfits: the fitted curves oscillate to follow the noise. Too much underfits: the curves are too rigid to follow the structure of the data. More regularization moves a model from the overfitting regime toward the underfitting one, and the goal is the sweet spot between.

10.2 Ridge regression and numerical stability

Besides reducing overfitting, the ridge penalty makes the solution $w_{L_2} = (A^TA + \lambda I)^{-1}A^Ty$ of Lecture 8 numerically better behaved, even when A^TA is invertible. Write the eigendecomposition $A^TA = QDQ^T$ with Q orthogonal and $D = \operatorname{diag}(\sigma_1, \ldots, \sigma_d)$, $\sigma_1 \geq \cdots \geq \sigma_d \geq 0$.

Definition 10.2 [Condition number]. The condition number of A^TA is $\kappa = \sigma_{\max}/\sigma_{\min}$, with $1 \leq \kappa \leq \infty$.

$\kappa = 1$ is maximally well behaved (robust to perturbations of the data), and $\kappa = \infty$ exactly when $\sigma_{\min} = 0$, that is, when A^TA is singular because features are linearly dependent. A large κ means small changes in y or A can cause large changes in the computed w.

Proposition 10.3 [Ridge improves conditioning]. *For $\lambda > 0$, $A^TA + \lambda I = Q(D + \lambda I)Q^T$ has eigenvalues $\sigma_i + \lambda$ and condition number*

$$\kappa_\lambda = \frac{\sigma_{\max} + \lambda}{\sigma_{\min} + \lambda} < \kappa \qquad \text{whenever } \sigma_{\max} > \sigma_{\min}.$$

Proof. $QQ^T = I$ gives $A^TA + \lambda I = QDQ^T + \lambda QQ^T = Q(D + \lambda I)Q^T$. If $\sigma_{\min} = 0$, $\kappa = \infty$ while κ_λ is finite. Otherwise, cross-multiplying, $\sigma_{\min}(\sigma_{\max} + \lambda) < \sigma_{\max}(\sigma_{\min} + \lambda)$ reduces to $\lambda\sigma_{\min} < \lambda\sigma_{\max}$. □

As $\lambda \to \infty$, $\kappa_\lambda \to 1$, a perfectly conditioned system, but $w_{L_2} \approx A^Ty/\lambda \to 0$: stability is bought with

bias, which is why λ must be chosen with held-out data.

10.3 Model selection and held-out data

We always need MODEL SELECTION: which model class to use (linear regression, a neural network), which features to use in linear regression, which ridge penalty λ (a HYPERPARAMETER), which neural-network architecture. Model selection cannot be done by MLE or optimization on the training data, since a more flexible model or a smaller penalty always fits the training data at least as well. We must use train/validation/test splits, as in Lecture 8.

- Use the validation set to pick the model and hyperparameters.
- The validation error of the winner is optimistic: it is the minimum over many candidates, so by the WINNER'S CURSE it is lower than the true error.
- After picking the model and hyperparameters, use the test set *once* to get an unbiased error estimate.

For example, a hospital model predicting diagnoses from chest x-rays should be tuned on validation images and reported on test images it has never influenced.

10.4 Regression metrics

For targets t_n and predictions $y(x_n, w)$, $n = 1, \dots, N$, with means $\bar{t}$ and $\bar{y}$, common metrics are:

Metric	Definition
Log-likelihood	$\frac{1}{N} \sum_{n=1}^{N} \log p(t_n \mid x_n, w)$
Mean squared error (MSE)	$\frac{1}{N} \sum_{n=1}^{N} \big(t_n - y(x_n, w)\big)^2$
Root mean squared error (RMSE)	$\sqrt{\frac{1}{N} \sum_{n=1}^{N} \big(t_n - y(x_n, w)\big)^2}$
Correlation r (or ρ)	$\dfrac{\sum_n (t_n - \bar{t})\big(y(x_n, w) - \bar{y}\big)}{\sqrt{\sum_n (t_n - \bar{t})^2 \; \sum_n \big(y(x_n, w) - \bar{y}\big)^2}}$
R^2	r^2
Mean absolute error (MAE)	$\frac{1}{N} \sum_{n=1}^{N} \lvert t_n - y(x_n, w) \rvert$

If two models achieve exactly the same MSE, they need not be equally good. A probabilistic regression model predicts a whole distribution, $p(y \mid x_n) = \mathcal{N}(y \mid w^T x_n, \sigma^2)$ with mean $y(x_n, w) =$

$\mathbb{E}[y] = w^T x_n$, not just a point. The held-out LOG-LIKELIHOOD gives a richer measure of fit by evaluating how likely the entire observed dataset is under the model's full probabilistic assumptions. For the Gaussian model,

$$\frac{1}{N}\sum_{n=1}^{N} \log \mathcal{N}(t_n \mid w^T x_n, \sigma^2) = -\frac{1}{2}\log(2\pi\sigma^2) - \frac{\text{MSE}}{2\sigma^2},$$

so two models with the same MSE but different noise variances σ^2 receive different log-likelihoods: the one whose stated uncertainty matches the actual errors scores higher.

10.5 Norms, the lasso, and the elastic net

Definition 10.4 [p-NORM]. For $x \in \mathbb{R}^d$ and $p \geq 1$, $\|x\|_p = \left(\sum_{i=1}^{d} |x_i|^p\right)^{1/p}$. The cases $p = 1$ (Manhattan or taxicab norm, $\|x\|_1 = |x_1| + \cdots + |x_d|$), $p = 2$ (Euclidean norm, $\|x\|_2 = \sqrt{x_1^2 + \cdots + x_d^2}$), and $p = \infty$ (max norm, $\|x\|_\infty = \max_i |x_i|$) are the most common. An unsubscripted $\|x\|$ means $\|x\|_2$.

The lasso of Lecture 8, $w_{L_1} = \arg\min_w (y - Aw)^T(y - Aw) + \lambda\|w\|_1$, is equivalent to the constrained problem

$$\arg\min_{w:\ \|w\|_1 < C(\lambda)} \ (y - Aw)^T(y - Aw)$$

for a bound $C(\lambda)$ that shrinks as λ grows; the corners of the L_1 ball produce sparse solutions. The ELASTIC NET adds both penalties, $\lambda_1\|w\|_1 + \lambda_2\|w\|_2^2$. Its constraint region keeps the corners of the L_1 ball (sparsity) but has curved edges like the L_2 ball (weight sharing among correlated features). It is much more expensive to tune than L_1 or L_2 alone: there are two hyperparameters, so cross-validation must search a two-dimensional grid of (λ_1, λ_2) values, refitting the model for every pair and every fold.

10.6 From regression to classification

Regression tries to "draw a line to trace out the data": a regression curve through (x, y) pairs. CLASSIFICATION tries to "draw a line to separate the classes": a DECISION BOUNDARY in input space $(x_1, x_2, \ldots)$, with labels such as $y \in \{1, 2\}$. The further a point is from the boundary, the more confident the prediction.

There are three ways to build a classifier.

1. *Generative*: model the class-conditional densities $p(x \mid C_k)$ and the class priors $p(C_k)$, so $p(x, C_k) = p(x \mid C_k)p(C_k)$, and obtain $p(C_k \mid x) = p(x \mid C_k)p(C_k)/p(x)$ by Bayes' theorem.
2. *Discriminative*: model $p(C_k \mid x)$ directly.
3. *Discriminant function*: find decision boundaries directly, modeling a map $f : x \mapsto k$.

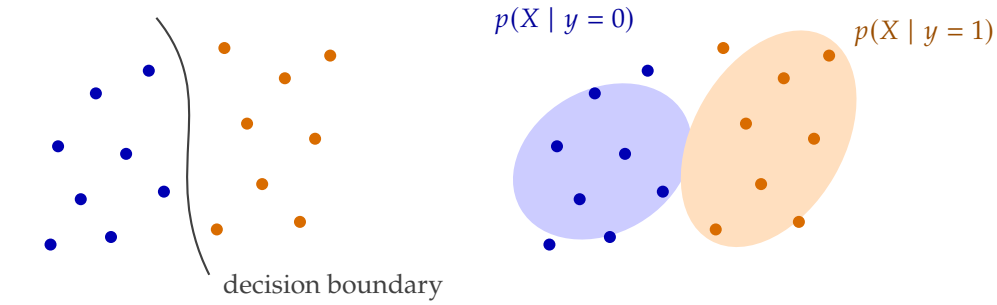
The first two are PROBABILISTIC CLASSIFIERS: they assign a probability $p(C_k \mid x)$ that the input belongs to each class. A directly built scoring function $f(C_k, x)$ gives no natural way to understand how confident a score is, while probabilistic classifiers all operate on the same scale, probability, which can be readily understood and compared across models.

10.7 Discriminative versus generative classifiers

A parent takes two children to a tiny zoo with only lions and elephants and asks them to learn to tell the animals apart. Later, watching a movie, the parent asks which animal is on the screen. Child A, who learns holistically, drew a picture of each animal and compares both pictures with the screen. Child B, who keeps only the most pertinent information, remembers distinguishing features (trunk or no trunk). Child A is generative; child B is discriminative (Figure 12).

Definition 10.5 [DISCRIMINATIVE AND GENERATIVE CLASSIFIERS]. A DISCRIMINATIVE model directly models the discriminative function $\text{score}(y = a) = f(X)$, which may or may not be probabilistic: $f(X) = p(y \mid X)$, for example $p(y = C \mid X, w) = \text{sigmoid}(w^T X)$ (logistic regression), or a "distance to the boundary". A GENERATIVE model is defined by the joint probability, factored as $p(y = C, X) = p(X \mid y = C)\, p(y = C)$, for example Gaussian class conditionals $p(X \mid y = C) = \mathcal{N}(X \mid \mu_C, \Sigma_C)$ with priors $p(y = 1) = 0.22$, $p(y = 0) = 0.78$, and then uses Bayes' rule,

$$p(y = C \mid X) = \frac{p(X \mid y = C)\, p(y = C)}{p(X)}.$$



Discriminative: model $p(y \mid X)$ Generative: model $p(X \mid y)\, p(y)$

Figure 12: A discriminative classifier models only the boundary between the classes; a generative classifier models the density of X within each class and derives the boundary from it. Schematic.

The terms need care: many generative models are not classifiers, many models with "discriminant" in the name are generative (linear discriminant analysis, Lecture 11), and logistic regression does classification, not regression. The two types differ as follows.

- *Capacity.* Discriminative models spend all their parameters on the decision boundary; generative models spend parameters on the full density of X in each class, which implies a discriminant

function. With the same capacity (roughly, number of parameters), a discriminative model can therefore yield more complex decision boundaries.

- *Generating data.* A generative model describes an (idealized) process by which the data were generated, so it can generate new pairs (x_i, y_i), and via Bayes' rule it also has discriminative properties. A discriminative model such as logistic regression does not have full generative properties: it can generate only the y_i given the x_i. Unsupervised models $p(x_i)$ are also often called generative, nowadays even non-probabilistic ones such as GANs.
- *Assumptions.* Generative models make strong, explicit assumptions about the statistics of X (class-conditional Gaussianity, independence among features), which discriminative models capture at most implicitly. Discriminative models therefore tend to be more robust to model misspecification, while generative models can be more accurate when the data adhere to their assumptions.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 10 (Spring 2026).

Lecture 11: Classification

Lecture 10 distinguished generative from discriminative classifiers. This lecture studies the basic generative classifier, Gaussian discriminant analysis: how to fit it, what decision boundaries it implies, and how it relates to logistic regression.

11.1 Gaussian discriminant analysis

Definition 11.1 [Gaussian discriminant analysis]. Gaussian discriminant analysis (GDA) is the generative model $p(X, y) = p(X \mid y)\, p(y)$ that uses Gaussians to model X conditioned on the class. For two classes,

$$p(X \mid y = 1, \mu_1, \Sigma_1) = \mathcal{N}(X \mid \mu_1, \Sigma_1), \qquad p(X \mid y = 0, \mu_0, \Sigma_0) = \mathcal{N}(X \mid \mu_0, \Sigma_0),$$

with a Bernoulli prior $p(y = 1) = \phi$, $p(y = 0) = 1 - \phi$ (for instance $\phi = 0.22$).

Despite its name, GDA is generative, not discriminative. The modeling choices are whether the classes share means (we always assume different means $\mu_1 \neq \mu_2$), whether they share a covariance, and whether the covariance is diagonal or spherical. Three cases recur:

1. two identical spherical (isotropic) Gaussians, $\Sigma_1 = \Sigma_2 = \Sigma = \mathrm{diag}(\sigma, \ldots, \sigma)$;
2. two identical Gaussians of arbitrary (anisotropic) covariance, $\Sigma_1 = \Sigma_2 = \Sigma$;
3. the most general case, with parameters $\{\mu_0, \Sigma_0, \mu_1, \Sigma_1\}$ all different and arbitrary covariances.

All of these variants readily generalize to $K > 2$ classes.

11.2 Fitting GDA by maximum likelihood

Take data from K classes, $X \in \mathbb{R}^{N \times D}$ and $Y \in \{1, \ldots, K\}^N$, and let n_k be the number of points with $y_i = k$. The MLE objective is the joint probability of the training data,

$$\log p(Y, X \mid \{\mu_k, \Sigma_k\}) = \sum_{\text{classes } k} \sum_{i:\, y_i = k} \big(\log \mathcal{N}(x_i \mid \mu_k, \Sigma_k) + \log p(y_i)\big).$$

That it is a likelihood of (Y, X), not of Y given X, is how one knows the model is generative. Conditioned on the class, the likelihood decouples into one Gaussian per class.

Proposition 11.2 [GDA maximum likelihood]. *The MLE fits each class separately, exactly as the MLE for*

a single Gaussian:

$$\hat{\mu}_k = \frac{1}{n_k} \sum_{i:\, y_i = k} x_i, \qquad \hat{\Sigma}_k = \frac{1}{n_k} \sum_{i:\, y_i = k} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^T, \qquad \hat{p}(y = k) = \frac{n_k}{N}.$$

Proof. The parameters (μ_k, Σ_k) appear only in the terms of class k, so each block is maximized on its own. In one dimension, for the n_k points $X_1, \ldots, X_{n_k}$ of class k,

$$\begin{aligned}\hat{\mu}_k, \hat{\sigma}_k^2 &= \arg\max_{\mu_k, \sigma_k^2} \ln P(X_1, \ldots, X_{n_k} \mid \mu_k, \sigma_k^2) = \arg\max_{\mu_k, \sigma_k^2} \sum_{i=1}^{n_k} \Big(-\frac{(X_i - \mu_k)^2}{2\sigma_k^2} - \ln \sigma_k - \frac{1}{2} \ln(2\pi) \Big) \\ &= \arg\min_{\mu_k, \sigma_k^2} \sum_{i=1}^{n_k} \frac{(X_i - \mu_k)^2}{2\sigma_k^2} + \ln \sigma_k.\end{aligned}$$

Both parameters must be found, and a closed form is possible because the equation for μ_k turns out not to involve σ_k:

$$\frac{\partial}{\partial \mu_k} = \sum_{i=1}^{n_k} \frac{-(X_i - \mu_k)}{\sigma_k^2} = 0 \implies \hat{\mu}_k = \frac{1}{n_k} \sum_{i=1}^{n_k} X_i.$$

Given $\hat{\mu}_k$,

$$\frac{\partial}{\partial \sigma_k} = \sum_{i=1}^{n_k} \Big(-\frac{(X_i - \hat{\mu}_k)^2}{\sigma_k^3} + \frac{1}{\sigma_k} \Big) = 0 \implies \hat{\sigma}_k^2 = \frac{1}{n_k} \sum_{i=1}^{n_k} (X_i - \hat{\mu}_k)^2,$$

which depends on the estimate $\hat{\mu}_k$. The multivariate case is the same computation with vector calculus. The prior terms $\sum_i \log p(y_i) = \sum_k n_k \log p(y = k)$ form the multinomial likelihood of Lecture 5, maximized at n_k/N. □

The class prior $p(y = C)$ can be set in two ways: (a) by MLE, the fraction of training data in each class; or (b) as a prior specified by domain knowledge, such as a doctor who knows the prevalence of Crohn's disease. Option (b) can be preferable even with training data, because the class proportions in a collected training set need not match those of the population on which the classifier will be used. Predictions then follow from $p(Y \mid X) \propto p(X \mid Y)\, p(Y)$.

11.3 Decision boundaries

Definition 11.3 [Decision boundary]. The decision boundary between classes a and b is the level set of $X \in \mathbb{R}^D$ where $p(Y = a \mid X) = p(Y = b \mid X)$.

By Bayes' rule the condition is $p(X \mid Y = a)p(Y = a) = p(X \mid Y = b)p(Y = b)$, or after taking logarithms

$$\log \mathcal{N}(X \mid \hat{\mu}_a, \hat{\Sigma}_a) + \log p(Y = a) = \log \mathcal{N}(X \mid \hat{\mu}_b, \hat{\Sigma}_b) + \log p(Y = b). \tag{11.1}$$

Equal priors, identical isotropic covariances $\Sigma_a = \Sigma_b = \sigma^2 I$. The priors cancel, and so do the Gaussian normalizers, leaving

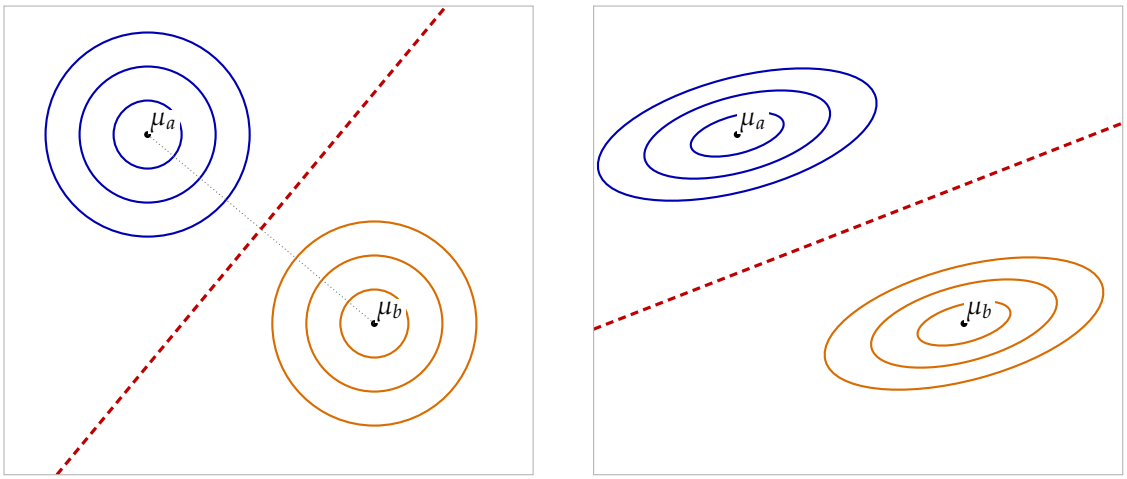
$$-\frac{1}{2\sigma^2}(X-\hat{\mu}_a)^T(X-\hat{\mu}_a) = -\frac{1}{2\sigma^2}(X-\hat{\mu}_b)^T(X-\hat{\mu}_b) \iff \|X-\hat{\mu}_a\|_2 = \|X-\hat{\mu}_b\|_2.$$

The boundary is the set of points equidistant from $\hat{\mu}_a$ and $\hat{\mu}_b$. Expanding, the quadratic terms $X^T X$ cancel and $2(\hat{\mu}_b - \hat{\mu}_a)^T X = \hat{\mu}_b^T \hat{\mu}_b - \hat{\mu}_a^T \hat{\mu}_a$: a line (a hyperplane), the perpendicular bisector of the two means.

Equal priors, identical arbitrary covariance $\Sigma_a = \Sigma_b = \Sigma$. Now the boundary is

$$(X-\hat{\mu}_a)^T \hat{\Sigma}^{-1}(X-\hat{\mu}_a) = (X-\hat{\mu}_b)^T \hat{\Sigma}^{-1}(X-\hat{\mu}_b),$$

where each side has ellipsoidal level sets, with axes shrunken and/or rotated in the same way for both classes. The spectral decomposition $\Sigma = USU^T$ tells us how to rotate and scale X so that its covariance is the identity: $\hat{X} = S^{-1/2}U^T(X-\mu_a)$ has $\mathrm{cov}(\hat{X}) = I$. In these coordinates both classes have identity covariance, and the condition becomes $\|\hat{X} - m_a\|_2 = \|\hat{X} - m_b\|_2$ for the transformed means $m_a = 0$ and $m_b = S^{-1/2}U^T(\mu_b - \mu_a)$: the isotropic case, whose boundary is a line. A linear change of coordinates maps lines to lines, so the boundary is still a line. (Directly: the term $X^T\hat{\Sigma}^{-1}X$ appears on both sides and cancels.)



$\Sigma_a = \Sigma_b = \sigma^2 I$ $\qquad$ $\Sigma_a = \Sigma_b = \Sigma$

Figure 13: GDA decision boundaries (dashed) with equal priors and a shared covariance, drawn exactly for $\mu_a = (-1.5, 1.5)$ and $\mu_b = (1.5, -1)$. Left: spherical covariance; the boundary is the perpendicular bisector of the means. Right: a shared anisotropic covariance (contours rotated 15°); the boundary is still a line, but no longer perpendicular to $\mu_b - \mu_a$.

Unequal priors. With a shared Σ, the terms $\log p(Y = a)$ and $\log p(Y = b)$ in (11.1) add a constant to the system. The boundary remains a line, shifted away from the more probable class.

Different covariances. With $\Sigma_a \neq \Sigma_b$, (11.1) reads

$$\ln P(A) - \frac{1}{2}(X-\hat{\mu}_A)^T\hat{\Sigma}_A^{-1}(X-\hat{\mu}_A) - \frac{1}{2}\ln|\hat{\Sigma}_A| = \ln P(B) - \frac{1}{2}(X-\hat{\mu}_B)^T\hat{\Sigma}_B^{-1}(X-\hat{\mu}_B) - \frac{1}{2}\ln|\hat{\Sigma}_B|.$$

The quadratic terms $X^T\hat{\Sigma}_A^{-1}X$ and $X^T\hat{\Sigma}_B^{-1}X$ no longer cancel, so the decision boundary is a quadratic in X (Figure 14).

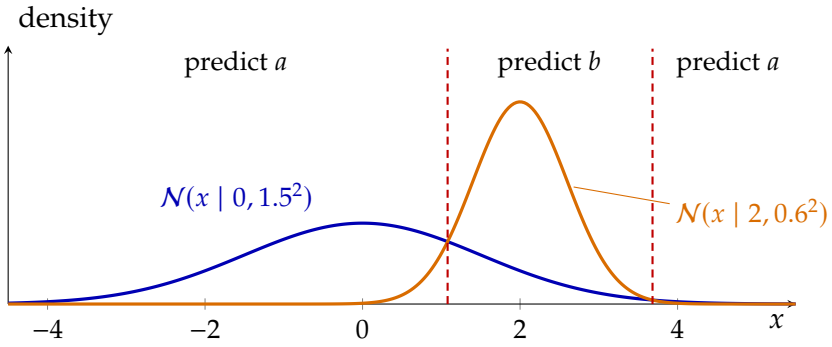


Figure 14: One-dimensional GDA with equal priors and unequal variances, classes $\mathcal{N}(0, 1.5^2)$ and $\mathcal{N}(2, 0.6^2)$. The quadratic boundary has two roots, $x \approx 1.08$ and $x \approx 3.68$ (dashed): the narrow class b wins only between them.

Definition 11.4 [LDA AND QDA]. When the class covariances are equal, the decision boundary is linear: LINEAR DISCRIMINANT ANALYSIS (LDA). When they are unequal, it is quadratic: QUADRATIC DISCRIMINANT ANALYSIS (QDA).

These results also hold for more than two classes, where the boundaries partition input space into linear (LDA) or curved (QDA) cells. A symmetric $D \times D$ covariance has $D(D+1)/2$ free entries, so with K classes QDA uses $(K-1)D(D+1)/2 \approx (K-1)D^2/2$ more parameters than LDA to define the covariance matrices. This could hurt because of the bias/variance trade-off.

11.4 LDA and logistic regression

The predictive conditional of LDA is

$$p(Y=a \mid X) \propto p(X \mid Y=a)\,p(Y=a) \propto \exp\Big(-\frac{1}{2}(X-\mu_a)^T\Sigma^{-1}(X-\mu_a)\Big)\,p(Y=a),$$

and similarly for $p(Y=b \mid X)$, with the same normalizer since Σ is shared.

Proposition 11.5 [LDA POSTERIOR IS LOGISTIC]. *Normalizing so that* $p(Y=a \mid X) + p(Y=b \mid X) = 1$,

$$p(Y=a \mid X) = \frac{1}{1+\exp\big(-(w^TX+w_0)\big)},$$

where

$$w = \Sigma^{-1}(\mu_a - \mu_b), \qquad w_0 = -\frac{1}{2}\mu_a^T\Sigma^{-1}\mu_a + \frac{1}{2}\mu_b^T\Sigma^{-1}\mu_b + \ln\frac{p(Y=a)}{p(Y=b)}.$$

Proof. Dividing numerator and denominator by the a term gives $p(Y = a \mid X) = 1/(1 + e^{-z})$ with

$$z = \ln\frac{p(X \mid Y=a)\,p(Y=a)}{p(X \mid Y=b)\,p(Y=b)}.$$

The quadratic terms $-\frac{1}{2}X^T\Sigma^{-1}X$ of the two exponents cancel, leaving $z = (\mu_a - \mu_b)^T\Sigma^{-1}X - \frac{1}{2}\mu_a^T\Sigma^{-1}\mu_a + \frac{1}{2}\mu_b^T\Sigma^{-1}\mu_b + \ln\frac{p(Y=a)}{p(Y=b)}$. □

Training a model of this form, $1/(1 + \exp(-w^T X))$, directly is the discriminative classifier called *logistic regression*, where the sigmoid squashes $w^T x_i$ into $[0, 1]$ (Lecture 12). So is GDA discriminative or generative? It is generative; the comparison is as follows.

1. An LDA model can always be written as a logistic regression (LR) model.
2. The converse is false: not every logistic regression model can be written as an LDA model.
3. So LDA makes strictly stronger modeling assumptions than LR, and LR is the more general model, able to trace out more complicated decision boundaries.
4. When the LDA assumptions are correct (class-conditional densities are Gaussian), LDA tends to be better than LR.
5. As the amount of data goes to infinity, the results become more similar.
6. This is generally true of generative versus discriminative models: when the generative model's assumptions are met it can beat its discriminative counterpart, presumably because it restricts the space of models.

11.5 Discrete features and naive Bayes

When the features x_i are discrete, the density part of the generative model, $p(X \mid Y = a)$, becomes a discrete probability distribution. If all features are assumed independent given the class,

$$p(X \mid Y = a) = \prod_{f=1}^{D} p(X_f \mid Y = a),$$

the result is the NAIVE BAYES model. It is naive because of the independence assumption, the analogue of a spherical Gaussian, and it can be augmented in various ways to mitigate the assumption, for example tree-augmented Bayes.

11.6 Kinds of classification problems

- Binary classification: two classes, responses $y \in \{0, 1\}$ (win or lose, disease or no disease).
- Multiclass classification: many classes, as in image labeling or sentiment analysis (positive, negative, or neutral).
- Structured prediction: predicting a structured object instead of a discrete class, as in translation and chatbots such as ChatGPT.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 11 (Spring 2026).

Lecture 12: Logistic Regression and Classifier Accuracy

Lecture 11 showed that the posterior of LDA has the form of a sigmoid of a linear function. Logistic regression fits that form directly. The second half of the lecture asks how to evaluate and compare classifiers.

12.1 From generative posteriors to the sigmoid

For any two-class generative model,

$$p(C_1 \mid x) = \frac{p(x \mid C_1)p(C_1)}{p(x \mid C_1)p(C_1) + p(x \mid C_2)p(C_2)} = \frac{1}{1+\exp(-a)} = \sigma(a), \qquad a = \ln \frac{p(x \mid C_1)p(C_1)}{p(x \mid C_2)p(C_2)},$$

where σ is the logistic sigmoid. For LDA the log-odds a is linear, $a = w^T x + w_0$ with $w = \Sigma^{-1}(\mu_1 - \mu_2)$ and $w_0 = -\frac{1}{2}\mu_1^T\Sigma^{-1}\mu_1 + \frac{1}{2}\mu_2^T\Sigma^{-1}\mu_2 + \ln\frac{p(C_1)}{p(C_2)}$ (Lecture 11). For K classes, with $a_k = \ln p(x \mid C_k)p(C_k)$,

$$p(C_k \mid x) = \frac{p(x \mid C_k)p(C_k)}{\sum_j p(x \mid C_j)p(C_j)} = \frac{\exp(a_k)}{\sum_j \exp(a_j)}.$$

12.2 Logistic regression

Definition 12.1 [Logistic regression]. For binary classification with classes $y_i \in \{a, b\}$, logistic regression is the discriminative model

$$p(y_i = a \mid x_i, w) = \frac{1}{1+\exp(-w^T x_i)} = \sigma(w^T x_i), \qquad p(y_i = b \mid x_i, w) = 1 - p(y_i = a \mid x_i, w),$$

so the two probabilities sum to 1. The hard class prediction is $\arg\max_k p(y_i^{\text{pred}} = k)$.

Linear regression's prediction $w^T x_i$ can exceed the range $[0, 1]$; the sigmoid squashes it into $(0, 1)$ (Figure 15).

Proposition 12.2 [Sigmoid properties]. *$\sigma(a) = 1/(1+e^{-a})$ maps $-\infty < a < \infty$ onto $0 < \sigma(a) < 1$, and*

$$1 - \sigma(a) = \frac{e^{-a}}{1+e^{-a}} = \sigma(-a), \qquad \sigma^{-1}(p) = \log\frac{p}{1-p}, \qquad \frac{d}{da}\sigma(a) = \sigma(a)\big(1-\sigma(a)\big) = \sigma(a)\sigma(-a).$$

The inverse σ^{-1} is the logit *function, so $\sigma = \text{logit}^{-1}$.*

Proof. The symmetry follows by multiplying $\sigma(-a) = 1/(1+e^{a})$ by e^{-a}/e^{-a}. Solving $p = 1/(1+e^{-a})$ gives $e^{-a} = (1-p)/p$. For the derivative, $\sigma'(a) = e^{-a}/(1+e^{-a})^2 = \sigma(a)\cdot\sigma(-a)$. □

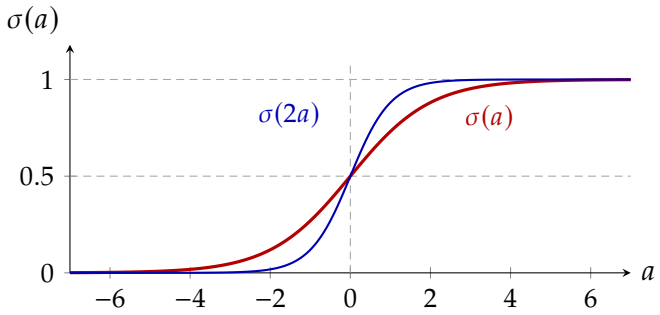


Figure 15: The sigmoid $\sigma(a)$ and $\sigma(2a)$. Doubling the input leaves the boundary $\sigma = \frac{1}{2}$ at $a = 0$ but makes the predictions more confident.

Doubling $w^T x_i$ makes linear-regression predictions twice as large. In logistic regression it makes the predictions more confident, pushing probabilities toward 0 and 1 without moving the boundary.

12.3 Multi-class logistic regression

Definition 12.3 [Softmax]. The softmax generalizes the sigmoid to $K > 2$ classes:

$$\text{softmax}(j, \{z_1, \ldots, z_K\}) = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}}.$$

Its values lie between 0 and 1 and are normalized over classes, $\sum_{j=1}^{K} \text{softmax}(j, \{z_1, \ldots, z_K\}) = 1$, so we can choose to treat them as probabilities.

For example, softmax of $z = (-5, -3, -7)$ is $(0.117, 0.867, 0.016)$. Scaling the inputs changes the confidence: $10^{-5}z$ gives nearly uniform outputs $(0.333, 0.333, 0.333)$, while $100z$ gives essentially $(0, 1, 0)$.

Softmax is translation invariant:

$$\frac{e^{z_a+c}}{e^{z_a+c} + e^{z_b+c}} = \frac{e^{z_a}e^{c}}{e^{z_a}e^{c} + e^{z_b}e^{c}} = \frac{e^{z_a}}{e^{z_a} + e^{z_b}}.$$

Binary logistic regression is a two-class softmax with the second weight vector fixed at $w' = 0$:

$$p(y_i = a \mid x_i) = \frac{1}{1+\exp(-w^T x_i)} = \frac{\exp(w^T x_i)}{\exp(w^T x_i)+1} = \frac{\exp(w^T x_i)}{\exp(w^T x_i)+\exp(w'^T x_i)}.$$

Reformulated as multi-class, $p(y_i = a \mid x_i) \propto \exp(w_a^T x_i)$ and $p(y_i = b \mid x_i) \propto \exp(w_b^T x_i)$, so $p(y_i = a \mid x_i) = \text{softmax}(1, \{w_a^T x_i, w_b^T x_i\})$. This is the same model as binary logistic regression

with $w = w_a - w_b$: by translation invariance only the difference matters, so infinitely many pairs (w_a, w_b) give the same predictions, and the system is overparameterized. Indeed, for any class j, $p(y_i = j \mid x) = 1 - \sum_{k \neq j} p(y_i = k \mid x)$ is already determined by the others.

The decision boundary $p(y_i = a \mid x) = p(y_i = b \mid x)$ means $\exp(w_a^T x) = \exp(w_b^T x)$, that is,

$$(w_a - w_b)^T x = 0,$$

a linear boundary in the features: in two dimensions, a line through the input space. A basis expansion changes this. With $h_\theta(x) = \sigma(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$ the boundary is a line, which may underfit; adding quadratic terms $\theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2$ gives a curved boundary; and high-order terms such as $\theta_6 x_1^3 x_2$ give a boundary contorted enough to overfit.

12.4 Maximum likelihood for logistic regression

Take $y_i \in \{0, 1\}$ without loss of generality, and write $p_i = p_1(x_i) = \sigma(w^T x_i)$ and $p_0(x_i) = 1 - p_1(x_i)$. The likelihood is naturally a product of Bernoullis,

$$L(\{y_i\} \mid \{x_i\}, w) = \prod_{i=1}^{N} p_1(x_i)^{y_i} \big(1 - p_1(x_i)\big)^{1-y_i}, \qquad LL(w) = \sum_{i=1}^{N} y_i \ln p_i + (1 - y_i) \ln(1 - p_i).$$

Proposition 12.4 [Logistic regression gradient]. *$\nabla_w LL(w) = \sum_{i=1}^{N} (y_i - p_i) x_i = X^T (y - p)$, where $y, p \in \mathbb{R}^N$ and X has rows x_i^T.*

Proof. By the chain rule and $\sigma' = \sigma(1 - \sigma)$, $\nabla_w p_i = p_i(1 - p_i) x_i$. Then

$$\nabla_w LL = \sum_{i=1}^{N} \Big(\frac{y_i}{p_i} - \frac{1 - y_i}{1 - p_i} \Big) p_i (1 - p_i) x_i = \sum_{i=1}^{N} \big(y_i (1 - p_i) - (1 - y_i) p_i \big) x_i = \sum_{i=1}^{N} (y_i - p_i) x_i. \qquad \square$$

Setting $X^T(y - p) = 0$ has no closed-form solution for w, because p depends on w through the sigmoid. As with mixtures of Gaussians (and soon neural networks), iterative optimization is needed, such as gradient ascent on LL, equivalently gradient descent on the negative log-likelihood:

$$w_{k+1} = w_k + \alpha_k \nabla_w LL(w_k) = w_k + \alpha_k X^T \big(y - p(w_k) \big).$$

For K classes the likelihood is multinomial, so the approach is also called multinomial regression: with $p_i^k = \text{softmax}(k, \{w_k^T x_i\})$,

$$L = \prod_{i=1}^{N} \prod_{k=1}^{K} (p_i^k)^{[y_i = k]}, \qquad LL = \sum_{i=1}^{N} \sum_{k=1}^{K} [y_i = k] \log p_i^k,$$

and the rest is analogous to the binary case.

12.5 Choosing between classifiers

Within cross-validation we need a metric to compute on each test fold. Suppose a logistic regression and a neural network output scores $p(y = 1 \mid x)$ for the same test points. We could use the held-out likelihood, or threshold at $p = 0.5$ and count misclassifications (accuracy). Both are useful if the model is CALIBRATED: its probabilities mean what they say, so that among points assigned probability p, a fraction about p are positive (checked by plotting the observed fraction of positives against the predicted probability).

Example: *A miscalibrated but useful classifier.* On these 11 test points (truth, logistic-regression score, neural-network score):

Truth	1	0	0	0	0	1	0	0	1	0	1
LR	0.88	0.77	0.59	0.81	0.7527	0.93	0.7194	0.5531	0.98	0.0839	0.8429
NN	0.9038	0.8455	0.4655	0.3204	0.2491	0.63	0.4983	0.6513	0.3806	0.1619	0.7028

At the "optimal" threshold 0.5, LR makes 6 errors and NN only 3, so accuracy would choose NN, the model containing less information. LR assigns every true 1 a higher score than every true 0, so any threshold between 0.81 and 0.8429 (roughly 0.8) gives LR zero errors, while no threshold gives NN fewer than 3.

Accuracy also requires choosing a threshold and is meaningless for non-probabilistic scores. ROC curves address these issues.

12.6 Confusion matrices and rates

Consider binary classifiers. Choose a threshold on the score and call all samples above it "1" (positive, for example healthy) and all below it "−1" (negative, for example unhealthy). Every test point falls into one of four categories (Figure 16):

- FALSE POSITIVE (FP): truly −1 but called 1;
- FALSE NEGATIVE (FN): truly 1 but called −1;
- TRUE POSITIVE (TP): truly 1 and called 1;
- TRUE NEGATIVE (TN): truly −1 and called −1.

With N test points, N = FP + FN + TP + TN; FP and FN are the mistakes. The counts form the CONFUSION MATRIX:

		Model prediction		
		Negative	Positive	
Ground truth	Negative	TN	FP	actual negatives = TN + FP
	Positive	FN	TP	actual positives = FN + TP

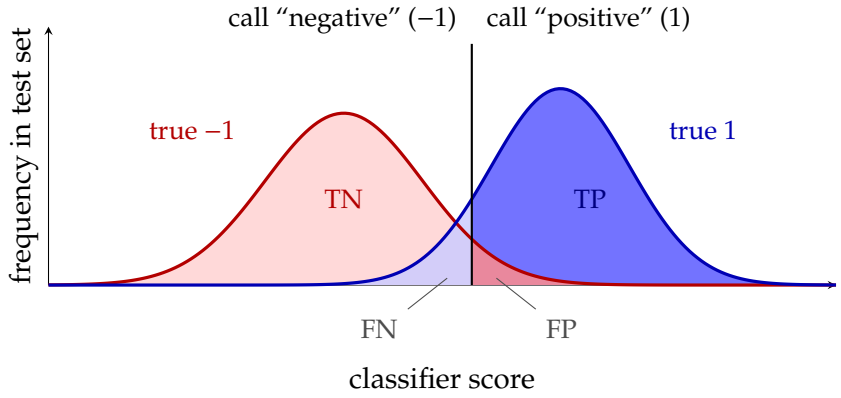


Figure 16: Score distributions of true negatives (red) and true positives (blue) on a test set, with a decision threshold. Points above the threshold are called positive: TP (dark blue) or FP (dark red); points below are called negative: TN (light red) or FN (light blue). Schematic.

Definition 12.5 [Accuracy and rates]. The accuracy at a threshold is

$$\text{accuracy}(\text{threshold}) = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} = \frac{\text{TP} + \text{TN}}{N}.$$

Rates normalize by the number of samples that could have received that call:

- TPR = TP/(FN + TP), the sensitivity or recall;
- TNR = TN/(TN + FP), the specificity;
- FNR = FN/(FN + TP) = 1 − TPR, the miss rate;
- FPR = FP/(TN + FP) = 1 − TNR, the fall-out;
- precision = TP/(TP + FP), the fraction of predicted positives that are positive.

Example: *Pitfalls of accuracy.* A spam filter sees 100 emails, 5 spam (1) and 95 ham (0). Classifying everything as ham has accuracy 95/100 = 0.95 but detects none of the spam; classifying everything as spam has accuracy 5/100 = 0.05 but detects all of it. Accuracy is problematic under class imbalance, and it still requires a threshold.

12.7 ROC curves

Moving the threshold changes FP, FN, TP, and TN. Sweeping it from high to low and plotting TPR (sensitivity) against FPR (1−specificity) traces the RECEIVER OPERATING CHARACTERISTIC (ROC) curve, from $(0, 0)$ (everything negative) to $(1, 1)$ (everything positive). A perfect classifier passes through $(0, 1)$; a random classifier lies on the diagonal TPR = FPR; curves toward the upper left are better. If neither of two classifiers dominates (their ROC curves cross), ask which regime matters: if few false positives are required, pick the method that is higher at small FPR, for example one that dominates up to FPR 40%.

Any instance classified as positive at some threshold is also positive at every lower threshold, which gives a simple algorithm.

Algorithm 12.1: ROC curve

Input: test labels $y_i \in \{0, 1\}$ and scores s_i, $i = 1, \ldots, N$
Output: points (FPR, TPR) of the ROC curve

sort the test instances by decreasing score
TP $\leftarrow 0$; FP $\leftarrow 0$; output $(0, 0)$
for each instance i in sorted order (lowering the threshold past s_i) **do**
    if $y_i = 1$ **then** TP $\leftarrow$ TP + 1
    else FP $\leftarrow$ FP + 1
    output (FP/#actual negatives, TP/#actual positives)

The curve is a staircase whose smoothness depends on the number of points, which is restricted by the number of test points and the uniqueness of the scores. Figure 17 applies the algorithm to the case study.

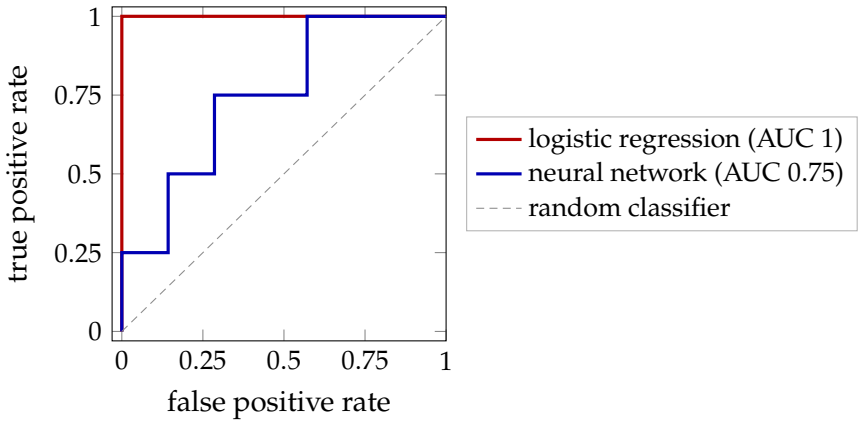


Figure 17: ROC curves of the case-study scores (4 positives, 7 negatives). The miscalibrated logistic regression separates the classes perfectly; the neural network does not.

Definition 12.6 [AUC]. The AREA UNDER THE ROC CURVE (AUC) summarizes a ROC curve in one number; bigger is better. A perfect classifier has AUC = 100% and a random one 50%.

The AUC can be computed by a slight modification of the ROC algorithm, a simple form of integration such as the trapezoid rule $\int_a^b f(x)\,dx \approx \sum_n \frac{1}{2}(f_n + f_{n+1})\Delta x_n$. It also has a probabilistic meaning: the AUC equals the probability that the classifier ranks a randomly chosen positive sample higher than a randomly chosen negative one. (Each negative passed while sweeping the threshold adds a column of width 1/#negatives and height equal to the fraction of positives already ranked above it, so the area is the fraction of positive-negative pairs ranked correctly, counting ties as one half.) In the case study, LR ranks all $4 \cdot 7 = 28$ pairs correctly, AUC = 1, while NN ranks 21 correctly, AUC = 0.75. The more separated the score distributions of the two classes, the larger the AUC.

Sometimes we know we would never operate above some FPR, for example when deciding on a medical intervention like chemotherapy, or on expensive follow-up biology experiments. Then the PARTIAL AUC, such as AUC(0.2), measures the area only over FPR ≤ 0.2.

12.8 Class balance, precision, and recall

ROC curves are insensitive to the balance of classes in the test set, because TPR and FPR each normalize within one class. To obtain accuracy from a ROC curve we need the balance: with a fraction π of actual positives, accuracy = π TPR + $(1 - \pi)(1 - \text{FPR})$, and knowing π we can find the point on the curve with optimal accuracy. Precision, by contrast, depends on the class distribution of the points being evaluated, while recall = TPR does not. A PRECISION-RECALL CURVE replaces FPR by precision and flips the axes, plotting precision against recall; its analogue of ROC-AUC is the AUPR. ROC curves are useful when we want invariance to the class distribution, and precision-recall curves when we care about (and know) the balance of classes at test time.

In summary, ROC curves give a more nuanced picture than counting misclassifications, need no decision threshold, summarize a binary classifier across all trade-offs between FPR and FNR, and allow comparison by AUC, by the whole curve, or by part of it. They ignore calibration, which can be a pro or a con. If calibrated uncertainty matters, as in medical treatment decisions or invasive diagnostics, in ML-based design (using a predictive model $p(y \mid x)$ to design the best small-molecule binder to a drug target), or in active learning, ROC curves are the wrong evaluation.

Many other evaluations are dictated by the domain. For a ranking of web pages where users look only at the top 4 results, sorted by $f(x \mid w, b)$, PRECISION@4 is the fraction of the top 4 that are relevant and RECALL@4 the fraction of relevant pages that appear in the top 4; with 2 of the top 4 relevant out of 3 relevant pages, precision@4 = 1/2 and recall@4 = 2/3. Only the top of the ranking matters.

12.9 Metrics as losses and cost-sensitive learning

Why not train on these metrics directly? They are often not differentiable (AUC and others involve hard thresholding), and ranking needs the entire set of data rather than a mini-batch, so SGD does

not apply. There are niche attempts to make metrics differentiable, but they are rarely used in practice.

Ordinary classification treats all types of misclassification the same, for example through a likelihood loss. A COST-SENSITIVE classifier lets different errors have different impact, specified by a user-defined COST MATRIX. For cancer screening, predicting cancer for a healthy patient costs 1, but missing a cancer costs 1000, and correct calls cost 0. A common and simple choice is a cost-balancing loss, which effectively reweights the training data by the costs:

$$\arg\min_{w,b} \left(1000 \sum_{i:\, y_i = 1} L\big(y_i, f(x_i \mid w, b)\big) + \sum_{i:\, y_i = -1} L\big(y_i, f(x_i \mid w, b)\big) \right).$$

Unlike ROC-type analysis, which only evaluates a trained model, this changes the objective function during training.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 12 (Spring 2026).

Lecture 13: Convergence, Momentum, Adam, and Stochastic Gradient Descent

Most models are trained by minimizing a loss with gradient descent. This lecture derives the method, analyzes its convergence on a quadratic approximation of the loss, and introduces the standard improvements: momentum, learning-rate schedules, adaptive learning rates (AdaGrad, RMSProp, Adam), and stochastic and mini-batch gradients.

13.1 Gradient descent

Training data form an $N \times d$ matrix whose rows are the inputs $x_1, \dots, x_N \in \mathbb{R}^d$, with labels $y_1, \dots, y_N$. For regression with $f(x) = w^T x$ the training loss is

$$L(w) = \sum_{i=1}^{N} \big(f(x_i) - y_i\big)^2,$$

and training means minimizing it over the model weights; the fitted model then predicts labels for new test inputs.

To minimize a hard function $f(x)$, approximate it near the current point x_t by its first-order Taylor expansion, and add a penalty that keeps the next point close to x_t, where the linear model is trustworthy:

$$x_{t+1} = \arg\min_x \left[f(x_t) + \nabla f(x_t)^T (x - x_t) + \frac{1}{2\eta} \|x - x_t\|^2 \right].$$

With $\nabla_x (a^T x) = a$ and $\nabla_x \|x - a\|^2 = 2(x - a)$, setting the gradient to zero gives $\nabla f(x_t) + \frac{1}{\eta}(x - x_t) = 0$, that is,

$$\boxed{x_{t+1} = x_t - \eta \nabla f(x_t)}.$$

Iterating this step is gradient descent: move against the gradient, with learning rate η. In the notation of the rest of the lecture, with error $E(w)$:

Algorithm 13.1: Batch gradient descent

Input: Error E, learning rate η (typically $0 < \eta < 1$), maximum iterations $\tau_{\max}$, tolerance ϵ.

$w^{(0)} \leftarrow$ Initialize() *// typically a small random value*

for $\tau = 1, \dots, \tau_{\max}$ **do**

  $w^{(\tau)} \leftarrow w^{(\tau-1)} - \eta \nabla E\big(w^{(\tau-1)}\big)$

  if Converged$\big(w^{(\tau)}, w^{(\tau-1)}, \epsilon\big)$ **then** terminate

A typical stopping condition is a change of less than ϵ in the update $\Delta w^{(\tau-1)}$. On a function with many local minima, gradient descent finds a nearby local minimum, which depends on the starting

point. The family of "nice" functions on which gradient descent provably finds the global minimum is the CONVEX FUNCTIONS.

Example: *The learning rate matters even for a quadratic.* Minimize $f(x) = 3x^2 + 4x - 2$; from $f'(x) = 6x + 4 = 0$, the minimizer is $x^* = -2/3$. Gradient descent gives

$$x_{t+1} = x_t - \eta(6x_t + 4) = (1 - 6\eta)x_t - 4\eta.$$

Substituting repeatedly and summing the geometric series $\sum_{j=0}^{t-1} a^j = (1 - a^t)/(1 - a)$ with $a = 1 - 6\eta$,

$$x_{t+1} = (1 - 6\eta)^t x_1 - \frac{1 - (1 - 6\eta)^t}{1 - (1 - 6\eta)} 4\eta.$$

If $|1 - 6\eta| < 1$, that is $0 < \eta < 1/3$, then $(1 - 6\eta)^t \to 0$ and $x_{t+1} \to -4\eta/(6\eta) = -2/3 = x^*$. Otherwise the iterates do not converge: too large a step overshoots the bottom of the parabola by more each time.

13.2 Quadratic approximation and the Hessian

To understand convergence, approximate the error near a point by a quadratic. For $f : \mathbb{R} \to \mathbb{R}$, the second-order Taylor approximation at $\hat{x}$ is

$$\tilde{f}(x) = f(\hat{x}) + f'(\hat{x})(x - \hat{x}) + \tfrac{1}{2} f''(\hat{x})(x - \hat{x})^2;$$

for example, for $f = \sin$ at $\hat{x} = 1$, $\tilde{f}(x) = \sin 1 + \cos 1\,(x - 1) - \tfrac{1}{2} \sin 1\,(x - 1)^2$.

Definition 13.1 [HESSIAN]. The HESSIAN of $E : \mathbb{R}^D \to \mathbb{R}$ is the matrix of second partial derivatives, $H = \nabla^2 E$ with $H_{jk} = \partial^2 E / \partial w_j \partial w_k$. It describes the local curvature of E and determines the type of a critical point.

The second-order Taylor expansion of the error surface at $\hat{w}$ is

$$\tilde{E}_{\hat{w}}(w) = E(\hat{w}) + (w - \hat{w})^T b + \tfrac{1}{2}(w - \hat{w})^T H (w - \hat{w}), \qquad b = \nabla E(\hat{w}), \quad H = \nabla^2 E(\hat{w}).$$

For example, $E(w) = (w_0 - 1)^2 + (w_1 - 2)^2 + 1$ has $\nabla E(w) = \big(2(w_0 - 1),\, 2(w_1 - 2)\big)^T$ and $H = 2I$.

At a critical point w^* we have $b = \nabla E(w^*) = 0$, leaving $\tilde{E}_{w^*}(w) = E(w^*) + \tfrac{1}{2}(w - w^*)^T H (w - w^*)$. Since H is symmetric, it has orthonormal eigenvectors u_i ($u_i^T u_j = \delta_{ij}$) with real eigenvalues λ_i, $H u_i = \lambda_i u_i$. Change variables to coordinates along the eigenvectors,

$$w - w^* = \sum_i \alpha_i u_i, \qquad \alpha_i = (w - w^*)^T u_i.$$

Proposition 13.2 [QUADRATIC ERROR IN EIGEN-COORDINATES]. $\tilde{E}_{w^*}(\alpha) = E(w^*) + \dfrac{1}{2} \sum_i \lambda_i \alpha_i^2$.

Proof. Using $Hu_j = \lambda_j u_j$ and then orthonormality,

$$(w - w^*)^T H(w - w^*) = \left(\sum_i \alpha_i u_i\right)^T \left(\sum_j \alpha_j \lambda_j u_j\right) = \sum_{i,j} \alpha_i \alpha_j \lambda_j u_i^T u_j = \sum_i \alpha_i^2 \lambda_i.$$

□

At $\alpha = 0$ the error is $E(w^*)$. Moving along u_i increases α_i^2, and the error rises or falls according to the sign of λ_i. Hence if the eigenvalues of H are all positive, w^* is a local minimum; all negative, a local maximum; mixed signs, a saddle point. The elliptical contours of constant error near a minimum are aligned with the eigenvectors of H, and are narrowest along the direction of largest eigenvalue.

13.3 Convergence on the quadratic approximation

Near a minimum, with all $\lambda_i > 0$, the error is minimized at $\alpha = 0$. Since $\partial \tilde{E}_{w^*}/\partial \alpha_i = \lambda_i \alpha_i$, and the u_i are orthonormal, gradient descent acts on each coordinate separately:

$$\alpha_i^{(\tau)} = \alpha_i^{(\tau-1)} - \eta \lambda_i \alpha_i^{(\tau-1)} = (1 - \eta\lambda_i)\alpha_i^{(\tau-1)}, \qquad \text{so} \qquad \boxed{\alpha_i^{(\tau)} = (1 - \eta\lambda_i)^\tau \alpha_i^{(0)}}.$$

Theorem 13.3 [Convergence condition]. *On the quadratic approximation, $\alpha^{(\tau)} \to 0$ for every starting point if and only if $|1 - \eta\lambda_i| < 1$ for all i, that is, $0 < \eta < 2/\lambda_{\max}$. A smaller $|1 - \eta\lambda_i|$ means faster convergence along u_i, and $1 - \eta\lambda_i < 0$ makes that coordinate oscillate in sign.*

To make the most progress per step we want η as large as possible, but $\eta \geq 2/\lambda_{\max}$ gives $|1 - \eta\lambda_{\max}| \geq 1$. With $\eta < 2/\lambda_{\max}$, the slowest coordinate is the one with $\lambda_{\min}$, whose factor satisfies $1 - \eta\lambda_{\min} > 1 - 2\lambda_{\min}/\lambda_{\max}$. So its distance to the minimum is bounded below by

$$\left(1 - \frac{2\lambda_{\min}}{\lambda_{\max}}\right)^\tau \alpha_{\min}^{(0)}$$

(for $\alpha_{\min}^{(0)} > 0$). The ratio $\lambda_{\min}/\lambda_{\max}$ is the reciprocal of the condition number of H. A small ratio implies slow convergence; gradient descent converges quickly on well-conditioned problems. In practice there are two failure modes (Figure 18): on a relatively flat surface, or with a step that is too small, progress is slow; on a poorly conditioned surface, a step large enough for the flat direction makes the steep direction oscillate.

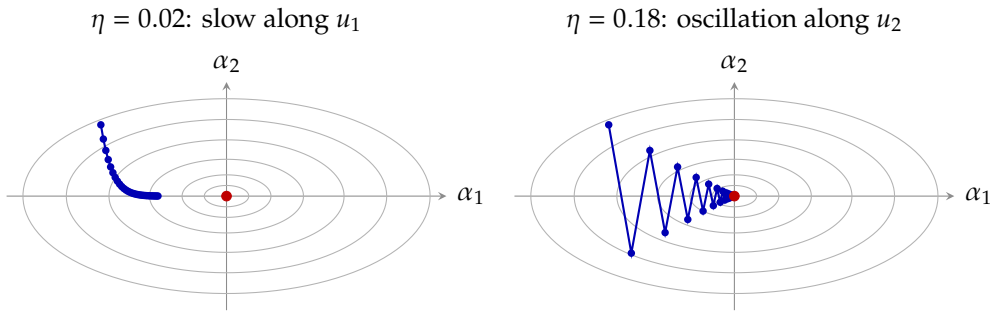


Figure 18: Gradient descent iterates $\alpha_i^{(\tau)} = (1 - \eta\lambda_i)^\tau \alpha_i^{(0)}$ on contours of $\frac{1}{2}(\lambda_1\alpha_1^2 + \lambda_2\alpha_2^2)$ with $\lambda_1 = 1$, $\lambda_2 = 10$, from $\alpha^{(0)} = (-4, 1.5)$. Left: with a small step the flat coordinate crawls (30 steps shown). Right: with $\eta = 0.18$, close to $2/\lambda_{\max} = 0.2$, the steep coordinate flips sign every step (factor -0.8). The minimum is the dot at the origin.

13.4 Momentum

Momentum accumulates update directions that recur from step to step:

$$\Delta w^{(\tau-1)} = -\eta \nabla E\big(w^{(\tau-1)}\big) + \mu\, \Delta w^{(\tau-2)}, \qquad w^{(\tau)} = w^{(\tau-1)} + \Delta w^{(\tau-1)},$$

with momentum hyperparameter $0 \le \mu < 1$ (often $\mu = 0.9$). The previous update must now be stored. The iterate "picks up speed" along directions of repeated updates.

- In a relatively flat direction, ∇E is roughly constant, and the updates add up to a geometric series:

  $$\Delta w^{(\infty)} = -\eta \nabla E\,(1 + \mu + \mu^2 + \cdots) = -\frac{\eta}{1-\mu}\nabla E.$$

  Momentum increases the effective learning rate, since $\eta \le \eta/(1-\mu)$.

- In a steep direction the gradient alternates in sign, and the series alternates:

  $$\Delta w^{(\infty)} = -\eta \nabla E\,(1 - \mu + \mu^2 - \cdots) = -\frac{\eta}{1+\mu}\nabla E.$$

  Momentum decreases the effective learning rate there, since $\eta/(1+\mu) \le \eta$, damping the oscillation.

13.5 Learning-rate schedules

A larger initial learning rate accelerates convergence, but to ensure convergence the learning rate is often reduced across iterations (Figure 19). Common schedules are:

- LINEAR, interpolating from η_{start} to η_{end} over T iterations,

$$\eta^{(\tau)} = \left(1 - \frac{\tau}{T}\right)\eta_{\text{start}} + \frac{\tau}{T}\,\eta_{\text{end}};$$

- POWER LAW (the most common), with hyperparameters $\eta_{\text{start}}, s, c$,

$$\eta^{(\tau)} = \eta_{\text{start}}\left(1 + \frac{\tau}{s}\right)^{-c};$$

- EXPONENTIAL, with hyperparameters $\eta_{\text{start}}, s, c$,

$$\eta^{(\tau)} = \eta_{\text{start}}\, c^{\lfloor \tau/s \rfloor};$$

- COSINE, common for LLMs, with hyperparameters $\eta_{\min}, \eta_{\max}, T$,

$$\eta^{(\tau)} = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\frac{\pi\tau}{T}\right).$$

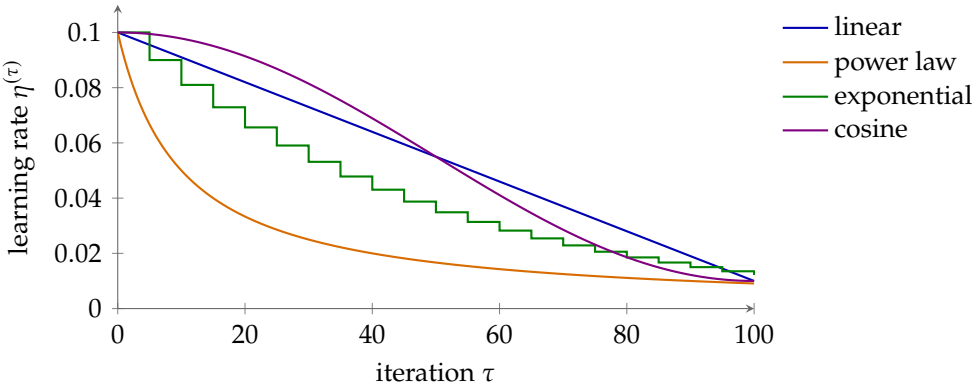


Figure 19: The four schedules over $T = 100$ iterations, starting at 0.1: linear to $\eta_{\text{end}} = 0.01$; power law with $s = 10$, $c = 1$; exponential with $s = 5$, $c = 0.9$; cosine with $\eta_{\max} = 0.1$, $\eta_{\min} = 0.01$. Parameter values chosen for illustration.

13.6 Adaptive learning rates

Adaptive methods give each coordinate w_i its own effective learning rate, based on the history of $g_i = \partial E(w)/\partial w_i$.

ADAGRAD (adaptive gradient) reduces the learning rate on dimensions with the greatest curvature, that is, the largest gradient magnitudes, by tracking a running sum of squared gradients:

$$r_i^{(\tau)} = r_i^{(\tau-1)} + g_i^2, \qquad w_i^{(\tau)} = w_i^{(\tau-1)} - \frac{\eta}{\sqrt{r_i^{(\tau)}} + \epsilon}\, g_i,$$

where $\epsilon \approx 10^{-8}$ ensures numerical stability. Because r_i only grows, large initial gradients leave the

learning rate overly small near convergence.

RMSProp focuses on recent gradients by replacing the sum with an exponentially weighted average,

$$r_i^{(\tau)} = \beta\, r_i^{(\tau-1)} + (1-\beta)\, g_i^2, \qquad r_i^{(0)} = 0,$$

with typically $\beta = 0.9$, and the same update for w_i.

Adam (adaptive moments) combines RMSProp with momentum, and is probably the most widely used gradient-descent update:

$$s_i^{(\tau)} = \beta_1 s_i^{(\tau-1)} + (1-\beta_1)\, g_i, \qquad r_i^{(\tau)} = \beta_2 r_i^{(\tau-1)} + (1-\beta_2)\, g_i^2, \qquad w_i^{(\tau)} = w_i^{(\tau-1)} - \eta \frac{s_i^{(\tau)}}{\sqrt{r_i^{(\tau)}} + \delta},$$

with $s_i^{(0)} = r_i^{(0)} = 0$ and typically $\beta_1 = 0.9$, $\beta_2 = 0.99$. Because the averages start at zero, early steps are too small. The bias-corrected version fixes this by using

$$\hat{s}_i^{(\tau)} = \frac{s_i^{(\tau)}}{1-\beta_1^{\tau}}, \qquad \hat{r}_i^{(\tau)} = \frac{r_i^{(\tau)}}{1-\beta_2^{\tau}}, \qquad w_i^{(\tau)} = w_i^{(\tau-1)} - \frac{\eta}{\sqrt{\hat{r}_i^{(\tau)}} + \delta}\, \hat{s}_i^{(\tau)}.$$

All of these variants fit one template, in which a rule $\textsc{Step}\big(\nabla E(w^{(\tau-1)}), \tau, \beta, \eta\big)$ computes the update $\Delta w^{(\tau-1)}$ and $w^{(\tau)} = w^{(\tau-1)} + \textsc{Step}(\cdot)$; plain gradient descent uses $\textsc{Step} = -\eta \nabla E$, and Adam is implemented inside Step.

13.7 Stochastic gradient descent

With N training points in D dimensions, the batch gradient is

$$\nabla E\big(w^{(\tau-1)}\big) = \frac{1}{N} \sum_{n=1}^{N} \nabla L\big(t_n, f_{w^{(\tau-1)}}(x_n)\big),$$

an average of N terms, each costing $O(D)$, so one update costs $O(ND)$.

The average training error is only a proxy. We would prefer to minimize the expected test error $E_{\text{test}}[w] = \mathbb{E}_{p(t_*, x_*)}\big[L(t_*, f_w(x_*))\big]$ over the unknown test distribution. If the training data are drawn from $p(t_*, x_*)$, then for large N

$$E_{\text{train}}[w] = \frac{1}{N} \sum_{n=1}^{N} L\big(t_n, f_w(x_n)\big) \approx E_{\text{test}}[w],$$

so the training error is an empirical estimate of the test error. The error at a single point,

$E_n(w) = L\big(t_n, f_w(x_n)\big)$, is also an estimate, only a noisier one. Thus

$$\underbrace{\nabla E_n\big(w^{(\tau-1)}\big)}_{\text{cheap approximation}} \approx \underbrace{\nabla E\big(w^{(\tau-1)}\big)}_{\text{best estimate, expensive}} \approx \underbrace{\mathbb{E}_{p(t,x)}\big[\nabla L\big(t, f_{w^{(\tau-1)}}(x)\big)\big]}_{\text{ideal, } p(t,x) \text{ unknown}}.$$

Algorithm 13.2: Stochastic gradient descent

```
w^(0) ← Initialize()
for τ = 1, ..., τ_max do
    n ← RandomInteger(0, N)
    w^(τ) ← w^(τ−1) + Step(∇E_n(w^(τ−1)), τ, β, η)
    if Converged(w^(τ), w^(τ−1), ε) then terminate
```

The stochasticity comes from the random gradient estimate ∇E_n, and each step costs $O(D)$ instead of $O(ND)$. In practice, rather than sampling independently, one often shuffles the data (often only once) and iterates over it; one complete pass over all the data points is an EPOCH.

A single point gives a noisy gradient. MINI-BATCH STOCHASTIC GRADIENT DESCENT averages over a random sample $\mathcal{B}$ of B data points, a MINI-BATCH:

$$\nabla E_{\mathcal{B}}\big(w^{(\tau-1)}\big) = \frac{1}{B}\sum_{b=1}^{B} \nabla L\big(t_{\mathcal{B}_b}, f_{w^{(\tau-1)}}(x_{\mathcal{B}_b})\big),$$

at cost $O(BD)$ per step. The estimators form a spectrum from noisy but cheap to accurate but expensive: single-point SGD, $O(D)$; mini-batch, $O(BD)$; batch, $O(ND)$. Any of the sampling-based procedures is called STOCHASTIC GRADIENT DESCENT (SGD); at $B = 1$ mini-batch SGD is plain SGD, so the batch size is just a hyperparameter. With shuffling, an epoch steps through consecutive blocks of B points, so with N points and $B = 2$ one epoch takes $N/2$ gradient steps.

Algorithm 13.3: Mini-batch SGD with shuffling

```
w^(0) ← Initialize(); τ ← 1
for e = 0, ..., e_max − 1 do
    ShuffleData()                                   // randomize the data ordering
    for n = 0, B, 2B, ... (below N) do
        w^(τ) ← w^(τ−1) + Step(∇E_{n,...,n+B−1}(w^(τ−1)), τ, β, η)
        if Converged(w^(τ), w^(τ−1), ε) then terminate
        τ ← τ + 1
```

Choosing the batch size trades off:

- larger batches give a better gradient estimate and enable hardware parallelism, since the per-point gradients can be computed in parallel;
- smaller batches are cheaper per step, so more steps can be taken, and their greater stochasticity can help escape local minima.

A typical design decision is to increase the batch size until the hardware parallelism is saturated, and to increase the learning rate in proportion to the batch size, taking bigger steps.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 13 (Spring 2026).

Lecture 14: MLE, MAP, and the Bias-Variance Trade-off

In linear regression with Gaussian noise, least-squares minimization is maximum likelihood estimation; with a zero-mean Gaussian prior on the weights, ridge regression is MAP estimation. This lecture revisits MLE versus MAP, then decomposes the expected test error into noise, squared bias, and model variance, and shows how regularization trades one for another.

14.1 MLE versus MAP: the bias of a coin

Let $Y \in [0, 1]$ be the unknown success probability of a coin, and let $x_1, x_2 \in \{0, 1\}$ be observed outcomes, with $P(x_i = 1 \mid Y) = Y$ and $P(x_i = 0 \mid Y) = 1 - Y$, that is,

$$P(x_i \mid Y) = Y^{x_i}(1 - Y)^{1-x_i}.$$

The Bayesian idea is to place a prior on Y. Use the discrete prior $P(Y = 0.9) = P(Y = 0.1) = 0.5$ (a Beta prior is another common choice).

Example: *Two heads.* Observe $D = (x_1, x_2) = (1, 1)$. The likelihood is

$$P(D \mid Y) = \prod_{i=1}^{2} Y^{x_i}(1 - Y)^{1-x_i} = Y^2.$$

Its log has derivative $\frac{d}{dY} \log Y^2 = 2/Y > 0$ on $(0, 1]$, so the likelihood increases in Y and $\hat{Y}_{\text{MLE}} = 1$: two heads make the MLE certain the coin always lands heads.

The MAP estimate maximizes the posterior $P(Y \mid D) = P(D \mid Y)P(Y)/P(D)$ over the two values the prior allows. With $P(D \mid Y = 0.1) = 0.1^2 = 0.01$ and $P(D \mid Y = 0.9) = 0.9^2 = 0.81$, $P(D) = 0.01 \cdot \frac{1}{2} + 0.81 \cdot \frac{1}{2} = 0.41$ and

$$P(Y = 0.9 \mid D) = \frac{0.81 \cdot 0.5}{0.41} \approx 0.988, \qquad P(Y = 0.1 \mid D) = \frac{0.01 \cdot 0.5}{0.41} \approx 0.012.$$

So $\hat{Y}_{\text{MAP}} = 0.9$: the prior keeps the estimate away from the extreme value suggested by only two observations.

14.2 A Gaussian prior on the weights

As for the coin, we can encode a prior belief about regression weights. A common prior is $w_j \sim \mathcal{N}(0, \tau^2)$ independently. Centered at zero, it has no strong effect unless the data support it; the variance τ^2 controls how strongly we expect small weights. The prior encodes the belief that

simpler models are preferred. With features $\phi(x_n)$, the $N \times M$ design matrix Φ has rows $\phi(x_n)^T$, and

$$p(\mathbf{t} \mid \mathbb{X}, w, \sigma^2) = \prod_{n=1}^{N} \mathcal{N}\big(t_n \mid w^T \phi(x_n), \sigma^2\big), \qquad p(w \mid \tau^2) = \frac{1}{(2\pi\tau^2)^{M/2}} \exp\Big(-\frac{\|w\|^2}{2\tau^2}\Big).$$

By Bayes' rule $p(w \mid \mathbf{t}) = p(\mathbf{t} \mid w)p(w)/p(\mathbf{t})$, and up to a constant

$$p(w \mid \mathbf{t}) \propto \exp\Big[-\frac{1}{2\sigma^2}\|\mathbf{t} - \Phi w\|^2 - \frac{1}{2\tau^2}\|w\|^2\Big].$$

Theorem 14.1 [Least squares plus Gaussian prior is ridge regression].

$$\hat{w}_{\mathrm{MAP}} = \arg\max_w p(w \mid \mathbf{t}) = \arg\min_w \Big[\frac{1}{2}\sum_n \big(t_n - y(\phi(x_n), w)\big)^2 + \frac{\lambda}{2}\|w\|^2\Big], \qquad \lambda = \frac{\sigma^2}{\tau^2}.$$

Proof. Since $p(\mathbf{t})$ does not depend on w, $\hat{w}_{\mathrm{MAP}} = \arg\max_w p(\mathbf{t} \mid w)p(w)$. Taking $-\ln$ turns this into $\arg\min_w \big[-\ln p(\mathbf{t} \mid w) - \ln p(w)\big]$. The two terms are

$$-\ln \prod_{n=1}^{N} \mathcal{N}\big(t_n \mid w^T \phi(x_n), \sigma^2\big) = \frac{1}{2\sigma^2}\sum_n \big(t_n - y(\phi(x_n), w)\big)^2 + \text{const},$$

$$-\ln p(w) = \frac{\|w\|^2}{2\tau^2} + \text{const},$$

with $y(\phi(x), w) = w^T\phi(x)$, so $\hat{w}_{\mathrm{MAP}} = \arg\min_w \big[\frac{1}{2\sigma^2}\|\mathbf{t} - \Phi w\|^2 + \frac{1}{2\tau^2}\|w\|^2\big]$. Multiplying by $\sigma^2 > 0$ does not move the minimizer. □

The objective is $\frac{1}{2}E_D(w) + E_w(w)$, the data term plus the regularizer $E_w(w) = \frac{\lambda}{2}\|w\|^2$. A narrower prior (smaller τ^2) gives a larger λ.

14.3 Bias, noise, and model variance

Learning must fit the data, explain what we observe, and generalize: predict the future and explain the unobserved, while still respecting the prior. Three sources of error compete.

- Bias: the expected deviation between the predicted value and the true value. It depends on the choice of model class f and on the learning procedure: if the true function lies outside the set of functions f_w reachable by the possible w, even the best f_w misses it. High bias means underfitting.
- Noise: the variability of the random noise in the process being modeled, from measurement variability, inherent stochasticity, or missing information. It is usually beyond our control.
- Model variance: the variability of the predicted value across different training datasets, that

is, sensitivity to variation in the training data. High variance means poor generalization, OVERFITTING.

14.4 The bias-variance decomposition

Fix a test point x. Observations are noisy, $t = h(x)+\epsilon$, with true function h, $\mathbb{E}[\epsilon] = 0$, and $\mathrm{Var}(\epsilon) = \sigma^2$, so t is a random variable. The model f_w can be any parametric function (for linear regression $f_w(x) = w^T x$). The training data are random, so the fitted w is also a random variable, independent of the test noise ϵ. The goal is the expected test error over repeated datasets, $\mathbb{E}\big[(t - f_w(x))^2\big]$.

Theorem 14.2 [BIAS-VARIANCE DECOMPOSITION].

$$\mathbb{E}\big[(t - f_w(x))^2\big] = \underbrace{\mathbb{E}\big[(t - h(x))^2\big]}_{\text{noise} = \sigma^2} + \underbrace{\big(h(x) - \mathbb{E}[f_w(x)]\big)^2}_{(\text{bias})^2} + \underbrace{\mathbb{E}\big[(\mathbb{E}[f_w(x)] - f_w(x))^2\big]}_{\text{model variance}}.$$

Proof. Subtract and add $h(x)$, and expand $(a + b)^2 = a^2 + b^2 + 2ab$ with $a = t - h(x) = \epsilon$ and $b = h(x) - f_w(x)$:

$$\mathbb{E}\big[(t - f_w(x))^2\big] = \mathbb{E}\big[(t - h(x))^2\big] + \mathbb{E}\big[(h(x) - f_w(x))^2\big] + 2\,\mathbb{E}\big[\epsilon\,(h(x) - f_w(x))\big].$$

By independence of ϵ and w, the cross term is $2\,\mathbb{E}[\epsilon]\,\mathbb{E}[h(x) - f_w(x)] = 0$. The first term is $\mathbb{E}[\epsilon^2] = \sigma^2$, the noise. The second, the model estimation error, is split the same way, subtracting and adding $\mathbb{E}[f_w(x)]$:

$$\begin{aligned}\mathbb{E}\big[(h(x) - f_w(x))^2\big] &= \mathbb{E}\big[(h(x) - \mathbb{E}[f_w(x)])^2\big] + \mathbb{E}\big[(\mathbb{E}[f_w(x)] - f_w(x))^2\big] \\ &\quad + 2\big(h(x) - \mathbb{E}[f_w(x)]\big)\,\mathbb{E}\big[\mathbb{E}[f_w(x)] - f_w(x)\big].\end{aligned}$$

Here $h(x) - \mathbb{E}[f_w(x)]$ is a constant, so it leaves the expectations; this makes the first term $(h(x) - \mathbb{E}[f_w(x)])^2$ and the cross term proportional to $\mathbb{E}\big[\mathbb{E}[f_w(x)] - f_w(x)\big] = 0$. □

In particular $\mathbb{E}[t] = h(x)$, $\mathbb{E}[\epsilon^2]$ is the noise, and $\mathbb{E}\big[(\mathbb{E}[f_w(x)] - f_w(x))^2\big]$ is the model variance.

14.5 Regularization trades variance for bias

In the ridge objective of Theorem 14.1:

- Increasing λ shrinks the weights, so the model is less flexible and the bias increases. But shrinking makes $\hat{w}_{\mathrm{MAP}}$ less sensitive to noise, so the variance decreases. Past a point the model underfits.
- As $\lambda \to 0$ the model becomes more flexible: bias decreases, variance increases, and the model

tends to overfit.

The regularization term is thus a mechanism that trades variance for bias; overfitting is high variance and underfitting is high bias. As a function of λ, the training error increases, while the test error is usually U-shaped, with a sweet spot at an intermediate λ (Figure 20).

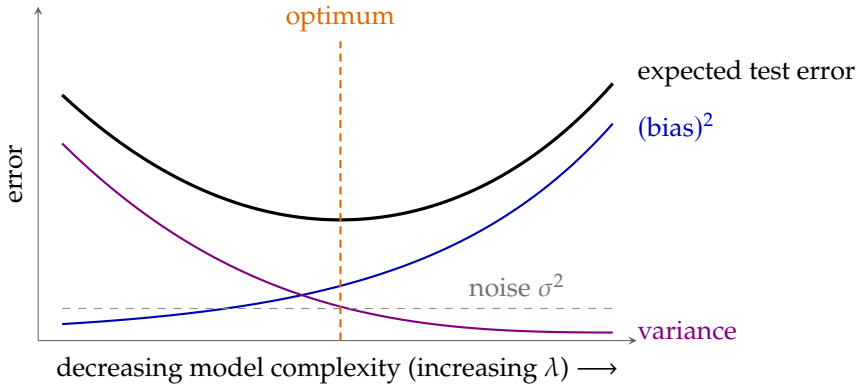


Figure 20: Schematic bias-variance trade-off. As model complexity decreases, the squared bias rises and the variance falls; their sum with the constant noise, the expected test error, is minimized at an intermediate complexity. No data are plotted.

So far, model complexity has been controlled by the number of features, the choice of features, and regularization. More data support more complexity: with more data the variance at each complexity is smaller, so the optimum moves toward more complex models. The value of λ sets the trade-off, larger values meaning more regularization, more bias, and less variance. Since the test error is unknown, λ is determined by validation: the validation error tracks the test error.

Example: *Repeated datasets.* Take $h(x) = \sin(2\pi x)$ with $x \sim \text{Uniform}(0, 1)$, and generate $L = 100$ independent datasets of $N = 25$ noisy observations $t_n = h(x_n) + \epsilon_n$ with $\epsilon_n \sim \mathcal{N}(0, 0.32)$. Fit to each a linear combination of $M = 24$ Gaussian basis functions plus a bias, with ridge penalty λ. For large λ ($\ln \lambda = 3$) the 100 fits agree closely with each other but their average is much flatter than $\sin(2\pi x)$: low variance, high bias. For small λ ($\ln \lambda = -3$) the fits vary widely, but their average follows $\sin(2\pi x)$ closely: low bias, high variance. Measured over this range, the squared bias grows and the variance shrinks as $\ln \lambda$ increases.

14.6 Application: ranking language models

LMArena (formerly Chatbot Arena) is a platform for holistic evaluation of large multimodal models, in which real user conversations and pairwise votes are crowdsourced to build a live human-preference leaderboard. In a battle, a user enters any prompt, two anonymized models answer side by side, and the user votes for the better one; the pairwise setting allows a finer-grained comparison than absolute ratings. The relative strengths of M models are estimated by logistic regression in the Bradley-Terry model: with X_i encoding the two models in battle i and Y_i the

winner,

$$\hat{\beta} = \arg\min_{\beta \in \mathbb{R}^M} \frac{1}{n} \sum_{i=1}^{n} \text{BCELoss}\big(\sigma(X_i^T \beta), Y_i\big),$$

where σ is the sigmoid and BCE the binary cross-entropy. A model's Arena Score is its coefficient in β, reported with bootstrapped confidence intervals.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 14 (Spring 2026).

Lecture 15: Learning with Gradient Descent

This lecture trains a model with gradient descent by hand on a small example, writing the loss as $J(w)$ rather than $E(w)$. The batch algorithm is the one of Lecture 13, $w^{(t+1)} = w^{(t)} - \eta \nabla J\big(w^{(t)}\big)$, with the same initialization, stopping rule, and hyperparameters η and $\tau_{\max}$. Training, learning, fitting, and optimizing all mean the same thing here: find the parameters w^* that minimize $J(w)$.

15.1 A worked example

Fit the depth-1 linear model

$$f(x) = w^T x + b, \qquad w \in \mathbb{R}^2,\ b \in \mathbb{R},$$

with unknown parameters $\theta = (w_1, w_2, b)$, so $d = 3$, to the $n = 4$ points shown, by minimizing the mean squared loss $J(\theta) = \frac{1}{4}\sum_{i=1}^{4}\big(y_i - f(x_i)\big)^2$ (Figure 21).

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

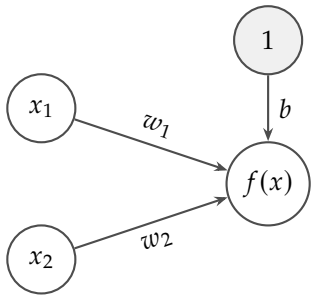


Figure 21: The four training points (left) and the depth-1 model $f(x) = w_1 x_1 + w_2 x_2 + b$ (right), with the bias b as the weight on a constant input 1.

Example: *Predictions and loss at one point.* For $\theta = (0, 1, 1)$ the model is $f(x) = x_2 + 1$, with predictions $(1, 2, 1, 2)$ on the four rows, so

$$J(\theta) = \tfrac{1}{4}\big[(0-1)^2 + (1-2)^2 + (1-1)^2 + (0-2)^2\big] = \tfrac{6}{4} = 1.5.$$

Writing out each row,

$$J(w_1, w_2, b) = \tfrac{1}{4}\big[b^2 + (1 - w_2 - b)^2 + (1 - w_1 - b)^2 + (w_1 + w_2 + b)^2\big].$$

The gradient is the vector $\nabla J = (\partial J/\partial w_1,\ \partial J/\partial w_2,\ \partial J/\partial b)^T$. Differentiating term by term, only the rows in which w_1 appears contribute to the first coordinate:

$$\frac{\partial J}{\partial w_1} = \tfrac{1}{4}\big[-2(1 - w_1 - b) + 2(w_1 + w_2 + b)\big] = \tfrac{1}{2}(2w_1 + w_2 + 2b - 1),$$

and in the same way

$$\nabla J(\theta) = \frac{1}{2}\begin{bmatrix} 2w_1 + w_2 + 2b - 1 \\ w_1 + 2w_2 + 2b - 1 \\ 2w_1 + 2w_2 + 4b - 2 \end{bmatrix}.$$

Example: *One gradient step.* At $w^{(0)} = (0, 1, 1)$ the gradient is $\nabla J\big(w^{(0)}\big) = (1,\ 3/2,\ 2)$. It is not zero, so $(0, 1, 1)$ is not a local minimum. With $\eta = 0.5$,

$$w^{(1)} = w^{(0)} - \eta \nabla J\big(w^{(0)}\big) = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} - 0.5 \begin{bmatrix} 1 \\ 3/2 \\ 2 \end{bmatrix} = \begin{bmatrix} -1/2 \\ 1/4 \\ 0 \end{bmatrix}.$$

The new predictions are $(0,\ 1/4,\ -1/2,\ -1/4)$, and the loss drops from $J\big(w^{(0)}\big) = 1.5$ to

$$J\big(w^{(1)}\big) = \tfrac{1}{4}\Big[0 + \big(\tfrac{3}{4}\big)^2 + \big(\tfrac{3}{2}\big)^2 + \big(\tfrac{1}{4}\big)^2\Big] = \tfrac{46}{64} \approx 0.718.$$

Since J is a convex quadratic, its minimum is where $\nabla J = 0$. The third equation gives $w_1 + w_2 + 2b = 1$; subtracting it from the first two gives $w_1 = w_2 = 0$, so $b = 1/2$ and $J = \frac{1}{4} \cdot 4 \cdot \frac{1}{4} = \frac{1}{4}$. The best linear model predicts the constant $1/2$: this dataset (the XOR of x_1 and x_2) is not fit at all by a linear function of its inputs.

15.2 The rest of the toolkit

The remainder of the lecture revisits, with J in place of E, the material of Lecture 13: the quadratic approximation and Hessian, convergence when $|1 - \eta\lambda_i| < 1$ and the condition number, momentum, learning-rate schedules, AdaGrad, RMSProp, and Adam, and stochastic and mini-batch gradient descent. One addition: eliminating the stored update, momentum is the single recursion

$$w^{(\tau)} = w^{(\tau-1)} - \eta \nabla J\big(w^{(\tau-1)}\big) + \mu\big(w^{(\tau-1)} - w^{(\tau-2)}\big),$$

since $\Delta w^{(\tau-2)} = w^{(\tau-1)} - w^{(\tau-2)}$: each step adds a fraction μ of the previous displacement to the gradient step.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 15 (Spring 2026).

Lecture 16: Entropy, Information, and Logistic Regression

Entropy measures how much information a random variable carries: the number of bits needed to compress it. Cross entropy and KL divergence measure the cost of compressing with the wrong distribution, and the negative log-likelihood of logistic regression turns out to be a cross entropy. This motivates the cross-entropy loss, for two classes and, with the softmax, for many.

16.1 Entropy and compression

Suppose each day it rains in Seattle with probability 80%, independently: a rain random variable X. We observe 100 days as a bit string 1101010110 . . .. How much information is in this sequence, and how far can it be compressed? Not necessarily to 100 bits: Alan, who observes the weather and sends it, and Claude, who receives it, know the statistics of X in advance (but not the realizations), and agree on a compression scheme beforehand, for example "when I see 1111 I send 01, otherwise I send 10 followed by the weather in the next 5 days". If it always rains, the sequence carries no information and compresses to 0 bits.

Definition 16.1 [Entropy]. The entropy of a random variable X with PMF p is

$$H(X) = -\sum_x p(x) \log p(x),$$

with log base 2 and $0 \log 0 = 0$ by continuity. It is the number of bits needed to compress X.

For a Bernoulli variable X_B, equal to 1 with probability p and 0 with probability $1 - p$,

$$H(\text{Rain}) = -\big[p \log p + (1-p) \log(1-p)\big]$$

(Figure 22). At $p = 0$ or $p = 1$ (never or always rain) it is 0 bits: perfectly compressible. At $p = 0.8$ it is $-[0.8 \log 0.8 + 0.2 \log 0.2] \approx 0.7219$ bits per day, and at $p = 0.9$ it is ≈ 0.469. Less surprise means less information and more compression. Only the fair coin, $p = 1/2$, has maximum surprise and maximum information, $H = 1$ bit per realization, and admits no compression.

Theorem 16.2 [Shannon's compression theorem, informal]. *Entropy is the ultimate limit to compression: n independent realizations of X can be compressed into about $nH(X)$ bits, and not fewer.*

So 100 days of rain data with $p = 0.8$ compress into about 73 bits, and with $p = 0.9$ into about 47 bits.

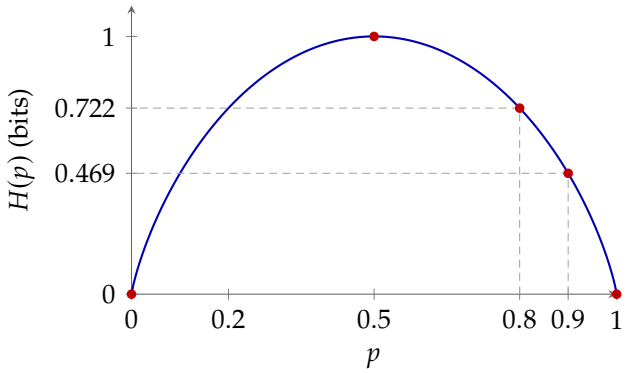


Figure 22: The binary entropy $H(p) = -[p \log_2 p + (1-p)\log_2(1-p)]$, maximal at the fair coin and zero for a deterministic outcome; dashed lines mark $p = 0.8$ and $p = 0.9$.

16.2 Why entropy: typical sequences

Two facts explain the theorem.

1. Describing one object from a list of L things takes $\log L$ bits: for $L = 4$, the codes $00, 01, 10, 11$ name banana, orange, apple, mango.
2. When compressing random sequences, only a few sequences need to be represented, those that appear with very high probability, the TYPICAL SEQUENCES. If there are T typical sequences, compressing with $\log T$ bits succeeds with high probability.

For 100 flips with $p = 0.1$ we expect about 10 heads. Suppose Alan and Claude represent only the sequences with exactly 10 heads (sunny days), declaring failure otherwise. There are $L = \binom{100}{10}$ ways to place 10 sunny days among 100 positions (the same as placing 90), and

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \approx 2^{nH(k/n)}, \qquad \text{so} \qquad \log L \approx 100\, H(0.1),$$

to first order in the exponent. In general the entropy of a random variable is essentially the logarithm of the number of typical sequences, per symbol; this is the ASYMPTOTIC EQUIPARTITION PROPERTY, and it is essentially the law of large numbers.

Example: *The horse race.* One of 8 horses wins, with probabilities $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \frac{1}{64}, \frac{1}{64}, \frac{1}{64}, \frac{1}{64}$. The naive code uses 3 bits ($000, \dots, 111$). The variable-length code

$$0,\ 10,\ 110,\ 1110,\ 111101,\ 111110,\ 111111,\ 111100$$

gives frequent winners short codewords. Its expected length is $\frac{1}{2}\cdot 1 + \frac{1}{4}\cdot 2 + \frac{1}{8}\cdot 3 + \frac{1}{16}\cdot 4 + 4\cdot\frac{1}{64}\cdot 6 = 2$ bits, and the entropy is $H = \sum_x p(x)\log\frac{1}{p(x)} = \frac{1}{2}\cdot 1 + \frac{1}{4}\cdot 2 + \frac{1}{8}\cdot 3 + \frac{1}{16}\cdot 4 + 4\cdot\frac{1}{64}\cdot 6 = 2$ bits. The code matches the Shannon limit, so it is optimal.

16.3 KL divergence and cross entropy

Definition 16.3 [KL divergence and cross entropy]**.** For two PMFs p, q, the Kullback-Leibler divergence and the cross entropy are

$$D(p \,\|\, q) = \sum_x p(x) \log \frac{p(x)}{q(x)}, \qquad H(p, q) = -\sum_x p(x) \log q(x),$$

with the corner cases $0 \log \frac{0}{q} = 0$ and $p \log \frac{p}{0} = +\infty$ for $p > 0$.

Theorem 16.4 [Cross entropy, KL divergence, and entropy]**.** $H(p, q) = D(p \,\|\, q) + H(p)$.

Proof. With $\log \frac{a}{b} = \log a - \log b$,

$$D(p \,\|\, q) = \sum_x p(x) \log p(x) - \sum_x p(x) \log q(x) = -H(p) + H(p, q).$$

□

In words: $H(p, q)$ is the total number of bits needed to describe a random variable with distribution p if the compression scheme was designed for q; $H(p)$ is the number needed with a scheme designed for p; and $D(p \,\|\, q)$ is the extra number of bits wasted by using q for p. As a sanity check, $H(p, p) = H(p)$.

Example: *A fair coin and a deterministic variable.* Let p be a fair coin (1 and 0 each with probability $\frac{1}{2}$) and q deterministic (1 with probability 1). Then $H(p) = 1$, $H(q) = 0$, and

$$D(p \,\|\, q) = \tfrac{1}{2} \log \tfrac{1/2}{1} + \tfrac{1}{2} \log \tfrac{1/2}{0} = +\infty, \qquad D(q \,\|\, p) = 1 \cdot \log \tfrac{1}{1/2} + 0 \cdot \log \tfrac{0}{1/2} = 1,$$
$$H(p, q) = -\left[\tfrac{1}{2} \log 1 + \tfrac{1}{2} \log 0\right] = +\infty, \qquad H(q, p) = -\left[1 \cdot \log \tfrac{1}{2} + 0 \cdot \log \tfrac{1}{2}\right] = 1.$$

Both identities hold: $H(p, q) = D(p \,\|\, q) + H(p) = \infty$ (a fair coin cannot be compressed with a scheme for a deterministic outcome) and $H(q, p) = D(q \,\|\, p) + H(q) = 1 + 0$. KL divergence is not symmetric.

16.4 Logistic regression and the cross-entropy loss

Logistic regression is not a regression: it is a method for binary classification, with target $y \in \{0, 1\}$. It models $p(y \mid x)$ directly through the sigmoid

$$S(z) = \frac{1}{1 + \exp(-z)},$$

which converts a score to a probability in $(0, 1)$: $P(Y = 1 \mid x) = S(w^T x)$. It does not assume that the data x are Gaussian. If one also assumes that x is Gaussian within each class, one gets Gaussian discriminant analysis: QDA if the class covariances differ, LDA if they are the same.

Example: *Predicting ad clicks.* Predict whether a customer clicks on a home-insurance ad from three features (age, owns a car, owns a home). Customer 1, with $x_1 = (30, 0, 0)$, did not click ($y_1 = 0$); customer 2, with $x_2 = (45, 0, 1)$, clicked ($y_2 = 1$). Take $w = (0.1, 1, 10)$. The score of customer 2 is $z_2 = w^T x_2 = 0.1 \cdot 45 + 1 \cdot 0 + 10 \cdot 1 = 14.5$, so $P(Y = 1 \mid x_2) = S(14.5) = 1/(1 + e^{-14.5}) \approx 0.9999$, in agreement with $y_2 = 1$. For customer 1, $z_1 = 3$ and $P(Y = 1 \mid x_1) = S(3) \approx 0.953$.

The model assigns a probability to each customer's click; the training data record what actually happened. The LIKELIHOOD of the data is the probability of the observed labels under the model. In the example above, it is

$$\begin{aligned} L(w) &= P(\text{customer 1 does not click}) \cdot P(\text{customer 2 clicks}) \\ &= \big(1 - S(3)\big)\, S(14.5) \approx 0.047 \cdot 0.9999 \approx 0.047, \end{aligned}$$

which is low: this w is a poor fit for customer 1.

In general, with $z_i = w^T x_i$, sample i contributes $S(z_i)$ if $y_i = 1$ and $1 - S(z_i)$ if $y_i = 0$, compactly $S(z_i)^{y_i}(1 - S(z_i))^{1-y_i}$. For n independent samples,

$$L(w) = \prod_{i=1}^{n} S(z_i)^{y_i}\big(1 - S(z_i)\big)^{1-y_i}, \qquad \log L(w) = \sum_{i=1}^{n} \big[y_i \log \mu_i + (1 - y_i)\log(1 - \mu_i)\big], \quad \mu_i = S(z_i).$$

Each term is minus the cross entropy between the two-point distributions $(y_i, 1 - y_i)$ and $(\mu_i, 1 - \mu_i)$, where for Bernoulli PMFs $H(p, q) = -[p \log q + (1 - p)\log(1 - q)]$. Therefore

$$\boxed{-\log L(w) = \sum_{i=1}^{n} H\big((y_i, 1 - y_i), (\mu_i, 1 - \mu_i)\big)}:$$

the negative log-likelihood is the CROSS-ENTROPY LOSS. Logistic regression produces soft scores for the classes 0 and 1 by computing a linear score and converting it to a probability with the logistic function, and the best parameters maximize the likelihood, that is, minimize the cross entropy.

16.5 Multiclass logistic regression

For k classes the sigmoid is replaced by the SOFTMAX. Given a vector of scores z, the softmax is the vector s with

$$s_i = \frac{\exp(z_i)}{C}, \qquad C = \sum_j \exp(z_j).$$

For $z = (1, 2, 3)$, $(e^1, e^2, e^3) \approx (2.72, 7.39, 20.09)$ and $C \approx 30.19$, so $s \approx (0.09, 0.24, 0.67)$. The softmax is unchanged by adding a constant to every score (it cancels between numerator and C), so $z = (-1, 0, 1)$ gives the same s.

With d features and k classes the weights form a $k \times d$ matrix W with rows $w_1^T, \ldots, w_k^T$; the scores are $z = Wx$, with entries $w_j^T x$, so each w_j has the same dimension as x. The class probabilities are

$\mathrm{softmax}(Wx)$.

Example: *Cross-entropy loss for three classes.* Let $w_1 = (1,0,0)$, $w_2 = (0,1,0)$, $w_3 = (0,1,1)$, and $x = (1,2,3)$. The scores are $(1,2,5)$, $C = e + e^2 + e^5 \approx 158.5$, and the probabilities are $p \approx (0.017, 0.047, 0.936)$. The loss is the cross entropy of the one-hot label y and p, $-\sum_i y_i \log p_i$. If the true class is 2, $y = (0,1,0)$ and the loss is $-\log_2 0.047 \approx 4.42$; if it is class 3, the loss is $-\log_2 0.936 \approx 0.095$. A classifier that is sure and correct pays $-\log 1 = 0$; one that is sure and wrong pays $-\log 0 = +\infty$.

The loss looks only at the probability of the correct class. For $y = (1,0,0)$, the predictions $(0.8, 0.1, 0.1)$ and $(0.8, 0, 0.2)$ have the same cross-entropy loss. The most straightforward way to give partial credit is to soften the true labels, for example $y_{\text{soft}} = \big(0.9,\ 0.1/(k-1),\ \ldots,\ 0.1/(k-1)\big)$.

Two standard image datasets: MNIST has 60,000 handwritten digits, 28×28 black-and-white pixels; CIFAR-10 has 60k 32×32 colour images in 10 classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck), 6k per class, split into 50k training and 10k test images. For MNIST, x has $28 \cdot 28 = 784$ pixels, so each w_j has length 784. Reshaping the fitted w_j of a 10-class logistic regression to a 28×28 image gives a picture that resembles digit j: the weights act as a template for the class.

16.6 The entropy of English

With 26 letters and a space, a uniform code needs $\log 27 \approx 4.75$ bits per letter. But English is far more predictable: "B*t th* En*l*sh l*ng***e i* c*mpre*sib*e" is readable with only 32 of its 44 characters given. A generative model for English can be used to generate fake text, denoise, compress, classify, and break cryptographic codes. Shannon's approximations generate text letter by letter:

- zero order: letters uniform over the 27 symbols (`XFOML RXKHRJFFJUJ ZLPWCFWKCYJ`);
- first order: letters drawn from the marginal letter frequencies (`OCRO HLI RGWR NMIELWIS`);
- second order (digrams): the first letter is drawn from the marginal, and each next letter from the conditional distribution given the previous letter, estimated from a histogram of which letters follow it.

For example, in the text `ON THE ONSET OF BOB` (19 symbols, counting spaces) the marginals are $P(\texttt{O}) = 4/19$ and $P(\texttt{N}) = P(\texttt{B}) = 2/19$. The four letters following an `O` are `N`, `N`, `F`, `B`, so $P(\texttt{N} \mid \texttt{O}) = 1/2$ and $P(\texttt{F} \mid \texttt{O}) = P(\texttt{B} \mid \texttt{O}) = 1/4$. These approximations of English go back to Shannon's 1948 paper, *A Mathematical Theory of Communication*.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 16 (Spring 2026).

Lecture 17: Neural Networks: Depth, Universal Approximation, and Activation Functions

Real high-dimensional data (images, audio, text) usually lie near a low-dimensional manifold inside the input space: a 64×64 image is a vector in $\mathbb{R}^{4096}$, but the pose, position, and orientation of a digit have only a few degrees of freedom. Fixed bases such as polynomials explode with the dimension. Modeling along the manifold instead makes the complexity scale with the manifold dimension, which requires representation learning: extracting features from raw data. Neural networks are *data-dependent bases*: they learn basis functions that adapt to the manifold, and which directions on it matter for the task.

17.1 Neurons

A biological neuron collects signals from other neurons through its dendrites, sums and processes them in the cell body, and transmits the result along its axon; synapses between neurons are excitatory or inhibitory. An artificial neuron keeps the summation and the processing unit.

Definition 17.1 [Artificial neuron]. An artificial neuron is a summation unit followed by an activation function f. With the constant input $x_0 = 1$, whose weight w_0 is the bias term,

$$y = f(w \cdot x) = f\Big(\sum_{d=0}^{D} w_d x_d\Big) = f(w_0 + w_1 x_1 + \cdots + w_D x_D).$$

With f the identity this is linear regression.

With the step function $f(z) = 1$ for $z > 0$ and $f(z) = 0$ for $z < 0$, the neuron is activated exactly on one side of the hyperplane $w \cdot x = 0$. A single such neuron computes AND of two binary inputs: the point $(1, 1)$ is separated from the other three by a line, for example $x_1 + x_2 - \frac{3}{2} = 0$. It cannot compute XOR, which is 1 at $(1, 0)$ and $(0, 1)$ and 0 at $(0, 0)$ and $(1, 1)$: no line separates these, so the decision boundary must be non-linear. Two layers suffice, since

$$x_1 \text{ XOR } x_2 = (x_1 \text{ AND } {\sim}x_2) \text{ OR } ({\sim}x_1 \text{ AND } x_2),$$

and each of the three operations on the right is linearly separable.

17.2 Feed-forward neural networks

A FEED-FORWARD NEURAL NETWORK with one hidden layer of M units computes M linear combinations of the inputs, the PRE-ACTIVATIONS (or logits), and passes them through an activation function h to get the outputs of the learned basis functions:

$$a_j^{(1)} = \sum_{i=1}^{D} w_{ji}^{(1)} x_i + w_{j0}^{(1)}, \qquad z_j^{(1)} = h\big(a_j^{(1)}\big), \qquad j \in \{1, \dots, M\}.$$

The superscript (1) marks parameters of the first layer. The output layer repeats the construction with an output activation f:

$$a_k^{(2)} = \sum_{j=1}^{M} w_{kj}^{(2)} z_j^{(1)} + w_{k0}^{(2)}, \qquad y_k(x, w) = f\big(a_k^{(2)}\big) = f\Big(\sum_{j=1}^{M} w_{kj}^{(2)}\, h\Big(\sum_{i=1}^{D} w_{ji}^{(1)} x_i + w_{j0}^{(1)}\Big) + w_{k0}^{(2)}\Big),$$

for $k \in \{1, \dots, K\}$. Adding constant units $x_0 = z_0 = 1$ absorbs the intercepts into the weight matrices, so $y = f\big(W^{(2)} h(W^{(1)} x)\big)$ (Figure 23).

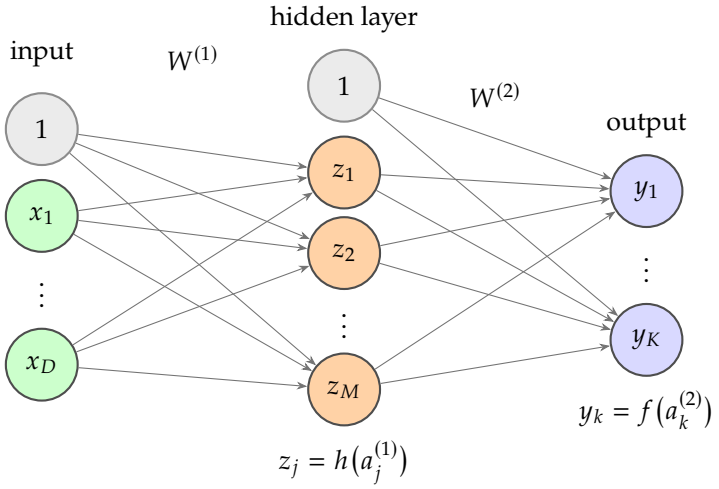


Figure 23: A network with one hidden layer. The constant units (gray) carry the bias weights $w_{j0}^{(1)}$ and $w_{k0}^{(2)}$.

A DEEP NEURAL NETWORK stacks L such layers:

$$z^{(l)} = h^{(l)}\big(W^{(l)} z^{(l-1)}\big), \quad l \in \{1, \dots, L\}, \qquad z^{(0)} = x, \quad z^{(L)} = y,$$

where $h^{(l)}$ and $W^{(l)}$ are the activation function and weights of layer l.

17.3 Universal approximation

Theorem 17.2 [Universal approximation; Cybenko 1989, Funahashi 1989]. *A continuous function $f : \mathbb{R}^D \to \mathbb{R}^K$ on a compact subset of $\mathbb{R}^D$ can be approximated to arbitrary accuracy by a neural network with one hidden layer.*

Neural networks are therefore universal function approximators, but the theorem leaves three questions open.

- It does not say how to find the right weights; training does that, by gradient descent.
- It does not say how big the hidden layer is; it can need to be exponential in D.
- It does not fix the activation function; ReLU works.

The width issue explains why networks are made deep rather than wide. Two layers of logic gates can represent any Boolean function, yet some functions are exponentially simpler to build with many layers, needing far fewer gates. Likewise one hidden layer can represent any continuous function, but some functions are exponentially simpler to represent with several layers, which means fewer parameters and plausibly less data.

17.4 Activation functions

The simplest hidden activation is the identity, but a hidden layer of identity activations is redundant, since the composition is again linear. The simplest non-linear differentiable choice is the sigmoid $\sigma(a) = 1/(1 + e^{-a})$. Its relatives are

$$\tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}} = 2\sigma(2a) - 1, \qquad \text{hard tanh } h(a) = \max\bigl(-1, \min(1, a)\bigr).$$

The sigmoid and tanh families saturate: their gradients go to 0 when the activations are large in magnitude. We want an activation whose gradient is non-zero at least for large positive activations:

$$\text{softplus } h(a) = \ln(1 + e^a), \qquad \text{ReLU } h(a) = \max(0, a),$$

and the leaky ReLU $h(a) = \max(0, a) + \alpha \min(0, a)$. Softplus, also called the soft ReLU, satisfies $h(a) \simeq a$ for $a \gg 1$. Neurons with negative ReLU activations receive no error signal; the leaky ReLU keeps a small slope α there (Figure 24).

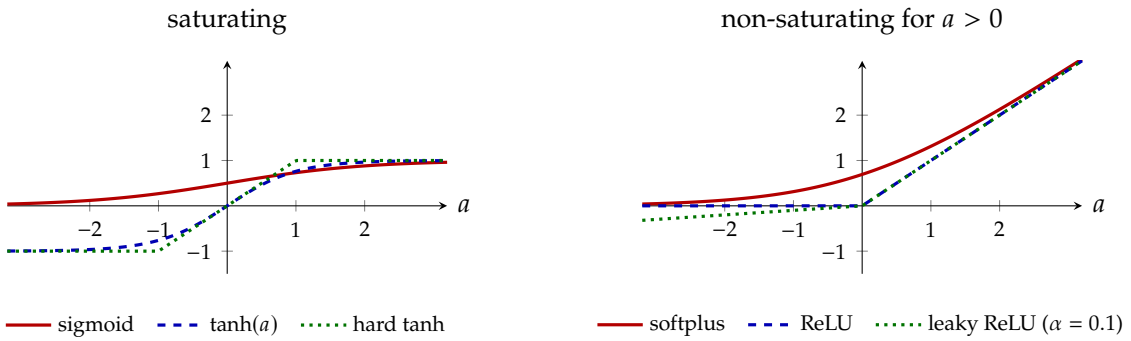


Figure 24: The activation functions of this lecture, plotted from their formulas.

17.5 Learning features: the XOR example

Given d features, one often expands them into, say, d^2 features and gets better performance. There are three ways to do this: problem-specific feature engineering; embedding in a very general space and using kernels; and learning the features from data, which is possible when the model is differentiable end to end. Hidden layers do the last.

Take the XOR data $\{(0,0) \mapsto 0,\ (0,1) \mapsto 1,\ (1,0) \mapsto 1,\ (1,1) \mapsto 0\}$ and first the linear model $f(x) = w^T x + b$ with $w \in \mathbb{R}^2$, $b \in \mathbb{R}$, so $\theta = (w_1, w_2, b)$ and $d = 3$, $n = 4$. The squared loss is

$$J(\theta) = \frac{1}{4}\sum_{i=1}^{4}(y_i - f(x_i))^2 = \frac{1}{4}\big[b^2 + (1 - w_2 - b)^2 + (1 - w_1 - b)^2 + (w_1 + w_2 + b)^2\big].$$

At $\theta = (0,1,1)$ the predictions are $f(x) = (1,2,1,2)$ and $J = \frac{1}{4}(1 + 1 + 0 + 4) = \frac{3}{2}$. Differentiating term by term,

$$\nabla J(\theta) = \frac{1}{2}\begin{bmatrix} 2w_1 + w_2 + 2b - 1 \\ w_1 + 2w_2 + 2b - 1 \\ 2w_1 + 2w_2 + 4b - 2 \end{bmatrix}, \qquad \nabla J(0,1,1) = \begin{bmatrix} 1 \\ 3/2 \\ 2 \end{bmatrix}.$$

The gradient is non-zero, so $(0,1,1)$ is not a local minimum; a gradient step with $\eta = 0.5$ moves to $\theta - \eta\nabla J = (-\frac{1}{2}, \frac{1}{4}, 0)$. Running gradient descent to convergence gives $w^* = (0,0)$, $b^* = \frac{1}{2}$, where the gradient vanishes. The model predicts $\frac{1}{2}$ for every input, for example for $x = (1,0)$: it has not learned XOR.

Deep linear model. Add a hidden layer of two linear units, $h = W^T x + c$ and $y = w^T h + b$, with $W \in \mathbb{R}^{2\times 2}$, $c, w \in \mathbb{R}^2$, $b \in \mathbb{R}$: $4 + 2 + 2 + 1 = 9$ parameters. But

$$y = w^T(W^T x + c) + b = w^T W^T x + w^T c + b = w'^T x + b',$$

so a deep linear model with any number of layers is a linear model end to end, and it again predicts the constant $\frac{1}{2}$.

ReLU hidden layer. Now let $h = g(W^T x + c)$ with the ReLU $g(z) = \max(z, 0)$ applied elementwise, and try

$$W = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \qquad c = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \qquad w = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \qquad b = 0.$$

Since $W^T x = (x_1 + x_2)(1, 1)^T$, we get $h = \big(x_1 + x_2,\ \max(x_1 + x_2 - 1, 0)\big)$ and

x	$W^T x + c$	h	$y = h_1 - 2h_2$
$(0,0)$	$(0,-1)$	$(0,0)$	0
$(0,1)$, $(1,0)$	$(1,0)$	$(1,0)$	1
$(1,1)$	$(2,1)$	$(2,1)$	0

The network is correct on all four inputs. The hidden layer maps the four points to three points of a new feature space where the classes are linearly separable (Figure 25).

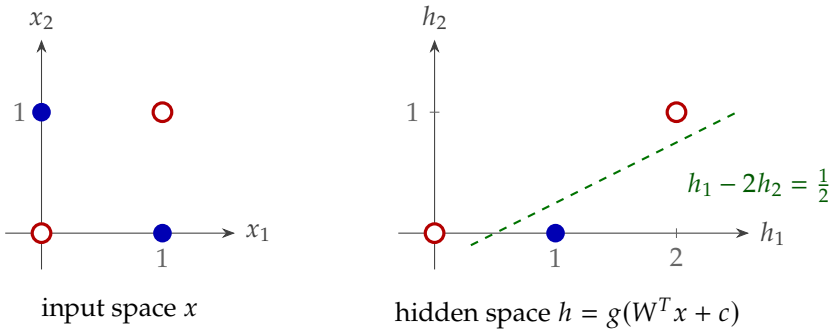


Figure 25: XOR before and after the ReLU hidden layer. Filled points have label 1, open points label 0. The hidden layer maps $(0,0) \mapsto (0,0)$, both $(0,1)$ and $(1,0)$ to $(1,0)$, and $(1,1) \mapsto (2,1)$; the dashed line, where the output $h_1 - 2h_2$ equals $\frac{1}{2}$, separates the classes.

So a network with one hidden layer and linear activations cannot learn XOR, and neither can a linear network with any number of hidden layers. A single hidden layer with ReLU activations can, and networks with one hidden ReLU layer can approximately learn any function, although the hidden layer can be exponentially wide in the input dimension.

The parameters are far from unique: other weights $W_{\text{new}}, c_{\text{new}}, w_{\text{new}}, b_{\text{new}}$ give the same output for every input x. For instance, permuting the hidden units changes nothing, and since $g(\alpha z) = \alpha g(z)$ for $\alpha > 0$, scaling the weights and bias into a hidden unit by α and its outgoing weight by $1/\alpha$ changes nothing either. This raises the questions of how to check whether two networks compute the same function (detecting stolen weights or verifying a watermark), and of *backdoors*: weights that look like the original but respond differently on specific inputs.

17.6 Building and training networks in PyTorch

PyTorch is the most widely used framework for building and training deep networks (related frameworks include TensorFlow and JAX). Its main features are automatic differentiation, support for hardware accelerators, and easy access to state-of-the-art architectures; training is some variant of gradient descent that you implement, using gradients computed automatically.

- `torch.tensor` encodes data, weights, and activations and behaves much like `numpy.ndarray`: construction (`torch.tensor([[1.0],[2.0]])`, `torch.ones(2,3)`), slicing (`x[:,3]`), `x.shape`, `x.dtype`, and operations such as `torch.mean(x, axis=1)` and `a.T @ x`. Unlike arrays, tensors are placed on devices (`x.to("cuda")`) and can record how they were computed (`x.grad_fn`), which enables automatic differentiation.
- `torch.nn.functional` holds stateless functions such as `relu`; `torch.nn` holds the building blocks. `nn.Module` is the base class of every model and layer: its `__init__` defines the parameters (wrapped as `nn.Parameter` so they are tracked) and `forward` defines the computation. Hundreds of layer modules exist, such as `nn.Linear` and `nn.Softmax`.

A multilayer perceptron extends `nn.Module`; autograd then computes the gradient of the loss with respect to all parameters and attaches it to each parameter:

```python
class MLPModel(nn.Module):
    def __init__(self, dims, activation="relu"):
        super().__init__()
        self.layers = nn.ModuleList(
            [nn.Linear(dims[i], dims[i+1]) for i in range(len(dims)-1)])
        self.act = nn.functional.relu if activation == "relu" else torch.tanh

    def forward(self, x):
        for i, layer in enumerate(self.layers):
            x = layer(x)
            if i < len(self.layers) - 1:   # no activation after the last layer
                x = self.act(x)
        return x

model.zero_grad()             # reset gradients (needed when iterating)
loss = loss_fn(model(x), t)
loss.backward()               # gradient of the loss
print(model.layers[0].weight.grad)
```

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 17 (Spring 2026).

Lecture 18: Neural Networks and Backpropagation

The set-up is the same as for earlier models. The training data are $S = \{(x_i, y_i)\}_{i=1}^N$ with $x_i \in \mathbb{R}^D$ (for classification, $y_i \in \{-1, +1\}$); the model class $f(x \mid w)$ is now a composition of functions, a neural network $y = h_L\big(W^{(L)} \cdots h_2\big(W^{(2)} h_1(W^{(1)} x)\big)\big)$; the loss is the log-likelihood $LL(w) = \sum_i \log p(y_i \mid x_i, w)$; and the learning objective is $\hat{w} = \arg\min_w \{-LL(w)\}$, solved by gradient descent on $J(w) = -LL(w)$.

Algorithm 18.1: Gradient descent on $J(w)$

```
w^(0) ← initialize
for τ = 1, ..., τ_max do
    w^(τ) ← w^(τ−1) − η ∇J(w^(τ−1))
    if converged(w^(τ), w^(τ−1), ε) then terminate
```

In practice networks are trained with stochastic gradient descent and its refinements, such as Adam (Lecture 13). Taking gradients of a huge composition looks complicated; BACKPROPAGATION is the recipe, and it is the calculus chain rule organized as an algorithm.

18.1 A simple example by the chain rule

For univariate functions, $\frac{d}{dt} f(x(t)) = \frac{df}{dx}\frac{dx}{dt}$. Consider univariate non-linear regression with all quantities scalar:

$$z = wx + b, \qquad y = \sigma(z), \qquad \mathcal{L} = \tfrac{1}{2}(y - t)^2,$$

where t is the target. Gradient descent needs $\partial\mathcal{L}/\partial w$ and $\partial\mathcal{L}/\partial b$. Differentiating the composition directly, as in a calculus class,

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial}{\partial w}\left[\tfrac{1}{2}(\sigma(wx + b) - t)^2\right] = (\sigma(wx + b) - t)\frac{\partial}{\partial w}(\sigma(wx + b) - t) = (\sigma(wx + b) - t)\,\sigma'(wx + b)\,x,$$
$$\frac{\partial \mathcal{L}}{\partial b} = (\sigma(wx + b) - t)\,\sigma'(wx + b).$$

The two derivations repeat the same work. Instead, work backwards from the loss, computing each intermediate derivative once:

$$\frac{d\mathcal{L}}{dy} = y - t, \qquad \frac{d\mathcal{L}}{dz} = \frac{d\mathcal{L}}{dy}\,\sigma'(z), \qquad \frac{\partial \mathcal{L}}{\partial w} = \frac{d\mathcal{L}}{dz}\,x, \qquad \frac{\partial \mathcal{L}}{\partial b} = \frac{d\mathcal{L}}{dz}.$$

The goal is not a closed form but a program that computes the derivatives efficiently. Write $\bar{v} = d\mathcal{L}/dv$, the ERROR SIGNAL of v. Then the backward computation reads

$$\bar{y} = y - t, \qquad \bar{z} = \bar{y}\,\sigma'(z), \qquad \bar{w} = \bar{z}\,x, \qquad \bar{b} = \bar{z}.$$

18.2 Computation graphs and backpropagation

A COMPUTATION GRAPH has a node for every input and computed quantity, and an edge from each node to the nodes computed directly from it. Computing the loss is the FORWARD PASS; computing the derivatives, in the reverse direction, is the BACKWARD PASS. Let $v_1, \dots, v_N$ be a TOPOLOGICAL ORDERING of the graph (parents before children), with v_N the variable whose derivatives we want, typically the loss. In the example, $v_1 = b$, $v_2 = w$, $v_3 = z$, $v_4 = y$, $v_5 = \mathcal{L}$ (the data x, t are constants). When every node has a single child, the chain rule gives $\bar{v}_i = \bar{v}_{\text{child}(i)}\, \partial v_{\text{child}(i)}/\partial v_i$, starting from $\bar{v}_N = 1$.

Nodes can have several children (FAN-OUT greater than 1). In regularized regression (Figure 26a) the weight w feeds both z and the penalty $\mathcal{R} = \frac{1}{2}w^2$, and $\mathcal{L}_{\text{reg}} = \mathcal{L} + \lambda\mathcal{R}$. In multiclass logistic regression, with $z_\ell = \sum_j w_{\ell j} x_j + b_\ell$, $y_k = e^{z_k}/\sum_\ell e^{z_\ell}$ and $\mathcal{L} = -\sum_k t_k \log y_k$, every z_ℓ feeds every y_k. This needs the multivariate chain rule: for $f(x(t), y(t))$ with scalar variables,

$$\frac{d}{dt} f(x(t), y(t)) = \frac{\partial f}{\partial x}\frac{dx}{dt} + \frac{\partial f}{\partial y}\frac{dy}{dt}.$$

The total effect of t on f is the sum over all the separate paths through which it exerts influence. For example, with $f(x, y) = y + e^{xy}$, $x(t) = \cos t$, $y(t) = t^2$,

$$\frac{df}{dt} = \left(ye^{xy}\right)(-\sin t) + \left(1 + xe^{xy}\right) 2t.$$

In error-signal notation, $\bar{t} = \bar{x}\,\frac{dx}{dt} + \bar{y}\,\frac{dy}{dt}$, where $\bar{x}$ and $\bar{y}$ are messages already computed by the program.

Algorithm 18.2: Backpropagation

Input: topological ordering $v_1, \dots, v_N$ of the computation graph

// forward pass

for $i = 1, \dots, N$ **do**

  compute v_i as a function of its parents $\text{Pa}(v_i)$

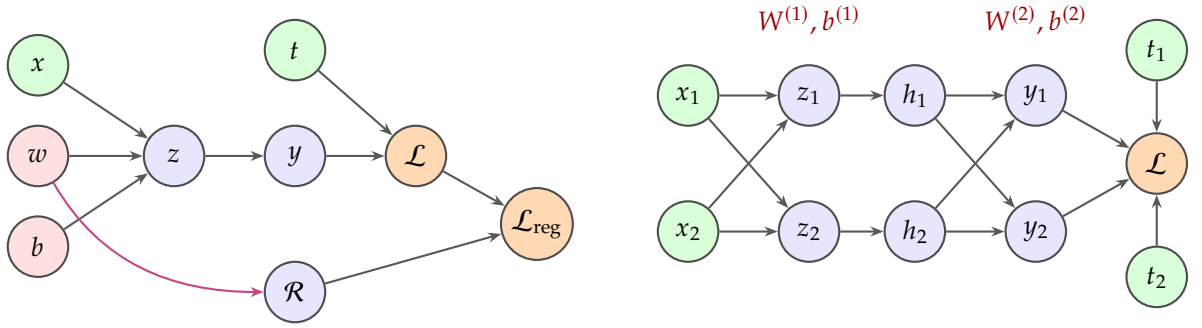
// backward pass

$\bar{v}_N \leftarrow \partial\mathcal{L}/\partial\mathcal{L} = 1$

for $i = N-1, \dots, 1$ **do**

  $\bar{v}_i \leftarrow \sum_{j \in \text{Ch}(v_i)} \bar{v}_j \frac{\partial v_j}{\partial v_i}$

The forward pass is needed for two reasons: the local derivatives $\partial v_j/\partial v_i$ are evaluated at the forward values (here $\sigma'(z)$ and x), and it tracks the loss during training.



(a) L_2-regularized regression (b) multilayer perceptron

Figure 26: Computation graphs. Parameters are red, data green, losses orange. In (a) the weight w has two children, z and $\mathcal{R}$. In (b) each z_i also receives the first-layer weights and bias, and each y_k the second-layer ones.

Regularized regression. The forward pass computes $z = wx + b$, $y = \sigma(z)$, $\mathcal{L} = \frac{1}{2}(y-t)^2$, $\mathcal{R} = \frac{1}{2}w^2$, and $\mathcal{L}_{\text{reg}} = \mathcal{L} + \lambda\mathcal{R}$. The backward pass gives

$$\bar{\mathcal{L}}_{\text{reg}} = 1, \qquad \bar{\mathcal{R}} = \bar{\mathcal{L}}_{\text{reg}}\frac{d\mathcal{L}_{\text{reg}}}{d\mathcal{R}} = \lambda, \qquad \bar{\mathcal{L}} = \bar{\mathcal{L}}_{\text{reg}}\frac{d\mathcal{L}_{\text{reg}}}{d\mathcal{L}} = 1, \qquad \bar{y} = \bar{\mathcal{L}}\,(y-t), \qquad \bar{z} = \bar{y}\,\sigma'(z),$$
$$\bar{w} = \bar{z}\frac{\partial z}{\partial w} + \bar{\mathcal{R}}\frac{d\mathcal{R}}{dw} = \bar{z}\,x + \bar{\mathcal{R}}\,w, \qquad \bar{b} = \bar{z}.$$

The two terms of $\bar{w}$ come from the two children of w.

Multilayer perceptron. For one hidden layer with sigmoid units and several outputs (Figure 26b), the forward and backward passes are

$$z_i = \sum_j w_{ij}^{(1)} x_j + b_i^{(1)}, \qquad \bar{\mathcal{L}} = 1, \qquad \bar{y}_k = \bar{\mathcal{L}}\,(y_k - t_k),$$
$$h_i = \sigma(z_i), \qquad \overline{w_{ki}^{(2)}} = \bar{y}_k\,h_i, \qquad \overline{b_k^{(2)}} = \bar{y}_k,$$
$$y_k = \sum_i w_{ki}^{(2)} h_i + b_k^{(2)}, \qquad \bar{h}_i = \sum_k \bar{y}_k\,w_{ki}^{(2)}, \qquad \bar{z}_i = \bar{h}_i\,\sigma'(z_i),$$
$$\mathcal{L} = \tfrac{1}{2}\sum_k (y_k - t_k)^2, \qquad \overline{w_{ij}^{(1)}} = \bar{z}_i\,x_j, \qquad \overline{b_i^{(1)}} = \bar{z}_i.$$

The fan-out of each h_i to all outputs is why $\bar{h}_i$ is a sum. In vector notation, with $z = W^{(1)}x + b^{(1)}$, $h = \sigma(z)$, $y = W^{(2)}h + b^{(2)}$, $\mathcal{L} = \frac{1}{2}\|t - y\|^2$:

$$\bar{y} = \bar{\mathcal{L}}\,(y-t), \quad \overline{W^{(2)}} = \bar{y}\,h^T, \quad \overline{b^{(2)}} = \bar{y}, \quad \bar{h} = W^{(2)T}\bar{y}, \quad \bar{z} = \bar{h} \odot \sigma'(z), \quad \overline{W^{(1)}} = \bar{z}\,x^T, \quad \overline{b^{(1)}} = \bar{z}.$$

18.3 Message passing and cost

Backpropagation is message passing: each node receives messages $\bar{v}_j \, \partial v_j / \partial v_i$ from its children, sums them into its error signal, and passes messages on to its parents. This gives modularity: each node only has to know how to differentiate with respect to its own arguments, and nothing about the rest of the graph. The gradient thus has three complementary views: geometric (the gradient in weight space), algebraic (the mechanics of computing the derivatives), and implementational (efficient computation on a computer).

Proposition 18.1 [Cost of backpropagation]. *The forward pass costs one multiplication per weight and the backward pass two, so a gradient costs about as much as two forward passes: linear in the number of parameters, inputs, and hidden units.*

Proof. In the forward pass, $w_{ij}^{(1)}$ appears in exactly one term of one sum, $z_i = \sum_j w_{ij}^{(1)} x_j + b_i^{(1)}$, carrying information from x_j to z_i. In the backward pass, $w_{ki}^{(2)}$ appears in two products: computing its own error signal $\overline{w_{ki}^{(2)}} = \bar{y}_k h_i$, and propagating the error signal backward in $\bar{h}_i = \sum_k \bar{y}_k w_{ki}^{(2)}$. The remaining operations are one per unit. □

18.4 Activation functions revisited

Sigmoidal and hyperbolic tangent activations have asymptotes at the top and bottom, so their gradient is near zero there. Gradient descent easily gets stuck with zero gradients, and such units become dead units. The rectified linear unit $\mathrm{ReLU}(x) = \max(0, x)$ removes half of the problem, for $x > 0$, but units with $x < 0$ are still dead: they pass back no gradient, so their incoming weights stop changing. Tricks to reduce the number of dead units include a smaller learning rate, batch normalization, and the leaky ReLU.

18.5 Alternatives to backpropagation

Finite differences. Numerically,

$$\frac{\partial}{\partial w_i} E(w; \mathcal{D}) \approx \frac{E(w + \epsilon I_i; \mathcal{D}) - E(w - \epsilon I_i; \mathcal{D})}{2\epsilon},$$

where I_i is the i-th standard basis vector. The full gradient of a D-dimensional $w \in \mathbb{R}^D$ needs $2D$ error evaluations, each costing $O(ND)$ for N data points, for a total of $O(ND^2)$ per gradient step, quadratic in the number of parameters. With a minibatch of $N \approx 1000$ and $D \gg 10^6$ parameters, $ND^2 \gg 10^{3+2 \cdot 6} = 10^{15}$; even stochastic gradient with batch size 1 costs $D^2 \gg 10^{12}$.

Symbolic differentiation. Computer algebra systems such as SymPy express functions symbolically and construct derivative expressions automatically. This is automatic (it applies to almost any expression, with some issues for control flow and loops) and exact (no ϵ), but inefficient: the derivative expressions can be much larger than the function and contain many repeated subexpressions. Modern AUTOMATIC DIFFERENTIATION instead performs backpropagation automatically, computing derivatives as needed.

18.6 Why do deep networks work so well?

Linear regression with polynomial features of arbitrarily high order is also a universal function approximator, so universality alone does not explain why deep networks sometimes dominate empirically. This is not fully understood; it has to do with the optimization landscape created by these massive architectures, and it is an active area of theoretical research.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 18 (Spring 2026).

Lecture 19: Backpropagation (continued) and Convolutional Neural Networks

19.1 From backpropagation to automatic differentiation

Lecture 18 computed gradients by the backward pass $\bar{v}_i = \sum_{j \in \mathrm{Ch}(v_i)} \bar{v}_j \, \partial v_j / \partial v_i$ over a topological ordering of the computation graph. Finite differences are no substitute: they need $2L$ evaluations of the loss for $w \in \mathbb{R}^L$, each a forward pass over N data points touching all L parameters, so $O(NL^2)$ per gradient step. Symbolic differentiation suffers from expression swell: for $f(x) = u(x)v(x)$, $f'(x) = u'(x)v(x) + u(x)v'(x)$, so u and v must be evaluated both for f and for f'; when u and v are themselves products, the duplication nests and the expressions grow rapidly in complexity.

Backpropagation used to be applied by hand: figure out the computation graph, code the forward pass, write down the local derivatives $\partial v_j / \partial v_i$, code them, and check the result against finite differences. Today one uses autodiff: we specify only the forward network, and the autodiff engine figures out the computation graph, calculates all needed derivatives, and applies the backpropagation algorithm. Through a ReLU it uses the subgradient: derivative 0 for $x < 0$ and 1 for $x \geq 0$.

19.2 Initialization

The loss surface of a neural network is highly non-convex, so the starting point affects both the quality of the solution and the speed of convergence. A good starting point is one with large gradients, for the fastest learning; for sigmoid, tanh, and ReLU, the steepest region of the activation is typically near zero. That suggests initializing all weights to zero, but then every unit in a layer computes the same value and receives the same gradient, so the units stay identical and the network never uses its width.

In practice, initialize the weights *near* zero, stochastically: $w_i^{(l)} \sim \mathcal{N}\left(0, \epsilon_{(l)}^2\right)$. For example, He initialization for ReLU sets $\epsilon_{(l)}^2 = 2/n_{\text{inputs}}$, where n_{inputs} is the number of inputs to the layer. It also helps to train from several initializations and pick the best network or average the results.

19.3 Batch normalization and input normalization

Initialization starts the activations in the region of healthy gradients, but once learning starts nothing keeps them there. Batch normalization controls the inputs to the non-linearities so that gradients keep flowing.

Definition 19.1 [Batch normalization]. Let $a_{km}^{(l)}$ be pre-activation m of layer l for the k-th data point

of an SGD mini-batch of K points. Batch normalization replaces it by

$$\hat{a}_{km}^{(l)} = \frac{a_{km}^{(l)} - \mu_m^{(l)}}{\sqrt{\left(\sigma_m^{(l)}\right)^2 + \delta}}, \qquad \mu_m^{(l)} = \frac{1}{K}\sum_{k=1}^{K} a_{km}^{(l)}, \qquad \left(\sigma_m^{(l)}\right)^2 = \frac{1}{K}\sum_{k=1}^{K}\left(a_{km}^{(l)} - \mu_m^{(l)}\right)^2,$$

the mean and variance over the mini-batch, with a small $\delta > 0$. The normalized $\hat{a}^{(l)}$ is then passed to the activation function.

In training these are just differentiable operations, so backpropagation handles them. At test time we want the mean and variance of each layer over all the training data, but training only ever sees batches. Computing them from the entire training set after training would be ideal but too expensive; instead, keep a running, exponentially decaying average of the batch statistics during SGD.

The training data are also normalized. Input variables may lie in very different ranges (a height in meters, a finger width in millimeters), which makes gradient descent move faster along some directions than others. Continuous variables are therefore standardized to zero mean and unit variance,

$$\mu_i = \frac{1}{N}\sum_{n=1}^{N} x_{ni}, \qquad \sigma_i^2 = \frac{1}{N}\sum_{n=1}^{N}(x_{ni} - \mu_i)^2, \qquad \tilde{x}_{ni} = \frac{x_{ni} - \mu_i}{\sigma_i},$$

and exactly the same transformation, with the training μ_i, σ_i, must be applied to validation and test data.

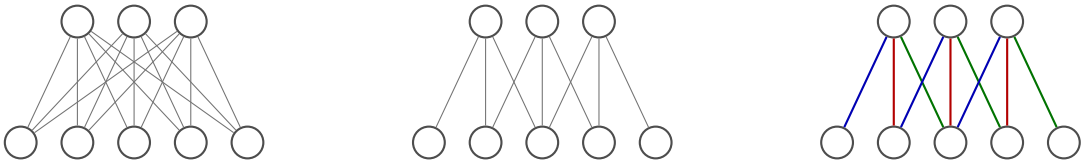
19.4 Why convolutional networks

So far all layers were FULLY CONNECTED: with $n_i(l)$ inputs and $n_o(l)$ outputs at layer l, the matrix W_l has $n_i(l) \times n_o(l)$ parameters, which adds up quickly. Fully connected layers cannot reuse features, because every region of the image (or of the hidden layer) has its own weights. A single layer with one weight vector per class does *global template matching*: trained on digit images, each class weight becomes a rigid template of a whole digit, and the predicted class is the template that best matches the image. A more compact and robust approach breaks images into small components and combines them, using a hierarchy of local feature detectors (edges, then parts, then objects) with distributed representations, feature sharing, and compositionality.

CONVOLUTIONAL NEURAL NETWORKS (CNNs, convnets) learn a set of feature detectors that are applied at every image location. They do this in two steps (Figure 27).

1. Hidden units are only LOCALLY CONNECTED: each looks at a small region of the input.
2. The weights are SHARED between all locations. One FILTER W scanned across the input maps it to an output FEATURE MAP; the depth of the output volume is the number of filters, that is, the number of feature maps.

Convolutional layers can be stacked, which gives a hierarchical decomposition into parts. On the ImageNet challenge (ILSVRC), error fell from 28.2% (2010) and 25.8% (2011) to 16.4% with the first CNN-based winner, the 8-layer AlexNet (2012), then 11.7% (2013), and 7.3% and 6.7% with the 19- and 22-layer VGG and GoogLeNet (2014).



(a) fully connected: each unit sees the whole input

(b) locally connected: each unit sees a small region

(c) convolutional: local, weights shared (same colors)

Figure 27: From fully connected to convolutional layers, in one dimension. In (c) all hidden units apply the same three weights to their own window.

19.5 Convolution

Intuitively, convolution scans a FILTER (also called a KERNEL or template) across the image, and at every position takes the elementwise product with the image patch and sums. The STRIDE is the step between positions: stride 1 skips nothing, stride 2 skips every other position.

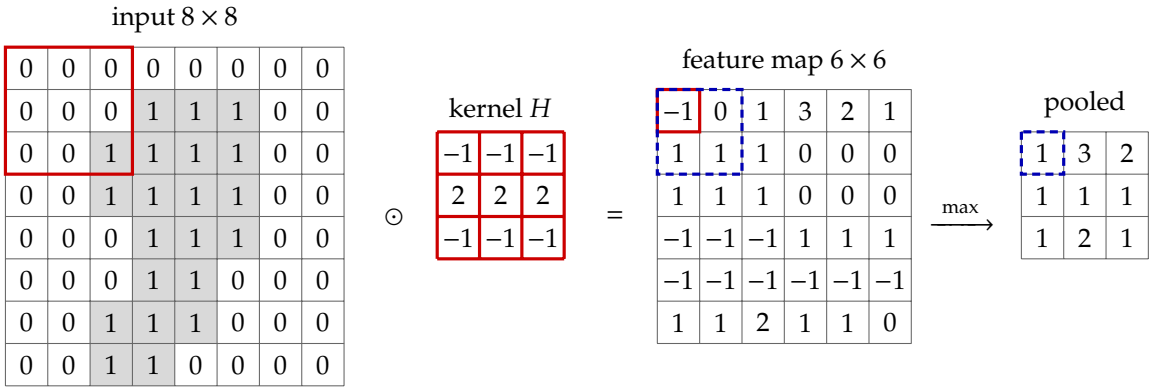


Figure 28: Scanning H over the image (stride 1, no padding). The red window gives the red output entry. Max pooling over 2×2 blocks (dashed) gives the pooled map.

Example: Figure 28 scans the horizontal-line detector H across a downsampled 8×8 image of a 9 with stride 1. The first window covers rows and columns 1 to 3 and gives $-1 \cdot (0+0+0) + 2 \cdot (0+0+0) - 1 \cdot (0+0+1) = -1$. The largest response, 3, is at the top stroke of the 9. The vertical-line

detector $V = H^T$ gives the second feature map

$$H = \begin{bmatrix} -1 & -1 & -1 \\ 2 & 2 & 2 \\ -1 & -1 & -1 \end{bmatrix}, \quad V = \begin{bmatrix} -1 & 2 & -1 \\ -1 & 2 & -1 \\ -1 & 2 & -1 \end{bmatrix}, \qquad \text{map}_V = \begin{bmatrix} -1 & 0 & 1 & 0 & 2 & -2 \\ -2 & 1 & 1 & 0 & 3 & -3 \\ -2 & 1 & 1 & 0 & 3 & -3 \\ -1 & -1 & 2 & 1 & 1 & -2 \\ -1 & -1 & 2 & 2 & -1 & -1 \\ -2 & 1 & 2 & 1 & -2 & 0 \end{bmatrix},$$

whose 2×2 max pooling is $\left[\begin{smallmatrix} 1 & 1 & 3 \\ 1 & 2 & 3 \\ 1 & 2 & 0 \end{smallmatrix}\right]$.

Definition 19.2 [Convolution]. For one-dimensional arrays a and b, the convolution is

$$(a * b)_t = \sum_\tau a_\tau \, b_{t-\tau}.$$

For two-dimensional arrays A and B, $(A * B)_{ij} = \sum_s \sum_t A_{st} \, B_{i-s,\, j-t}$. An n-dimensional convolution uses an n-dimensional filter.

Convolution is commutative, $a * b = b * a$, and linear, $a * (\lambda_1 b + \lambda_2 c) = \lambda_1 a * b + \lambda_2 a * c$. The index $t - \tau$ *flips* the filter before the elementwise products (*flip-and-filter*); the scanning computation above applies the kernel unflipped, which is the same as convolving with the flipped kernel. Since the filters of a network are learned, the two conventions are interchangeable. For example,

$$(2, -1, 1) * (1, 1, 2) = (2, 1, 4, -1, 2) = \begin{bmatrix} 1 & & \\ 1 & 1 & \\ 2 & 1 & 1 \\ & 2 & 1 \\ & & 2 \end{bmatrix} \begin{bmatrix} 2 \\ -1 \\ 1 \end{bmatrix},$$

so convolution is multiplication by a matrix W_k with $5 \times 3 = 15$ entries but only 3 free parameters. This is why convnets have relatively few parameters. In two dimensions,

$$\begin{bmatrix} 1 & 3 & 1 \\ 0 & -1 & 1 \\ 2 & 2 & -1 \end{bmatrix} * \begin{bmatrix} 1 & 2 \\ 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 & 5 & 7 & 2 \\ 0 & -2 & -4 & 1 \\ 2 & 6 & 4 & -3 \\ 0 & -2 & -2 & 1 \end{bmatrix}.$$

Familiar image filters show what kernel values do:

$$\underbrace{\begin{bmatrix} 0 & 1 & 0 \\ 1 & 4 & 1 \\ 0 & 1 & 0 \end{bmatrix}}_{\text{blurring}}, \quad \underbrace{\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}}_{\text{oriented edges}}, \quad \underbrace{\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}}_{\text{sharpen}}.$$

In a neural network the filter entries $w_{11}, \ldots, w_{33}$ are not designed but learned from the data with backpropagation.

19.6 Size and cost of a convolutional layer

Proposition 19.3 [Output size and cost]. *For a $D \times D$ image, N filters of size $K \times K$, stride 1, and no zero-padding, each filter produces a $(D-K+1) \times (D-K+1)$ feature map, so the layer outputs $N(D-K+1)^2$ values. Each output entry costs K^2 multiplications, so a layer with F filters costs $FK^2(D-K+1)^2$.*

Proof. A $K \times K$ window fits at $D-K+1$ horizontal and $D-K+1$ vertical offsets. For example $D=4$, $K=3$ gives a 2×2 output. □

A grayscale image of size $J \times K$ with F filters of size $M \times M$ gives F feature maps of size $(J-M+1) \times (K-M+1)$. An image with depth C (for example $C=3$ color channels) uses F kernels of size $M \times M \times C$, sometimes described as $F \times C$ filters of size $M \times M$; each feature map sums over the C channels and the output still has F maps of size $(J-M+1) \times (K-M+1)$. Through a network the spatial dimensions of the features decrease while the depth usually increases. To keep the spatial size, zero-pad the input: with a 3×3 kernel, one border of zeros turns the 8×8 image into a 10×10 array and the feature map is again 8×8.

19.7 Pooling and the full architecture

A pooling layer downsamples feature maps. Max pooling with 2×2 blocks replaces each block by its maximum (Figure 28). Pooling reduces the size of the feature maps and builds in invariance to small transformations, such as a face that is not perfectly centered.

A convnet stacks two kinds of layers deeply: convolutional (detection) layers, each a set of filters producing feature maps, followed by a non-linearity $y_i = \max(0, z_i)$ on the convolution outputs z_i; and pooling layers that subsample these maps. Because of pooling, higher-layer filters cover a larger region of the input than equal-sized filters in lower layers. After the feature-construction layers, typically a few (for example two) fully connected layers feed a classifier (such as softmax) or a regressor (such as linear regression).

Training is gradient descent with backpropagation, as before, with two twists: because the weights

are shared, the gradient of a filter is accumulated over all positions where it was applied; and through max pooling the gradient is backpropagated only through the unit that won the max (technically a subgradient).

The architecture (depth, width, layer types) is chosen by cross-validation, as throughout the course. A well-known example is VGG, which takes a 224×224×3 input through stacks of convolution+ReLU layers and max pooling ($224 \times 224 \times 64$, $112 \times 112 \times 128$, $56 \times 56 \times 256$, $28 \times 28 \times 512$, $14 \times 14 \times 512$, $7 \times 7 \times 512$) to fully connected layers of width 4096 and a softmax. All such architectures rest on the same fundamentals (layers, backpropagation for gradient descent, convolution, pooling), and innovations such as ResNets sometimes change the game. A common and effective shortcut is TRANSFER LEARNING: take an architecture fully trained on some data set, freeze the parameters of the generic feature-extraction layers, and retrain only the task-specific top layer(s) that do the regression or classification.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 19 (Spring 2026).

Lecture 20: Convolutional Neural Networks (continued)

20.1 Finishing convolutional networks

This lecture completes the convolutional networks of Lecture 19 (local connectivity, shared filters, feature maps, padding, max pooling). The neurons (hidden units) of a CNN layer are the entries of its feature maps, and they include the non-linearity: a convolutional layer computes pre-activations z_i by convolution and activations $y_i = \mathrm{ReLU}(z_i)$, and a pooling layer then downsamples these activation maps. Max pooling is standard; average pooling, which replaces each block by its mean, is possible but much less common.

In the full network, convolutional layers do feature detection, pooling layers downsample and give local invariance to the learned features (filters), and fully connected layers make the final prediction through a head: a binary or multi-class classifier or a regressor, given by its likelihood (for example binomial or Gaussian).

The receptive field of a neuron is the region of the input that affects it. Because of pooling, neurons in higher layers have larger receptive fields than neurons in lower layers, even when all filters have the same size; larger strides or filter sizes have the same effect.

20.2 Training convolutional networks

The loss is the negative log-likelihood (cross-entropy), minimized by gradient descent with backpropagation. The gradient of a shared 2D filter is accumulated over all positions where it was applied. Max pooling passes the gradient only to the input that won the max: its local derivative $\partial v_j / \partial v_i$ is the subgradient, 1 for the winner and 0 for the rest.

Example: Max pooling the 2×2 region $X = \left[\begin{smallmatrix} 1 & 3 \\ 2 & 9 \end{smallmatrix}\right]$ gives $Y = 9$. If the gradient arriving from the deeper layers is $\partial L / \partial Y = 5$, then

$$\frac{\partial L}{\partial X} = \begin{bmatrix} 0 & 0 \\ 0 & 5 \end{bmatrix}.$$

The value 9 was the only element that contributed to the output, so it receives the entire gradient; the other elements are disconnected from the output and receive 0.

Cross-validation is too expensive for networks of this size, so the architecture (depth, width, layer types) is chosen on a hold-out validation set; the well-known VGG(-Face) architecture from 2014 already has millions of parameters. Reusing a model trained on lots of data, with frozen feature-extraction layers and a retrained top (Lecture 19), is a common alternative that can work quite well.

20.3 Early stopping

During training it is standard to plot a LEARNING CURVE: the training and validation error against the number of gradient steps. The training error keeps decreasing, while the validation error eventually starts to increase. EARLY STOPPING stops training, returning to the saved model checkpoint, when the validation error stops decreasing.

Early stopping is a form of regularization. Models typically start with relatively small weights, so stopping gradient descent early leaves the weights $\tilde{w}$ between the near-zero starting point and the error-minimizing weights w^* (Figure 29). It can therefore be viewed as a form of WEIGHT DECAY, the usual name for regularization in neural networks.

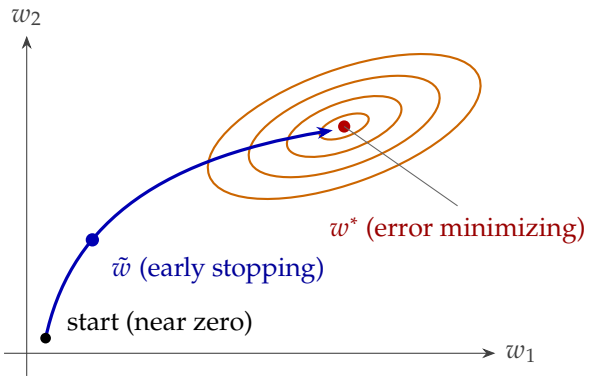


Figure 29: Early stopping as regularization (schematic). Ellipses are level sets of the training error around w^*; gradient descent from near zero is stopped at $\tilde{w}$, which has smaller weights than w^*.

20.4 Dropout

Definition 20.1 [DROPOUT]. DROPOUT regularizes a network by randomly disabling neurons in each iteration of SGD. For every unit m of layer l sample $r_m^{(l)} \sim \text{Bernoulli}(1-\rho)$ and use the modified activations

$$\tilde{z}_m^{(l)} = z_m^{(l)}\, r_m^{(l)},$$

typically with $\rho \leq 0.5$. At test time set $r_m^{(l)} = 1-\rho$, using all the activations, scaled.

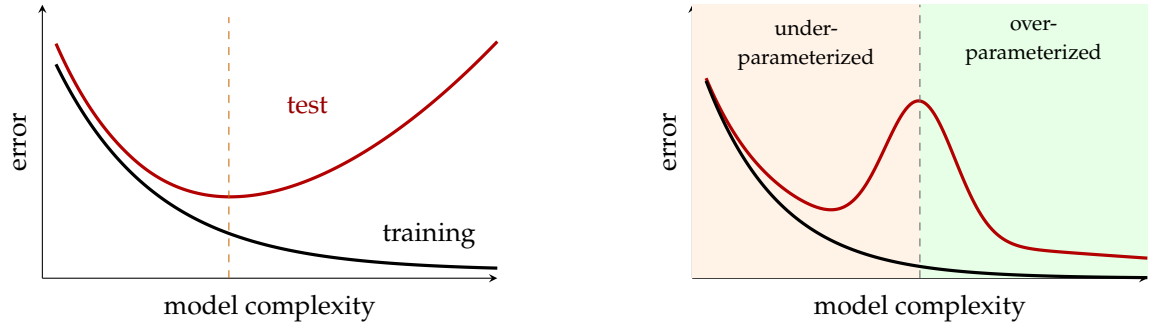
Each iteration thus trains a random subnetwork. The test-time scaling makes each activation equal to its expectation during training, $\mathbb{E}\big[z_m^{(l)} r_m^{(l)}\big] = (1-\rho)\, z_m^{(l)}$.

20.5 Double descent

Recall the bias-variance tradeoff (Lecture 14):

$$\text{test error} = \text{variance} + \text{bias}^2 + \text{irreducible noise}.$$

As model complexity grows, the training error decreases, the bias falls, and the variance rises, so the test error is U-shaped with an optimal complexity in between. Large neural networks trained with SGD show a second regime: past the point where the model can fit the training data, the test error decreases again, a phenomenon called DOUBLE DESCENT (Figure 30). Very complex models seem to self-regularize, likely because of SGD, and the optimum moves to the over-parameterized regime.



(a) classical bias-variance view

(b) double descent

Figure 30: Test error (red) and training error (black) against model complexity (schematic). (a) The classical U-shaped test error, with its optimum dashed. (b) Double descent: after a peak at the boundary between the under- and over-parameterized regimes, the test error falls again.

In experiments with ResNet18 of varying width (Nakkiran et al., ICLR 2020), the test error first follows the classical curve, peaks in a critical regime near the interpolation threshold where the training error reaches zero, and then decreases, so the largest models achieve the lowest test error. Plotted against training epochs, the test error can also descend twice (*epoch-wise* double descent), so for sufficiently large models early stopping could decrease generalization.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 20 (Spring 2026).

Lecture 21: Transformers

21.1 Deep networks as feature transformations

A deep network transforms raw features layer by layer, and each layer is a different representation of the data. The intermediate representations are called embeddings; the final layers represent the abstract concepts associated with the prediction task. Transformers, together with CNNs and residual connections, are the architectural ideas behind most of today's models, in language, vision, multimodal learning, reinforcement learning, and robotics. Attention itself predates the transformer: it was introduced for neural machine translation (Bahdanau, Cho, and Bengio, ICLR 2015), and the transformer paper (Vaswani et al., NeurIPS 2017) built a network from attention alone.

The multi-layer perceptron is a stack of [linear layer + nonlinearity], for example

$$y = \mathrm{ReLU}\big(W^{(2)}\,\mathrm{ReLU}(W^{(1)}x)\big).$$

It is highly expressive, but it has many parameters and a fixed input size. Applied to an image with P pixels, a fully connected layer from P inputs to P outputs has P^2 weights, with separate parameters for every pixel: the network must learn a separate representation of a texture at every location. A slightly bigger image brings new pixels that have no weights, so it needs a new model.

21.2 Convolutional networks and their receptive field

A convolutional layer slides (convolves) one neuron across the input, with zero padding at the border. The same weights are reused at every location (weight sharing): a 3×3 filter on a one-channel image has only $3 \times 3 \times 1$ ($w \times h \times c$) weights, used across the whole image, and stacking such layers gives deep transformed representations. A bigger image is handled by the same weights without additional training (though a large change in resolution may require deeper models and more training). So convolutional networks are parameter efficient, encode an inductive bias of spatial invariance, and allow varying input dimensions, at least spatially (not in the channel dimension).

Their weakness is locality. Understanding a part of an image often needs broader context: a red patch with a badge is recognizable as a firefighter's costume only together with the rest of the picture (a child in a firefighter jacket with Spider-Man face paint). Each activation of a convolutional layer sees only a window of the layer below.

Proposition 21.1 [Receptive field]. *In a one-dimensional stack of width-3 convolutions, an activation in layer ℓ depends on $2\ell + 1$ input pixels.*

Proof. Layer 1 sees 3 pixels. If an activation in layer $\ell - 1$ sees a contiguous window of $2\ell - 1$ pixels, an activation in layer ℓ combines three adjacent such windows, each shifted by one pixel, which together cover $2\ell - 1 + 2 = 2\ell + 1$ pixels. □

So with four layers over nine pixels, only the top-center activation sees the whole input (Figure 31): convolutional networks build local representations, and only the top layers have full context. The transformer removes this limitation; we build it from first principles, starting from language.

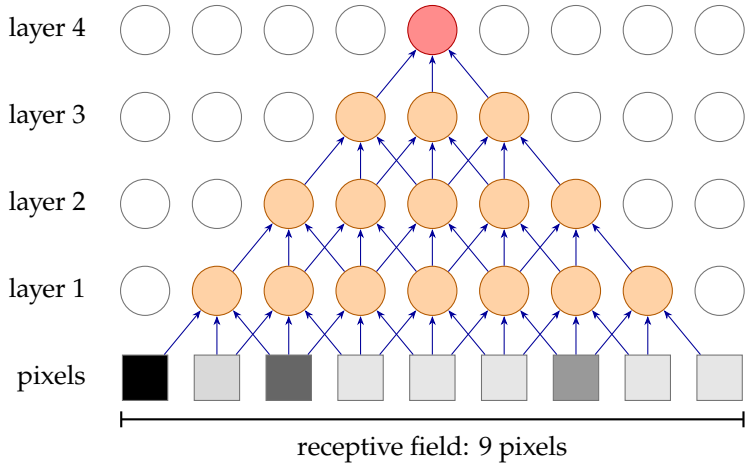


Figure 31: Receptive field of the top-center activation of four width-3 convolution layers over nine pixels. Highlighted units are those it depends on: 3, 5, 7, and finally all 9 pixels.

21.3 Representing documents as vectors

The first step in processing text is to map a document to a vector. In the BAG-OF-WORDS representation, a document (say a tweet) becomes a long vector indexed by the vocabulary, of length about 50,000, with a 1 (or a count) at the index of each word it contains; a tweet has only 5 to 10 nonzeros, so the vector is very sparse. Frequent words such as "the" and "and" dominate such vectors, while a useful representation should highlight words that are atypically frequent in this document.

Definition 21.2 [TF-IDF]. For word i in a document belonging to a collection of documents,

$$\mathrm{TF}(i) = \frac{\#\text{ times word } i \text{ appears in the document}}{\#\text{ words in the document}},$$
$$\mathrm{IDF}(i) = \log \frac{\text{total number of documents}}{\text{number of documents containing word } i},$$

and $\text{TF-IDF}(i) = \mathrm{TF}(i) \cdot \mathrm{IDF}(i)$. The inverse document frequency measures how uncommon word i is in the collection.

Example: A document has 100 words, and "football" appears 2 times, so TF = 2/100. If "football" appears in 300 of 1000 documents, IDF = log(1000/300) and TF-IDF = $\frac{2}{100} \log \frac{1000}{300} \approx 0.024$ (natural

logarithm).

21.4 Recurrent neural networks

A feed-forward network N (fully connected, convolutional, or both) maps an input to an output. A RECURRENT NEURAL NETWORK applies the same network repeatedly, feeding its output back as the next input. Typically each step has two inputs, the previous state and the previous output symbol, and produces an output symbol and a new state. It processes a sequence and produces another sequence, as in time series or text.

Image captioning. Microsoft COCO (2014) has about 120K images with 5 sentence descriptions each, such as "man in black shirt is playing guitar." The baseline captioner (Karpathy and Fei-Fei; Vinyals et al., CVPR 2015) runs a CNN on the image and uses its feature vector of dimension D as the initial state h_0 of an RNN. The first input is a <START> token; each step outputs a distribution over words, and the chosen word ("A", then "man", "is", ...) is fed in at the next step until <END>. One can also feed the image features at every step.

A classification CNN flattens its last feature map into a final representation, maps it to logits, and applies a softmax (logits $12, 11, 15$ for cat, dog, apple give probabilities $\approx 0.05, 0.02, 0.94$). Before flattening, however, the last convolutional layer is an $L \times L \times D$ grid (for example $D = 512$): each D-dimensional vector in the grid represents one block of the image, such as the top-left block containing an arm, a guitar, and a window.

21.5 Soft and hard attention

Attention lets the captioner look at different blocks for different words (Show, Attend and Tell; Xu et al., ICML 2015). Keep the $L \times L$ grid of feature vectors. At each step the RNN outputs, in addition to the distribution over words, a distribution p over the L^2 locations. An ATTENTION module combines the features into an ATTENDED FEATURE VECTOR (or CONTEXT VECTOR) $z \in \mathbb{R}^D$, which is fed to the next RNN step together with the previous word.

Definition 21.3 [SOFT AND HARD ATTENTION]. Let $a_1, \ldots, a_{L^2} \in \mathbb{R}^D$ be the feature vectors and $p_1, \ldots, p_{L^2}$ the attention distribution. SOFT ATTENTION returns the expectation $z = \sum_\ell p_\ell a_\ell$. HARD ATTENTION returns a single sampled feature vector: $z = a_\ell$ with probability p_ℓ.

With $L = 2$, features a, b, c, d, and attention weights $0.1, 0.2, 0.7, 0$, soft attention gives $z = 0.1a + 0.2b + 0.7c$, while hard attention returns a, b, or c with probabilities 0.1, 0.2, 0.7 (Figure 32). Soft attention is end-to-end differentiable, so the whole captioner trains by backpropagation. Hard attention involves sampling and is not differentiable, so it must be trained with the REINFORCE algorithm.

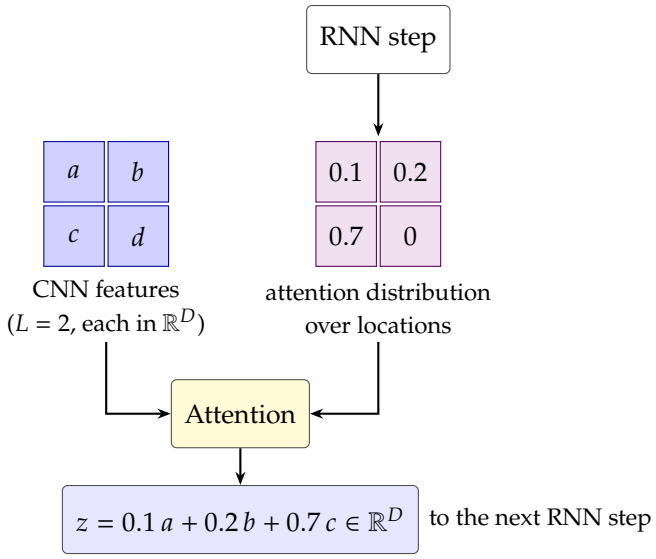


Figure 32: Soft attention over a 2×2 grid of CNN feature vectors. The RNN step produces the distribution over locations; the context vector is the weighted average of the feature vectors.

The attention maps are interpretable: when generating "frisbee" or "dog", the weight concentrates on the frisbee or the dog in the image. The same mechanism serves translation, where each output word gets a distribution over the input words ("Mi gato es el mejor" $\rightarrow$ "My cat is the best"), and many other tasks: attention over input sounds in speech recognition, over input frames in video captioning, and over the image in visual question answering.

Location-based addressing. Here the RNN outputs a distribution over a fixed number of locations. If the number of locations is not fixed in advance, we need CONTENT-BASED ADDRESSING instead: weight each item by how well it matches a query, the subject of Lecture 22.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 21 (Spring 2026).

Lecture 22: Transformers (continued)

22.1 Content-based attention

Location-based attention (Lecture 21) outputs a distribution over a fixed set of locations. Content-based attention instead weights each item by how well it matches a query q: given CNN features $x_1, \ldots, x_n \in \mathbb{R}^D$ (say $D = 512$) and a query $q \in \mathbb{R}^D$ describing what to look for (for example, an outline of Africa), attend to the parts of the image that look like q. The job of attention is to produce a probability distribution $p_1, \ldots, p_n$ over a *variable* number n of items, with p_i large when item i looks like the query.

The hardmax of a vector puts all mass on its largest entry; the softmax is the smooth version,

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}.$$

For $x = (1, 2, 3)$, $\text{hardmax}(x) = (0, 0, 1)$ and $\text{softmax}(x) = (e^1, e^2, e^3)/(e^1 + e^2 + e^3) \approx (0.09, 0.24, 0.67)$. On a matrix with rows $x_1, \ldots, x_N$, softmax acts on each row separately: row n of $\text{softmax}(X)$ is $\text{softmax}(x_n)$.

The similarity measure comes from inner-product search, a basic information-retrieval method: assign a key vector to every document, build a query vector, and return the documents in decreasing order of their inner product with the query. Instead of hand-designed vectors, the keys and queries will be produced by learned linear layers.

Definition 22.1 [Content-based attention]. In the simplest setting, keys and values are the features themselves:

$$p_i = \frac{\exp(q^T x_i)}{\sum_{j=1}^n \exp(q^T x_j)}, \qquad y = \sum_{i=1}^n p_i x_i.$$

More generally the query and keys are mapped to a common space of dimension D_k (say $D_k = 20$) by learned matrices: $q = W_q x_q$ for a query input x_q, and keys $f(x_i) = W_k x_i$. Then $p_i \propto \exp\big(q^T f(x_i)\big)$, while the output is still a combination $y = \sum_i p_i x_i$ of the values $x_i \in \mathbb{R}^D$.

22.2 Self-attention

Context determines meaning. In "I swam across the river to get to the other bank" and "I walked across the road to get cash from the other bank", the meaning of "bank" depends on the surrounding words. In translation, "The animal didn't cross the street because it was too tired" needs "it" (French *il*) to refer to the animal, while "... because it was too wide" needs "it" (*elle*) to refer to the street: the representation of a word needs connections to both earlier and later words. Likewise, an image

patch showing a child's face is understood with the patches around it. SELF-ATTENTION lets every token gather context from all tokens.

Token embeddings. The transformer takes as input a set of N TOKEN EMBEDDINGS in $\mathbb{R}^D$, the basic building blocks of an image or a text, stacked as the rows of $X \in \mathbb{R}^{N\times D}$. For images these are patches: a 16×16 patch with 3 color channels gives a vector of $16 \cdot 16 \cdot 3 = 768$ numbers. For text, each word (in practice, each word part) is mapped to a learned 768-dimensional embedding. A transformer layer maps $X \in \mathbb{R}^{N\times D}$ to $Y \in \mathbb{R}^{N\times D}$.

First attempt: average pooling. Extract the "context" of each token in isolation with a linear layer, $V = XW^{(v)}$ with $W^{(v)} \in \mathbb{R}^{D\times D_v}$ learned (think $D_v = D$ for now; what V encodes is ultimately decided by training). Then add the average context,

$$Y_n = X_n + \frac{1}{N}\sum_{i=1}^{N} V_i.$$

This combines all the extracted context, needs no additional parameters, and behaves similarly for any number of tokens. But it treats all context as equally important. For "bank", the word "river" is the most important and "swam" is important; in "The food was good, not bad at all", the word "bad" could even be harmful to the representation of "food", unless it is first transformed by its own context "not".

Keys and queries. Each token also gets a key and a query:

$$K = XW^{(k)}, \qquad Q = XW^{(q)}, \qquad W^{(k)}, W^{(q)} \in \mathbb{R}^{D\times D_k},$$

with $D_k \le D$ typically. The inner product $Q_n^T K_i$ encodes the relevance of token i to the representation of token n, and all of them form the INNER-PRODUCT MATRIX $Z = QK^T \in \mathbb{R}^{N\times N}$, $Z_{ni} = Q_n^T K_i$. Separate key and query maps allow *asymmetric* relevance: "bank" can query "river" without "river" having to query "bank". The entries Z_{ni} can be negative and need not sum to one along a row, so we apply a row softmax.

Definition 22.2 [SELF-ATTENTION]. Given $X \in \mathbb{R}^{N\times D}$, form $Q = XW^{(q)}$, $K = XW^{(k)}$, $V = XW^{(v)}$. The ATTENTION MATRIX and the output are

$$A = \mathrm{softmax}_{\mathrm{row}}\Big(\frac{QK^T}{\sqrt{D_k}}\Big) \in \mathbb{R}^{N\times N}, \quad A_{ni} = \frac{\exp\big(Z_{ni}/\sqrt{D_k}\big)}{\sum_{j=1}^{N}\exp\big(Z_{nj}/\sqrt{D_k}\big)}, \quad Y = \mathrm{Attention}(K, Q, V) = AV.$$

Each row of A is nonnegative and sums to one, and $Y_n = \sum_i A_{ni} V_i$: token n queries every token, including itself (Figure 33).

The scaling by $\sqrt{D_k}$ keeps the variance of the inner product of D_k terms close to one, which improves training: if the entries of a query and a key are independent with mean 0 and variance 1, then $\mathrm{Var}(Q_n^T K_i) = \sum_{j=1}^{D_k}\mathrm{Var}(Q_{nj}K_{ij}) = D_k$. The operation $\mathrm{Attention}(K, Q, V)$ itself has no parameters;

the learnable parameters of the layer are $W^{(q)}, W^{(k)}, W^{(v)}$, and since every step is differentiable, the layer is trained end to end by backpropagation. In PyTorch:

```python
def self_attention(Q, K, V):
    d = Q.shape[-1]
    return torch.softmax(Q @ K.T / math.sqrt(d), dim=-1) @ V

X = torch.randn(100, 10)                        # N = 100 tokens, D = 10
WQ, WK, WV = (torch.randn(10, 10) for _ in range(3))
Q, K, V = X @ WQ, X @ WK, X @ WV
Y = self_attention(Q, K, V)
```

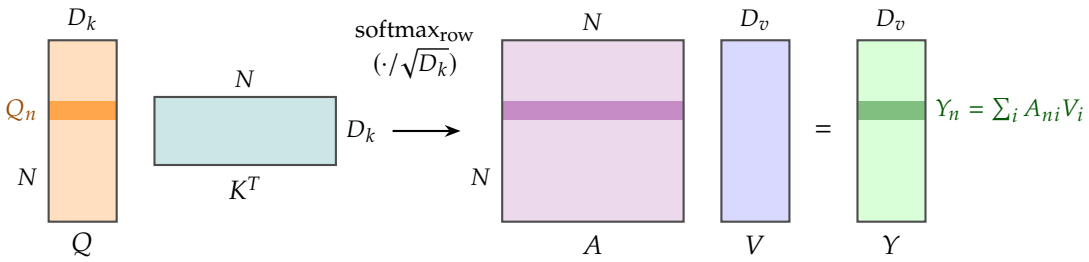


Figure 33: Self-attention in matrix form. Row n of Q (the query of token n) is compared with every key; the softmax of those scores is row n of A, and row n of the output averages the rows of V with these weights.

Cost. Constructing K, Q, and V takes $DD_v + 2DD_k$ parameters ($3D^2$ when $D_v = D_k = D$), independent of N. Computing $\text{softmax}(QK^T/\sqrt{D_k})$ costs $O(N^2 D_k)$ and computing AV costs $O(N^2 D_v)$, the dominant term. A fully connected layer on the flattened features in $\mathbb{R}^{ND_v}$ would instead need $(ND_v)^2 = N^2 D_v^2$ parameters and $O\big((ND_v)^2\big)$ computation.

22.3 Attention as a soft lookup table

A Python dictionary `D = {"A": 51, "B": 42, "C": 31}` returns `D["A"] = 51`: the query must equal a key exactly. A SOFT DICTIONARY makes keys and queries vectors and returns a mixture of values weighted by the match. With keys $k_1 = (1,0,0)$, $k_2 = (0,1,0)$, $k_3 = (0,0,1)$ and values $v_1 = 51$, $v_2 = 42$, $v_3 = 31$, the query $q = (0,1,0)$ matches k_2 exactly (a hard lookup returns 42), while $q = (1,1,0)$ matches k_1 and k_2 equally. Attention returns

$$D[q] = \sum_i p_i v_i, \qquad p = \text{softmax}(q^T k_1, q^T k_2, q^T k_3);$$

for $q = (1,1,0)$ the weights are $(e,e,1)/(2e+1) \approx (0.42, 0.42, 0.16)$, so $D[q] \approx 44.1$, mostly an even blend of 51 and 42.

22.4 Multi-head attention

There may be more than one way to extract and query the context. Multi-headed attention makes H parallel copies of the K, Q, V matrices with separate weights $W_h^{(v)}, W_h^{(k)}, W_h^{(q)}$, $h = 1, \ldots, H$. Head h produces H_n^h for token n, and the heads are combined by concatenation and a linear layer,

$$A_n = \text{vstack}\left[H_n^1, \ldots, H_n^H\right] W^{(o)}.$$

Often a smaller value dimension is used, such as $D_v = D/H$. Creating and storing keys and values for every head is costly, so keys and values can be reused across several queries: grouped-query attention (GQA) shares one key/value pair among each group of query heads, and multi-query attention shares a single key/value pair among all query heads. Multi-head attention is the most expressive and multi-query the most efficient; most current models use GQA.

22.5 The transformer layer

A transformer layer combines multi-head self-attention with a token-level neural network, typically a two-layer MLP, using residual connections and layer normalization to improve training (Figure 34):

$$Z_n = \text{LayerNorm}\big(X_n + \text{MHSA}(X)_n\big), \qquad \tilde{X}_n = \text{LayerNorm}\big(Z_n + \text{MLP}(Z_n)\big).$$

The MLP is needed because the attention output AV is a linear combination of linear transformations $V = XW^{(v)}$ of the input (even though A is a nonlinear function of X).

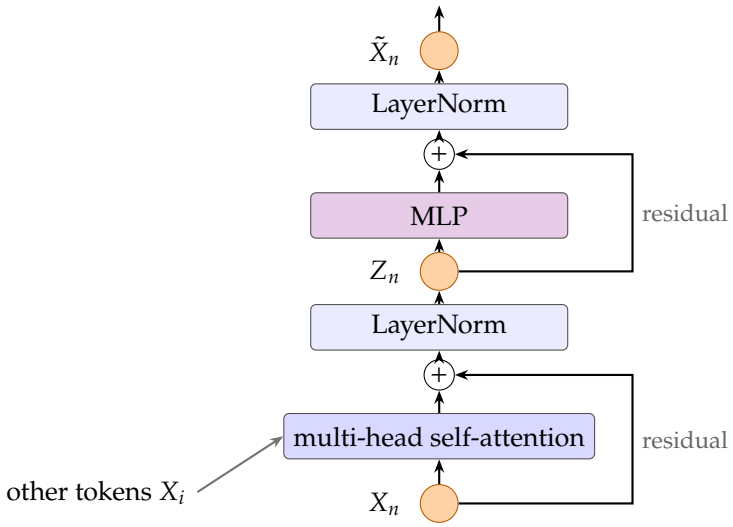


Figure 34: One transformer layer for token n: multi-head self-attention over all tokens, then a token-wise MLP, each followed by a residual addition and layer normalization.

Unrolled over the tokens, the MLP and layer norms are applied to each token separately with shared parameters, while attention connects all the tokens. Hence the number of parameters does not

increase with the number of tokens, and the per-token computations run in parallel. A transformer stacks many such layers, each with its own weights (not shared across layers).

Reading the output. For image classification, the top layer has one output per token. They can be combined by pooling and a linear layer, or by concatenation and a layer. A common trick is to add one more token, a learned `<class>` embedding, and use that token's top-layer embedding for the prediction task.

22.6 Token order and positional encoding

Proposition 22.3 [Permutation equivariance]. *Let P be an $N \times N$ permutation matrix. Then* Attention *applied to the permuted tokens PX returns PY: permuting the input tokens permutes the outputs.*

Proof. The tokens PX give PQ, PK, PV, and $(PQ)(PK)^T = P(QK^T)P^T$, which permutes the rows and the columns of Z in the same way. A row softmax commutes with this simultaneous permutation, so the attention matrix becomes PAP^T, and the output is $PAP^TPV = PAV = PY$ since $P^TP = I$. □

Each output $Y_n = \sum_i A_{ni} V_i$ is a weighted average, which does not depend on the order of the tokens. The token-wise MLP is also equivariant, so a whole transformer is. This is the right behavior for a network on *sets* of objects, but a problem for images and text: a shuffled grid of patches makes "the robot on the right" meaningless, and "The food was good not bad at all!" and "The food was bad not good at all!" differ only by a small permutation.

Definition 22.4 [Positional encoding]. A positional encoding modifies the raw features to contain the position of each token (feature engineering): a vector $r_n \in \mathbb{R}^D$ encoding position n is added to the input embedding,

$$\tilde{x}_n = x_n + r_n.$$

Concatenating vstack$[x_n, r_n]$ instead would increase the dimension and change the model. Adding does not destroy x_n: in high dimensions x_n and r_n are likely to be almost orthogonal, and a linear layer applied to the concatenation would compute $W_1 x_n + W_2 r_n$, a sum anyway. An ideal positional encoding

1. gives a unique representation $r_n \in \mathbb{R}^D$ to each location;
2. is bounded on the scale of the features (for example in $[-1, 1]$), for stable training;
3. easily expresses relative distances between tokens (an inductive bias: relative location matters in images and language);
4. works for an arbitrary number of tokens N, since sentences and images vary in size.

Learned positional embeddings (used by GPT-1) treat every r_n as a parameter, randomly initialized

with small values and learned by gradient descent. They are expressive and easy to implement, but the maximum N must be known in advance, and relative distances are not easily computed (they must be learned).

Sinusoidal encodings (the original transformer) combine waves of different frequencies: for a constant L,

$$r_{ni} = \begin{cases} \sin\big(n/L^{i/D}\big), & i \text{ even}, \\ \cos\big(n/L^{(i-1)/D}\big), & i \text{ odd}. \end{cases}$$

This is like a binary encoding of the position, but continuous: in binary, the lowest bit alternates with period 2, the next with period 4, and so on, and here coordinates oscillate at geometrically spaced frequencies.

Proposition 22.5 [Relative positions are linear]. *For every offset Δ there is a rotation matrix R_Δ, not depending on n, with $r_{n-\Delta} = R_\Delta r_n$. Moreover, $r_n^T r_m = \sum_{j=0}^{D/2-1} \cos\big(\omega_j(n-m)\big)$ with $\omega_j = L^{-2j/D}$, which depends only on $n-m$ and is largest, $D/2$, at $n = m$.*

Proof. Coordinates $2j$ and $2j+1$ form the pair $\big(\sin(\omega_j n), \cos(\omega_j n)\big)$. By the angle-difference formulas,

$$\begin{pmatrix} \sin(\omega_j(n-\Delta)) \\ \cos(\omega_j(n-\Delta)) \end{pmatrix} = \begin{pmatrix} \cos\omega_j\Delta & -\sin\omega_j\Delta \\ \sin\omega_j\Delta & \cos\omega_j\Delta \end{pmatrix} \begin{pmatrix} \sin(\omega_j n) \\ \cos(\omega_j n) \end{pmatrix},$$

so R_Δ is block diagonal with these 2×2 rotations. For the inner product, each pair contributes $\sin(\omega_j n)\sin(\omega_j m) + \cos(\omega_j n)\cos(\omega_j m) = \cos(\omega_j(n-m))$. □

So a linear layer can query a key at a fixed offset, and inner products of the encodings decay with distance (Figure 35).

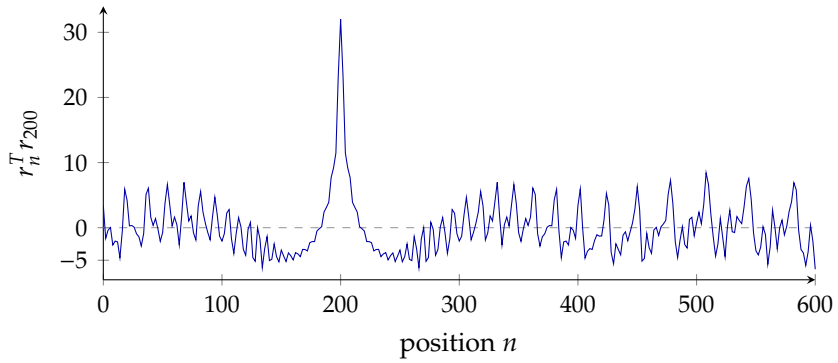


Figure 35: Inner product $r_n^T r_{200}$ of sinusoidal encodings with $D = 64$ and $L = 30$ for positions $0 \le n \le 600$, computed from the formula above. It peaks at $D/2 = 32$ when $n = 200$, drops off quickly with $|n - 200|$, and stays small and oscillating far away.

22.7 The encoder transformer

The original transformer has an encoder and a decoder. The ENCODER TRANSFORMER alone is standard for computer vision and for language embedding tasks:

1. construct the input embeddings (for example, pixel patches);
2. apply a positional encoding (for example, sinusoidal);
3. apply many repeated transformer layers;
4. use the final outputs for the downstream task (for example, by pooling or through a special token).

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 22 (Spring 2026).

Lecture 23: LLM Training and Applications

23.1 Transformers for classification and self-supervision

The original transformer has an encoder and a decoder; decoder-only architectures drop the encoder and keep only the decoder stack on top of the input embeddings. To classify a sequence, such as whether "Restaurant wasnt bad <eos>" is a positive review, embed the tokens, pass them through L transformer blocks (self-attention followed by a token-wise MLP), and apply a linear head to a representation of the whole input. A causal model uses the final representation of the last token, which has attended to every other token.

Example: *Where the parameters are.* GPT-3 175B (DaVinci) has $d \approx 12{,}000$ (12,288), context $N = 2048$ tokens, and 96 transformer blocks. The attention matrices W_q, W_k, W_v and the output projection are $d \times d$, about $4d^2 \approx 603$M parameters per block. The MLP maps $d \to 4d \to d$, so it has $2 \cdot d \cdot 4d = 8d^2 \approx 1.21$B. Across the 96 blocks this is about 58B for attention and 116B for the MLPs: most of a transformer's parameters are in the MLPs, not in attention.

In supervised learning we start from pairs $\{(x, y)\}$ and train f so that $f(x)$ is close to y, minimizing $\sum_i \|f(x_i) - y_i\|^2$ or, more generally, a training loss $\text{loss}(f(x), y)$ by gradient descent. Self-supervision gets a good representation without labels by creating a fake supervised task from the data itself. For text, the task is to predict the next word. Predicting the next word is knowledge: completing "The capital of California is _____" requires knowing "Sacramento" (and not the earlier capitals San Francisco, 1862; Benicia, 1853; Vallejo, 1852). A next-word predictor can answer questions, tell stories, and accomplish tasks: this is generative AI. In practice we model the next *token*.

23.2 Causal language modeling

Definition 23.1 [Causal language model]. A causal language model predicts the next token given the ordered tokens of the context, for example Pr("EECS-189" | "The best class at UC Berkeley is"). It generates text by auto-regressive decoding: sample a next token, append it to the context, and repeat until a stop token,

$$\text{is} \to \text{EECS-189} \to \text{!} \to \texttt{<stop>}.$$

To compute Pr("hat" | "the cat in the") with a transformer:

1. *Tokenize* the context into token ids: the, cat, in, the $\mapsto$ 142, 307, 153, 142.
2. *Embed*: look up each id in a learned embedding table with one row of length D per vocabulary entry.
3. Optionally add a fixed *positional encoding* to each embedding (one row per position, up to the

maximum context length).

4. Pass the embeddings through L repeated transformer blocks (layer norm, *masked* attention, residual, layer norm, MLP, residual). Each token ends with a $1 \times D$ vector; ignore all but the last one.
5. Multiply the last vector by the transposed embedding table (a $D \times |\text{vocabulary}|$ matrix) to get one logit per vocabulary entry, and apply a softmax: the probability that each token is the next token.

Training. ChatGPT is one example of a LARGE LANGUAGE MODEL (LLM): *G*enerative (designed to model the creation of text), *P*re-trained (trained on lots of naturally occurring data), *T*ransformer. Training uses every position at once. Prepend a `<start>` token (id 0) and shift the text by one: the inputs `<start>`, the, cat, in, the have labels the, cat, in, the, hat, so the first position predicts the first word. At each position, the output vector times the transposed embedding table, followed by a softmax, gives a distribution over the vocabulary, and the model minimizes the cross-entropy loss of the labels, which is maximum likelihood.

Masking. If tokens could attend to future tokens, the label at each position would be visible in the input ("in" could simply be copied from the next position). So a causal model uses MASKED ATTENTION: token n attends only to tokens $i \le n$,

$$\text{Attn}(K, Q, V)_n = (\alpha V)_n = \sum_{i=1}^{n} \alpha_{ni} V_i, \qquad \alpha_{ni} = \frac{\exp\left(Q_n^T K_i / \sqrt{D_k}\right)}{\sum_{j=1}^{n} \exp\left(Q_n^T K_j / \sqrt{D_k}\right)} \quad (i \le n),$$

and $\alpha_{ni} = 0$ for $i > n$. Equivalently, the scores $Q_n^T K_i / \sqrt{D_k}$ with $i > n$ are set to $-\infty$ before the row softmax, so α is lower triangular (Figure 36). Unrolled, the decoder-only transformer computes Pr(you | Can), Pr(predict | Can you), Pr(the | Can you predict), Pr(next | Can you predict the), ... in one pass, each attention layer looking only at the current and earlier tokens.

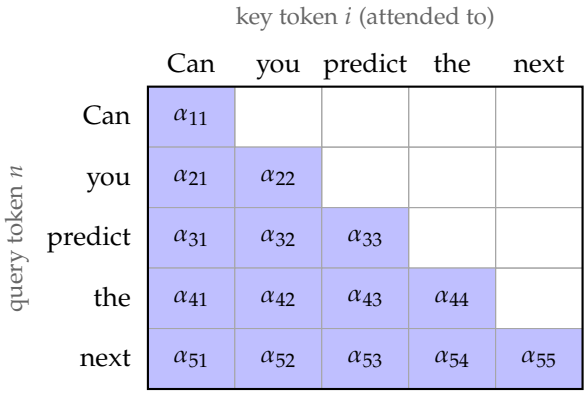


Figure 36: Masked attention on the context "Can you predict the next". Row n holds the weights α_{ni} used by token n; shaded entries ($i \le n$) may be nonzero, blank entries ($i > n$) are zero, and each row sums to one.

Encoder-decoder transformers. The original transformer used an encoder for the input and a

decoder for the output. In translation, the encoder processes "Hola, cómo estás?" with ordinary (unmasked) attention. The decoder generates "<s> Hello, how are . . . " with masked attention over its own tokens, followed in each block by CROSS-ATTENTION, whose keys and values come from the encoder outputs.

Visual language models use a vision encoder to turn image patches into embeddings and a learned *adapter* to map those into token embeddings, which are fed to the LLM alongside the text tokens ("Describe this image:"); the LLM then predicts the next token ("This . . . ") as usual.

23.3 A modern LLM

The Llama-3 architecture (see its single Python file of code) is a decoder-only transformer with:

- *RMSNorm*, a form of layer normalization, to improve training stability;
- a feed-forward network with *SwiGLU* activations;
- residual connections, also for training stability;
- *grouped-query attention* with rotary position encodings (RoPE) applied to the queries and keys;
- the block repeated N times to increase model size; after the last block come an RMSNorm, a linear layer, and a softmax.

Unlike the transformer layer of Lecture 22, which normalizes after each residual addition (POST-NORM), Llama normalizes the input of each sublayer, $x + \text{Attn}(\text{Norm}(x))$, keeping the residual path free of normalization (PRE-NORM); pre-norm works better.

Llama-3 70B Instruct has hidden size 8192, 80 layers, 64 query heads, and 8 key/value heads; GPT-3 175B has 96 layers, $d_{\text{model}} = 12{,}288$, and 96 heads of dimension 128.

Pre-training on everything. The model is trained on a large collection of data to learn generalizable patterns. GPT-3's data mix was Common Crawl (filtered, 410B tokens, 60% of the mix), WebText2 (19B, 22%, documents from Reddit links), Books1 (12B, 8%), Books2 (55B, 8%), and Wikipedia (3B, 3%). Llama-3 models were trained on 15.6T tokens from an undisclosed data mix; the 405-billion-parameter model used 16K H100 GPUs (about $25K each) and 39.3M GPU hours.

23.4 Post-training

A pre-trained model can't chat. Generative pre-training captures knowledge (finishing "Four score and seven years ago . . . " requires memorizing the text), but the resulting model only continues a statement: given "What is attorney client privilege?" it might generate "Provide a concise answer using an example from class." Post-training adjusts the model's behavior so that it follows instructions and chats like a human.

Supervised fine-tuning (SFT) runs additional training iterations on a task that differs from pre-training: a new objective (translate a sentence, follow an instruction) and new training data from a new domain, such as chat conversations. It uses a smaller learning rate to avoid CATASTROPHIC FORGETTING, where fine-tuning destroys pre-training knowledge and earlier post-training behaviors. Vicuña, the first open-source model "comparable" to ChatGPT (judged with an LLM as the judge), fine-tuned LLaMA-13B on only 70K high-quality ShareGPT conversations (about 800MB), and helped launch academic open-source generative-AI research.

Definition 23.2 [LoRA]. LOW-RANK ADAPTATION (LoRA) fine-tunes a low-rank perturbation of each pre-trained weight matrix $W \in \mathbb{R}^{D\times D}$:

$$W' = W + AB, \qquad A \in \mathbb{R}^{D\times r},\ B \in \mathbb{R}^{r\times D}, \quad r \ll D,$$

so a layer computes $W'x = Wx + A(Bx)$. Only A and B are learned; W is frozen. A is initialized randomly ($\mathcal{N}(0, \sigma^2)$) and B is initialized to zero, so before additional training the model (and its loss) is unchanged.

LoRA can be applied to all weights in the model. It trains $2Dr$ instead of D^2 parameters per matrix, which reduces cost and catastrophic forgetting, and many fine-tuned variants can share the same base weights at inference.

Synthetic data augmentation. An LLM, given prompts, can turn the original data into synthetically enriched data used to fine-tune another LLM. This encodes behaviors and teaches domain knowledge.

Reinforcement learning from human feedback (RLHF) further aligns the model with human preferences using preference data of the form "chat A is better than chat B". A reward model is learned from the human preference labels, starting from the SFT model, and the LLM is then optimized against it with reinforcement learning. RLHF makes models more robust and better in safety situations, but it can be unstable and difficult to train.

Direct preference optimization (DPO) instead applies the Bradley–Terry model directly, recasting preference learning as maximum likelihood. For a prompt x with preferred response y_w and rejected response y_l, a reference model π_{ref} (the SFT model), the logistic function σ, and a scale $\beta > 0$,

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right].$$

This is the Bradley–Terry likelihood $\Pr(y_w \succ y_l) = \sigma(r_w - r_l)$ with the implicit reward $r = \beta \log(\pi_\theta/\pi_{\text{ref}})$, so the LLM is optimized directly, with no separate reward model or reinforcement-learning loop.

23.5 Prompting and test-time compute

Context can guide a model and extract its capabilities without any gradient updates.

- Zero-shot: the model sees only a task description ("Translate English to French: cheese ⇒"); this relies on the model already "knowing" how to complete the task.
- One-shot and few-shot: the prompt also contains one or a few examples ("sea otter ⇒ loutre de mer", "peppermint ⇒ menthe poivrée", . . .). Learning a task from examples in the context is in-context learning.
- Retrieval-augmented generation (RAG) gives an LLM specialized knowledge: a retriever fetches relevant documents (say, news websites for "What happened in the stock market today?"), and the LLM receives the augmented query "Given the following set of documents: . . . What happened in the stock market today?".
- Chain-of-thought prompting shows worked reasoning in the examples. With standard prompting a model answers the question "The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?" wrongly with 27; when the example answer spells out its steps, the model writes "$23 - 20 = 3$, $3 + 6 = 9$" and answers 9 correctly.

Reasoning models, popularized by OpenAI's o1, produce long chains of thought with planning, reflection, self-refinement, and backtracking. Asked how many numbers in $\{1, 2, \ldots, 50\}$ have a perfect-square factor other than one, such a model counts multiples of $4, 9, 16, 25, 36, 49$ ($12 + 5 + 3 + 2 + 1 + 1 = 24$), notices double counting and switches to inclusion-exclusion, rechecks, and arrives at 19 after about 450 lines (3000 words) of reasoning.

Agentic systems let LLMs explore the world. The LLM is prompted with a goal and a list of tools (respond, search the web, calculator, email). It reasons ("To answer this question, I need to check the stock market news for today"), emits a tool call (search the web for "Stock Market News"), and the tool output is appended to the history for the next step, which may reason further and call another tool before responding. In such Reason–Act (ReAct) agents the human sets the goal and the model iterates between reasoning, calling tools, and output; its actions can reveal information and also modify the world.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 23 (Spring 2026).

Lecture 24: Self-Supervised Learning

24.1 Self-supervision and transfer learning

Supervised learning learns from labeled data: collect a large data set, label it, train a model, and deploy, which is costly. Unsupervised learning learns from unlabeled data by discovering patterns, through clustering of similar instances, density estimation, or dimensionality reduction; the model is not told what to predict. Neither resembles how humans learn, so we want methods that learn from raw data with no annotations.

Definition 24.1 [Self-supervised learning]. Self-supervised learning is representation learning from unlabeled data: the model learns useful feature representations by solving a pretext task, a supervised problem whose labels are created automatically from the data itself. The learned representation is then transferred to a downstream task.

Example: Pretext task: rotate each of millions of unlabeled cat and dog images by 0°, 90°, or 180° and train a ConvNet to predict the rotation; the labels come for free. Downstream task: fine-tune the learned model to classify cat versus dog from very few labeled examples.

Transfer learning is the shortcut that makes this work. Start from a fully trained image-recognition network with its weights (most well-known models are available pre-trained in TorchVision) and re-purpose it for a new recognition task: what the early layers learned are features useful for recognizing many things in images. The network splits into a trained *feature extractor* and a final linear classifier. For a new classification problem:

- with a small data set (C classes), *freeze* the feature extractor and reinitialize and train only the last layer (FC-C);
- with a bigger data set, train more of the top layers (here, all three fully connected layers) and freeze the rest;
- fine-tuning initializes the whole network with the pre-trained weights and trains all of it with a low learning rate, for example to recognize bakeries.

Pretext learning matters because representations matter: two classes that form a disk inside a ring are not linearly separable in Cartesian coordinates (x, y), but in polar coordinates (r, θ) they are separated by a threshold on r. The challenges of self-supervised learning are choosing a pretext task suited to the application, the lack of a gold standard for comparing learned representations, and choosing a loss, since there is no single objective like test accuracy in supervised learning.

Approaches fall into three families. *Generative* methods predict part of the input signal (autoencoders, GANs such as super-resolution, colorization, inpainting). *Discriminative* methods predict something

about the input signal (context prediction, rotation, jigsaw, clustering, contrastive learning). *Multimodal* methods use an additional signal besides RGB images (video, 3D, sound, language).

24.2 Generative pretext tasks

Definition 24.2 [Autoencoder]. An autoencoder learns, from unlabeled data x, an encoder f and a decoder g,

$$f(x) = s(Wx + b) = z, \qquad g(z) = s(W'z + b') = \hat{x}, \qquad \text{so that} \quad h(x) = g(f(x)) = \hat{x} \approx x,$$

where s is a nonlinearity such as the sigmoid, z is the latent representation (the code), and $\hat{x}$ is the reconstruction of x. Training minimizes the reconstruction loss $\|g(f(x)) - x\|$.

h approximates the identity function, but z is lower dimensional than x: the code is an information bottleneck, so the network must reproduce its input from a compressed learned encoding (Figure 37). A convolutional autoencoder uses a convolutional encoder F and decoder G, for example on images of handwritten digits, with loss $\|G(F(x)) - x\|$.

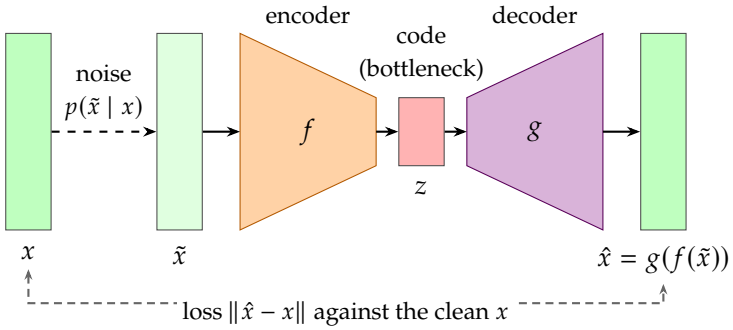


Figure 37: A (denoising) autoencoder. The encoder maps the input to a low-dimensional code z and the decoder reconstructs $\hat{x}$. A plain autoencoder encodes x itself; a denoising autoencoder encodes a corrupted $\tilde{x} \sim p(\tilde{x} \mid x)$, and both compare $\hat{x}$ with the clean x.

Denoising autoencoders learn a representation that is robust to noise. A corruption process $p(\tilde{x} \mid x)$ is applied stochastically to the input, for example setting a random subset of the inputs to 0, each with probability v, or adding Gaussian noise. The reconstruction $\hat{x} = g(f(\tilde{x}))$ is computed from the corrupted input, and the loss compares it with the noiseless x. The network still encodes the input but cannot simply mimic the identity function: it cannot trust each feature of x independently, so it must learn the patterns in the data, which forces the hidden layer to capture their general structure.

Colorization. Training data are pairs of color and grayscale images, made by passing color images through a grayscale filter; the pretext task is to predict the colors of the objects in a grayscale image. To paint objects with suitable colors, the model must understand them: the sky is blue, clouds are white, and the mountain is green. A convolutional encoder-decoder is trained with an ℓ_2 loss

between the actual color image and the predicted colorization.

Cross-channel prediction (the split-brain autoencoder) removes some of the image channels and predicts them from the others. The image X is split into a grayscale channel X_1 and color channels X_2; one network $\mathcal{F}_1$ predicts $\hat{X}_2$ from X_1, another $\mathcal{F}_2$ predicts $\hat{X}_1$ from X_2, and the two predictions are combined into the reconstruction $\hat{X}$, trained with a reconstruction loss such as cross-entropy. Any split of the channels into X_1 and X_2 can be used.

Context encoders (inpainting) remove a region of an image and predict the missing piece, which requires understanding the content of the whole image to produce a plausible replacement. An encoder and a decoder with several convolutional layers, joined by a fully connected layer, reconstruct the image under the Euclidean loss $\mathcal{L}_{\text{rec}}$. Reconstruction loss alone gives blurry fills without fine detail, so a GAN branch is added, whose adversarial discriminator judges the filled region, with total loss

$$\mathcal{L} = \lambda_{\text{rec}}\mathcal{L}_{\text{rec}} + \lambda_{\text{gan}}\mathcal{L}_{\text{gan}}.$$

The joint loss produces the most realistic images. Masks may be a central square, random blocks, or random region shapes; random blocks and random regions give better features than the central region. On PASCAL VOC the learned features are worse than supervised ImageNet features but comparable to other unsupervised methods and better than random initialization: segmentation reaches 29.7%, against 19.8% from a random Gaussian initialization and 48.0% with ImageNet labels.

Super-resolution. Training data are pairs of images and their down-sampled low-resolution versions (say, 2× smaller); the pretext task is to predict the high-resolution image. SRGAN's generator, a fully convolutional network, maps the low-resolution image to a high-resolution one; the training loss combines an ℓ_2 term with a CONTENT LOSS (part of the perceptual loss) that compares features of the actual and predicted images, and a discriminator distinguishes real from generated high-resolution images.

24.3 Discriminative pretext tasks

Rotation. Rotate each image by a random multiple of 90° (0°, 90°, 180°, 270°) and train a single ConvNet to predict which rotation was applied, a 4-class classification problem. To recognize the rotation, the model must understand the location and type of the objects in the image. On CIFAR-10, features learned this way give 89.06% classification accuracy with 4 rotations, against 88.51% with 8 rotations (multiples of 45°) and 87.46% or 85.52% with only 2 (0°, 180° or 90°, 270°). On ImageNet, linear classifiers on the frozen rotation features reach top-1 accuracy close to that of supervised features in early layers, for example 38.7% against 44.2% on the third convolutional layer, and far above random features (16.9%).

Relative patch position. Sample a patch, treat it as the center of a 3×3 grid, and sample one of its 8 neighbors; the pretext task is to predict the position of the neighbor relative to the center, an 8-class

classification (for the patch directly below the center, label 7). The two patches go through two ConvNets with shared weights (a siamese network), their features are concatenated, and dense layers classify. The model must understand the spatial context of images. To keep it from solving the task with trivial low-level cues, add gaps between the patches, jitter their positions slightly, randomly down-sample some patches and up-sample them again, and randomly drop one or two color channels of some patches.

Jigsaw puzzles. Extract a 3×3 grid of 9 patches, shuffle them, and predict the positions of all 9. Every patch goes through the same convolutional layers with shared weights, and the combined features pass through fully connected layers. There are $9! = 362{,}880$ possible puzzles; the patches are shuffled according to a fixed set of 64 permutations chosen to have the largest Hamming distance between them, and the output is the index of the permutation.

Deep clustering. Cluster the images by content (mountains, temples, ...) with k-means on the network's features, and use the cluster assignments as pseudo-labels: each cluster is a class, and the network is trained by backpropagation to predict an image's cluster. To assign images to the right clusters, the model must learn their content.

24.4 Contrastive representation learning

Augmented views of the same image (rotated, cropped, recolored) are *positives* x^+ for a reference x, and other images (a dog for a cat) are *negatives* x^-. Given a score function s, we want an encoder f with

$$s\big(f(x), f(x^+)\big) \gg s\big(f(x), f(x^-)\big),$$

high scores for positive pairs and low scores for negative pairs.

Definition 24.3 [Contrastive loss]. Given one positive x^+ and $N-1$ negatives $x_1^-, \dots, x_{N-1}^-$,

$$L = -\mathbb{E}_x\left[\log \frac{\exp s\big(f(x), f(x^+)\big)}{\exp s\big(f(x), f(x^+)\big) + \sum_{j=1}^{N-1} \exp s\big(f(x), f(x_j^-)\big)}\right].$$

This is the cross-entropy loss of an N-way softmax classifier whose logits are the scores and whose correct class is the positive.

SimCLR uses cosine similarity as the score. Positive pairs come from data augmentation: two random transformations $t, t' \sim \mathcal{T}$ (random cropping, color distortion, and blur) of the same x give $\tilde{x}_i, \tilde{x}_j$. The encoder produces representations $h_i = f(\tilde{x}_i)$, $h_j = f(\tilde{x}_j)$, and a projection network g maps them to $z_i = g(h_i)$, $z_j = g(h_j)$, the space where the contrastive loss maximizes agreement.

CLIP (contrastive language-image pre-training) uses multimodal data, images with captions, which opens a new world for self-supervision. An image encoder gives a d-dimensional representation $r_I(x)$ and a text encoder gives $r_T(t)$; they are trained so that matching image-text pairs have large

inner products and non-matching pairs small ones. Training needs a large multimodal data set of images and captions, such as LAION-2B or DataComp-12B.

Definition 24.4 [CLIP loss]**.** For a batch of N images $I_1, \dots, I_N$ with captions $T_1, \dots, T_N$, let $S_{ij} = r_I(I_i)^T r_T(T_j)$. Each image i defines an N-way classification problem whose correct class is its own caption, and each caption likewise. The CLIP loss is the sum of these softmax cross-entropy losses:

$$L = -\sum_{i=1}^{N} \log \frac{e^{S_{ii}}}{\sum_{j=1}^{N} e^{S_{ij}}} - \sum_{j=1}^{N} \log \frac{e^{S_{jj}}}{\sum_{i=1}^{N} e^{S_{ij}}}.$$

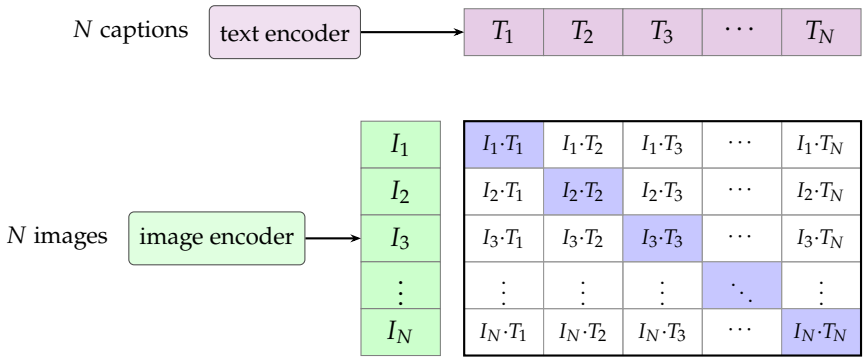


Figure 38: CLIP pre-training on a batch of N image-caption pairs. Entry (i, j) is $I_i \cdot T_j = r_I(I_i)^T r_T(T_j)$; the diagonal holds the matching pairs. Each row is a softmax classification over captions and each column over images.

After training, the two encoders classify images *zero-shot*, for example on MNIST, CIFAR-10, or ImageNet, with no labeled training images. Turn each class name into a caption such as "A photo of a {object}" (plane, car, dog, . . . , bird), encode the captions with the text encoder, encode the test image, and predict the class whose caption has the largest inner product with the image. Any label set works: given the labels Persian, Greek, and Turkish baklava, a CLIP demo assigned a photo 76%, 12%, and 12%.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 24 (Spring 2026).

Lecture 25: AI for Protein Engineering

This unit surveys two applications of machine learning in biology, structure prediction and design, and the statistical machinery they need. A protein is a string of length L over an alphabet of 20 amino acids that folds into a three-dimensional structure; green fluorescent protein, for instance, is a sequence of length 238. The three objects of interest are the sequence, the structure it folds into, and its function (Figure 39).

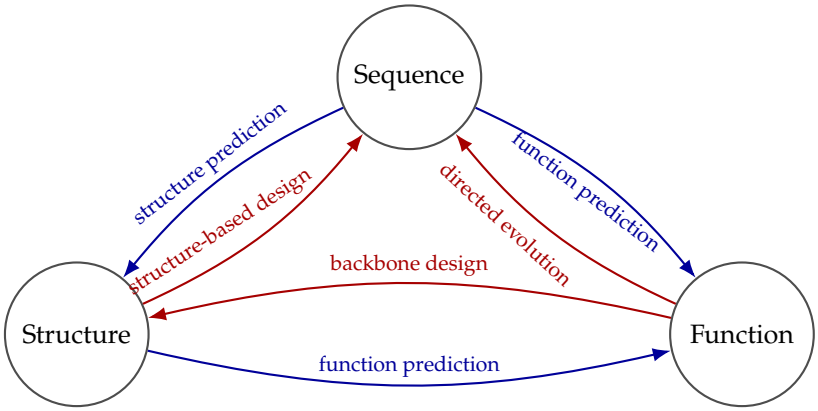


Figure 39: The sequence, structure and function triangle. Prediction tasks (blue) map sequence to structure and structure or sequence to function; design tasks (red) run the arrows backwards.

25.1 Structure prediction

The amino acid sequence determines the structure, a fact going back to Anfinsen (Nobel Prize 1972), so sequence to structure is a well posed prediction problem, and it has been studied for decades. Progress is measured by the average global distance test (GDT_TS) on the biennial CASP benchmark, where a score above 90 is considered roughly equivalent to an experimentally determined structure. From 2006 to 2016 the state of the art sat near 30 to 40; AlphaFold reached about 60 in 2018 and AlphaFold2 close to 90 in 2020, the first time the best method was deep learning based.

The key prior insight AlphaFold2 exploits is coevolution: amino acids in direct physical contact tend to covary across related proteins, since a mutation that enlarges one residue is more likely to be tolerated if a contacting residue shrinks to make space. Hence a multiple sequence alignment (MSA) of homologous sequences carries information about which positions touch in space.

AlphaFold2 is an "almost end-to-end" network. From the input sequence it builds an MSA representation by genetic database search and a pair representation from residue pairings and structural templates; these two representations are refined jointly by the Evoformer (48 blocks), and a structure module (8 blocks) using a rotation-equivariant attention architecture emits 3D atom

coordinates together with per-residue confidence. The whole pipeline is recycled three times. The output can still violate physics, so the predicted coordinates are refined by a classical energy-based relaxation, that is, coordinate-restrained gradient descent under a force field.

Three observations about the achievement: the problem was long studied, well defined, with clear data, benchmarks and a clear criterion for improvement; the structural data it trained on cost, conservatively, about US$20 billion to produce (Burley et al., 2023); and the method leaned heavily on prior work in the field (template-based modeling, coevolution modeling, contact prediction, energy functions).

25.2 Design and why structure prediction does not solve it

Design is the reverse direction, and it is hard for a basic reason: the design space has size $\sim 20^L$ and is highly rugged, and it is discrete, so there are no gradients to follow. For scale, 20^L passes the number of grains of sand on earth ($\sim 10^{18}$) and the number of atoms in the universe ($\sim 10^{80}$) at very short lengths. Two existing strategies navigate it: physics-based energy functions that model structure and binding without data (Rosetta, roughly 1997 to 2023, recognized by the 2024 Nobel Prize), and laboratory directed evolution, which iterates selection on the property of interest (roughly 1993 to the present, 2018 Nobel Prize). Machine-learning-augmented design, using generative models and function and structure prediction, dates from about 2018. Nature itself solved the problem a third way, by evolution over millions of years.

AlphaFold2/3 did not solve design. One rarely knows which structure is needed; even given one, the inverse map, structure to sequence, is needed (decent machine learning solutions for it exist); and the bottleneck is predicting which proteins have the desired function, usually extrapolatively, far from the training data.

25.3 Trends: everything is density estimation

Five current directions, each of which is a familiar statistical object in disguise.

1. *Representation learning*: unsupervised or self-supervised learning on large databases of millions of natural proteins (with Transformers), or on single families (Potts models, VAEs). This is approximate density estimation of $p_\theta(\text{sequence})$.
2. *Conditional generative models for sequences*, either structure-conditioned ("inverse folding") or conditioned on a control tag such as the protein family. This is conditional density estimation and sampling, $\text{seq} \sim p_\theta(\text{seq} \mid C)$, for instance with an autoregressive Transformer or a diffusion or flow model.
3. *Conditional generative models for structure*, sampling $\text{backbone} \sim p_\theta(\text{backbone} \mid F)$, paired with inverse folding to recover a sequence. Such a model is only as good as the function predictor $p(F \mid \text{backbone})$ it is conditioned on.

4. *Function prediction* from sequence with few or no labels (zero-shot and few-shot), leveraging evolutionary information or large unsupervised models, that is, $p_\theta(\text{function} \mid \text{sequence})$ and $p_\theta(\text{sequence} \mid \text{MSA})$.
5. *Structure prediction in the gaps left by AlphaFold*: orphan proteins with few homologs, dynamics and conformational distributions, and binding to other molecules.

25.4 In design we are the adversary

Optimizing an input to maximize a trained network's output is exactly the construction of an adversarial example: a stop sign plus imperceptible noise is classified as a yield sign, and an image optimized to maximize a "banana" score becomes abstract art rather than a banana. In design the same optimization yields sequences that the model scores highly and that do not fold at all, so naive design is pathology finding rather than engineering.

This motivates the design-specific research problems: the tension between using the trained model to extrapolate and knowing that it is not trustworthy over most of protein space (a question related to causality); estimating EPISTEMIC uncertainty, about the model itself, rather than only the ALEATORIC uncertainty of the measurement; building suitable inductive biases into the networks; and designing *distributions* over sequences rather than individual sequences.

25.5 Conditioning a generative model for design

The central technical question is how to sample $x \sim p(x \mid y)$ given an unconditional generative model $p(x)$ over sequences (such as ESM3 or ProteinMPNN), a property y of interest (an enzyme classification number, solubility, expression), and either labeled data $\{(x_i, y_i)\}_i$ or a predictive model $p(y \mid x)$. There are three statistically correct options:

1. *bake it in*: train a conditional model $p_\theta(x \mid y)$ from scratch;
2. *update it*: start from the unconditional model and update it using calls to the predictive model or data (CbAS, DPO);
3. *guide it*: freeze the unconditional model and steer it at generation time, as diffusion and flow models allow.

For any plug-and-play strategy, the only correct operation is Bayes' rule,

$$p(x \mid y) = \frac{p(y \mid x)\, p(x)}{p(y)},$$

combining the property predictor $p(y \mid x)$ with the unconditional model $p(x)$. The obstacle is the normalizing constant $p(y)$.

Diffusion, flow matching and score models estimate $\nabla_x \log p_\theta(x)$ rather than $p_\theta(x)$, and this is exactly what kills the constant: at noise level t,

$$\nabla_x \log p_t(x \mid y) = \nabla_x \log \frac{p_t(x)\, p_t(y \mid x)}{p_t(y)} = \underbrace{\nabla_x \log p_t(x)}_{\text{unconditional model}} + \underbrace{\nabla_x \log p_t(y \mid x)}_{\text{guidance}},$$

since $\nabla_x \log p_t(y) = 0$. In practice the guidance term carries a strength γ:

$$\nabla_{x_t} \log p^{(\gamma)}(x_t \mid y) = \nabla_{x_t} \log p(x_t) + \gamma\, \nabla_{x_t} \log p(y \mid x_t).$$

25.6 Guidance on discrete state spaces

Sequences, graphs and text are discrete, so $\nabla_x p_\theta(x)$ does not exist. Two consequences: standard diffusion and score matching do not apply, and standard guidance $\nabla_x p_\phi(y \mid x)$ cannot be used either. Mitigations are to relax x into a real-valued space and snap back, to run diffusion or flow on the multinomial simplex, or to use continuous-time Markov chains (CTMCs). The CTMC route replaces the score by a transition rate matrix R_t,

$$p(x_{t+dt} = \tilde{x} \mid x_t = x) = \delta_{x,\tilde{x}} + R_t(x, \tilde{x})\, dt,$$

so a trajectory holds in a state and jumps at random times. CTMCs support not only diffusion but also guidance, on both diffusion and flow models, and Bayes' rule becomes easy again: the guided rates are

$$\log R_t^{(\gamma)}(x, \tilde{x} \mid y) = \log R_t(x, \tilde{x}) + \gamma \Big(\log p(y \mid \tilde{x}, t) - \log p(y \mid x, t) \Big),$$

the discrete analogue of adding $\gamma \nabla \log p(y \mid x)$: the ratio of predictor probabilities at the candidate and current states takes the place of the gradient. Guiding ProteinMPNN this way (ProteinGuide) shifts the distribution of measured log enrichment of generated variants far to the right of the unguided model's.

25.7 Designing a library instead of a sequence

A case study in designing distributions. The goal is an optimal starting LIBRARY, a distribution over sequences, for a downstream selection campaign; with a poor library most of its mass is wasted on sequences that fail immediately. The pipeline has four steps.

1. Fit a predictive model f of fitness from sequence and test it on held-out data.
2. Validate the predictions experimentally; measured and predicted quantities correlated at Pearson 0.99 ($p = 0.0007$) over five sequences spanning the predicted range.

3. Invert the predictive model to trace a diversity-fitness optimality curve, solving

$$\arg\max_{\phi} \; \mathbb{E}_{p_\phi(x)}\big[f(x)\big] + \lambda\, H[p_\phi],$$

   where H is the entropy of the designed distribution p_ϕ and $\lambda \geq 0$ trades expected fitness against diversity. Sweeping λ gives a frontier of libraries from high-fitness and narrow to diverse and weak.

4. Validate selected points of the frontier in the laboratory (correlation 0.96, $p = 0.001$), and check that the designed libraries retain far more distinct sequences after selection than the standard library, and that they also perform better on a downstream objective they were not designed for.

The entropy term is what makes this design of a distribution rather than of a point: maximizing predicted fitness alone collapses onto the predictor's pathologies.

25.8 Fitness landscapes and epistasis

A second case study concerns pairs of interacting proteins. In the experimental design, 5 positions of protein A and 6 positions of protein B are varied over the same 5 amino acids, so the library contains all $5^{6+5} = 5^{11} \approx 4.9 \times 10^7$ combinations, and it is subjected to five rounds of selection of increasing stringency.

The data are read counts per round, and the statistical model chains them: a distribution $p_0(s)$ over sequences at round 0 is transformed by selection into $p_1(s), \ldots, p_R(s)$, with observed counts $N_r(s)$ emitted at each round. Selection strength is governed by a fitness $f(x) = -$energy predicted by a neural network, which is fit to all rounds jointly. On held-out data the model predicted counts with Spearman correlation 0.962, Pearson 0.919 and MSE 0.367.

The fitted f is then analyzed rather than merely used.

Proposition 25.1 [Epistatic expansion]. *Any fitness function on sequences of length L over an alphabet of size A can be written exactly as a linear combination of epistatic terms up to order L,*

$$f(x) = a_1 x_1 + a_2 x_2 + a_{12} x_1 x_2 + \cdots,$$

and extracting all coefficients amounts to a spectral transformation of f. Naively this costs $O(A^{2L})$; it can be done in $O\big((2A-1)^L\big)$, and a partial transform up to order K costs only $O\Big(A^L + \binom{L}{K}(2A-1)^K\Big)$.

The terms with a single index are additive effects, and those with several indices are epistasis: interaction between positions, either within one protein (intramolecular) or across the two partners (intermolecular). Measured against the crystal structure, the strength of pairwise epistasis scales inversely with the Cα-Cα contact distance of the two positions, for intra-A, intra-B and cross-interface

pairs alike: strong interactions occur between residues that touch.

Finally, the geometry of the landscape distinguishes the natural interface from a synthetic one. Simulating about 4 million fitness-increasing evolutionary trajectories and defining the ACCESSIBILITY of a sequence pair as the fraction of trajectories that terminate there, the natural interface concentrates trajectories on one dominant optimum (accessibility 0.74) while the synthetic interface spreads them over many shallow optima (accessibility 0.13). The natural landscape is funneled toward its optimum; the synthetic one is not.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 25 (Spring 2026).

Lecture 27: LLMs, Agents, and Environments

This unit defines what an agent is, what an environment is, and how agents are evaluated, trained, and optimized.

27.1 From an LLM to an agent

An LLM is a box that takes tokens and produces tokens: given "I like to eat apples and" it returns "Bananas". Nothing more. It cannot run a web search, read a document, or read email; one performs inference with it by running it locally (vLLM, Ollama) or by calling an API. Four constructions are built on top of it, and only the last is an agent.

Tool use. Prompt the model with "what tool do you want to use next?", take the text it produces, for instance a shell command such as `curl -s "https://www.google.com/search?q=..."`, execute it on the command line, and feed the result back. This is an LLM with tool use. It is not an agent.

Retrieval augmented generation. For a query such as "what is Alex's email", first run a retrieval step: a keyword search in a knowledge base returns text that seems relevant, typically through a vector database (Pinecone, Qdrant, Weaviate, or even Redis or Elastic). Then construct a prompt of the form "here is the query, here is some text that may help, please answer the query" and call the LLM. This is retrieval augmented generation (RAG). It is not an agent either: it is a hard-coded workflow consisting of a retrieval tool call followed by an LLM call.

Workflow. Chain several LLM calls with code between them: clean up the user's question with Python, build a prompt that extracts the user's intent, call the LLM, construct a new prompt from its output, and let a second LLM call decide which of two branches of Python code to execute. Control flow is now decided by an LLM, yet this is still a workflow, since the graph of steps is fixed in advance.

Agent. In a reason and act (ReAct) loop (Figure 40) the program gives the current state of the world to the LLM and asks it to think; the LLM produces thoughts, is asked what it wants to do next, and emits a command; the command is executed, the state of the world is updated, and the loop repeats. The LLM now has *agency*: it decides what to do and for how many steps. An agent is an LLM in a loop, using tools, deciding what to do next.

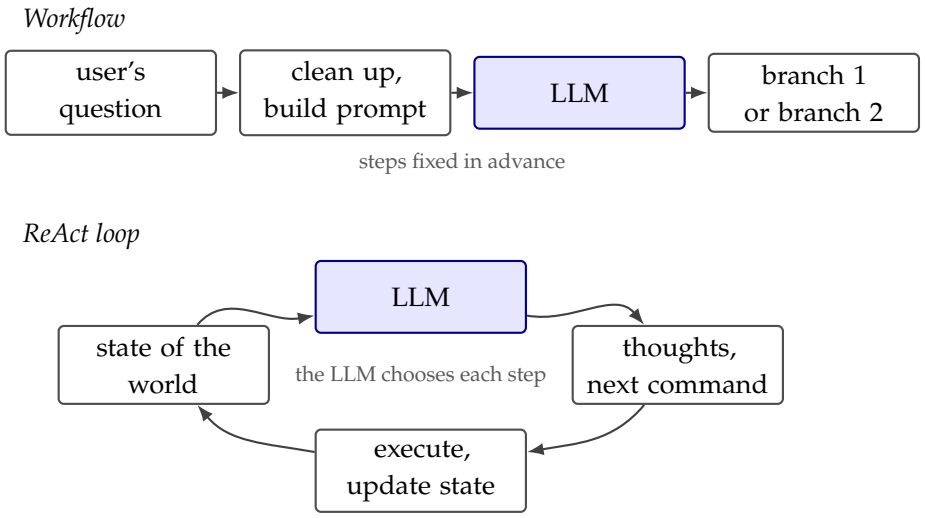


Figure 40: A fixed workflow (top) against the ReAct loop of an agent (bottom). In the workflow the sequence of steps is written in advance, even when an LLM call selects a branch; in the loop the LLM chooses each next command and the number of iterations.

27.2 How agents are built, and the long-horizon problem

Practice evolved in stages: one big prompt instructing the LLM; then, when the prompt grew too long, a split into several prompts with different roles (a MULTI-AGENT SYSTEM is a multi-prompt system); then frameworks for directed graphs of prompts (LangChain, Microsoft AutoGen). The main obstacle is the LONG-HORIZON PROBLEM: agents built by prompting become unstable after three to five steps.

The field as a whole moved through four phases: LLMs as embeddings for representation learning, pretrained and then fine-tuned per task (BERT and the like, 2018 to 2020); LLMs as assistants, pretrained and then post-trained with RLHF or DPO (ChatGPT, 2022 to 2024); LLMs with tools, going from one LLM call to many (LangChain, AutoGen, RAG, 2024 to 2025); and autonomous agents (from 2026). The current frontier shows a data spend shift: heavy investment in creating environments and tasks rather than static datasets, with deep-research agents and command-line coding agents as the successful instances.

27.3 Evaluation: from questions to environments

Evaluation changed with the object being evaluated, from asking what AI knows to asking what it can do.

LM evaluations	Agent evaluations
Data are questions and answers	Data are environments
Compare inputs and outputs	Compare actions
Well-defined criteria	More ambiguity in success
Single turn	Interactive, multi-turn

TERMINAL-BENCH is a concrete instance: an open-source framework (Harbor) for building agent evaluations and environments for the command line, together with terminal-bench-core, a benchmark of 80 and growing hand-crafted expert-level tasks.

Definition 27.1 [ENVIRONMENT]. An ENVIRONMENT is a container with three parts: a TASK description (for example, "my Python installation is broken, I cannot install packages with pip"), the ENVIRONMENT itself, a Dockerfile that builds the machine and puts it in the required state, and VERIFIERS, tests that check whether the task has been completed.

The agent runs inside that container (Figure 41). The HARNESS, now usually called the agent, is a program that runs the LLM calls and handles memory management, context, and tool calling; the underlying model is called over an API. Any command-line agent can be dropped into exactly the same environment, which is what makes the comparison fair.

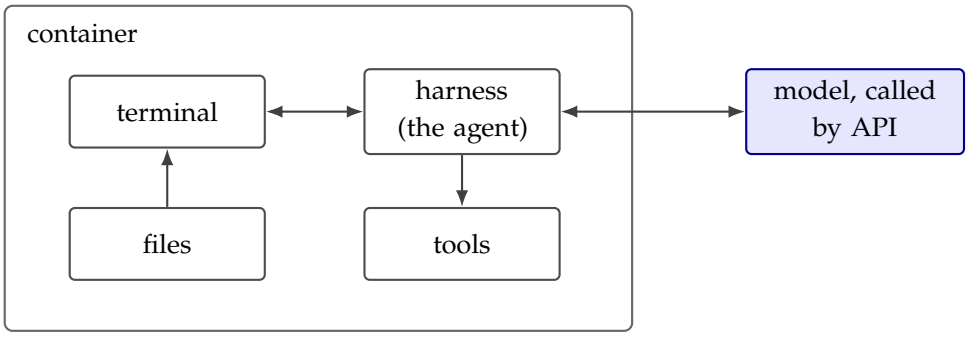


Figure 41: An environment: a container holding the terminal, the files, the tools, and the harness that runs the loop, with the model called from outside. Verifiers (not shown) run after the episode to decide success.

Each of the 80 tasks comes with a verified ground-truth solution, test cases, and its own containerized environment. Examples include compiling exotic code from source, scientific computing workflows, data science and DevOps tasks, and data processing pipelines: transforming a CSV file with pandas, recovering as many rows as possible from a corrupted SQLite database into a JSON file with a prescribed format, or installing and running a legacy operating system in a virtual machine. Adaptors let other benchmarks run in the same framework. The benchmark is hard: the best models resolve fewer than half of the tasks.

A practical lesson from building such agents: agents are extremely good at the file system and the command line. They search with superhuman `grep`, and the file system is hierarchical, composes

well with Unix commands, and carries permissions, so it can serve as the agent's long-term memory and let it control its own context. In experiments, dumping a workspace to a folder of JSON files and letting a command-line agent answer questions by searching it works better than a dedicated tool server. The architectural guidance is therefore to rely on the file system and the command line as much as possible, and to give the agent some agency, rather than to expose a hundred tools.

27.4 Using environments to improve an agent

Given an environment, there are three ways to improve the agent.

Supervised fine-tuning (SFT). Ask a teacher model to solve the task and record its trajectory, then update the weights of the trained agent to increase the probability of that trajectory. This is like reading solved homework problems.

Reinforcement learning (RL). No teacher: ask the student to solve the task itself and record the trajectory, then update the weights to increase the probability of high-reward trajectories and decrease that of low-reward ones. This is like trying to solve the problems yourself. RLVR (RL with verifiable rewards) is the case where the correct answer to each exercise is written down, so success can be checked automatically; an environment with many tests acts as the autograder. Test coverage and robustness to reward hacking are the critical research problems there. GRPO runs the same task as a group of, say, eight rollouts, and increases the probability of the good trajectories within the group while decreasing that of the bad ones, using the group as its own baseline.

GEPA. No weights at all: ask the student to solve the task, show the trajectory and its reward to a reflection model, and let that model propose how to update the prompt. This is valuable precisely because it improves agents built on closed-weight models, where the weights are not available to update.

How complex do environments need to be? The METR measurements of the time horizon of software engineering tasks that a model can complete 80% of the time have been doubling roughly every seven months, consistently over six years: about 30 minutes at the end of 2025, and on that trend 100 minutes in 2026, 5 hours in 2027, 17 hours in 2028, and 60 hours, that is 2.4 days, in 2029. Environments must grow to match, which is the key challenge: long-horizon verifiable tasks, such as translating a C compiler to Rust, or training a transformer to add numbers with as few weights as possible.

27.5 GEPA

GEPA (genetic-Pareto) is a prompt optimizer that uses natural language reflection to learn rules by trial and error, and maintains a population of prompts. The contrast with RL is the feedback signal: RL has the agent try to solve tasks and gives back a single number, the reward, whereas GEPA gives the whole trajectory to another LLM and asks it to evolve the prompt. It optimizes agents without

needing their weights.

Algorithm 27.1: GEPA

Input: Training set of tasks, starting prompt, evaluation function.
Output: Optimized prompt.

```
Split the training set into a dev set and a validation set
Initialize a pool of candidate prompts with the starting prompt
while budget remains do
    choose a candidate from the Pareto front of validation scores
    Run the system on a minibatch of dev examples, recording intermediate feedback
    Call an LM to propose alternatives from the scores and feedback, by mutating one
      candidate or crossing over two
    Score the alternatives on the validation set and update the pool and its Pareto front
return the candidate with the best average validation score
```

The Pareto front compares candidates across individual validation items, preserving specialists even when their average score is mediocre. This diversity supports sample-efficient search; in the reported comparison, GEPA outperforms GRPO with fewer rollouts.

Evolved prompts add task-specific strategies and worked examples. For multi-hop retrieval, they target connected concepts missing from the first-hop results instead of simply paraphrasing the question.

Since GEPA is a text evolution engine, with each node of its search tree holding a set of texts evolved for a target metric, the texts need not be prompts:

- *prompt learning* that generalizes to unseen data (HotpotQA, AIME);
- *inference-time search*, that is, test-time scaling (NPU and CUDA kernel generation);
- *agent architecture discovery*, where a node holds the agent's code as text, so GEPA proposes new agent workflows: from a chain-of-thought ARC-AGI agent scoring 44% it discovered a six-step agent scoring 49.5% held out, and a two-step agent taking a MATH subset of 500 questions from 56 to 76.2;
- *adversarial prompt search*: to find prompts that confuse the agent, invert the reward. On AIME a plain instruction prompt scored 76% with one model, and the inverted search padded those same instructions with irrelevant trivia, dropping it to 10%.

Verifiable environments combine a container, task, and verifier. Evaluations guide prompt optimization, improvements to tools and memory, and SFT/RL post-training. The challenge is to build realistic tasks with long horizons.

Source. Jennifer Listgarten and Alex Dimakis, *CS 189/289A: Introduction to Machine Learning*, Lecture 27 (Spring 2026).